

Mohit Arora

The Art of Hardware Architecture

Design Methods and Techniques
for Digital Circuits

 Springer

The Art of Hardware Architecture

Mohit Arora

The Art of Hardware Architecture

Design Methods and Techniques
for Digital Circuits



Springer

Mohit Arora
Freescale Semiconductor
Faridabad
India
mohit.arora@mc.com

ISBN 978-1-4614-0396-8 e-ISBN 978-1-4614-0397-5
DOI 10.1007/978-1-4614-0397-5
Springer New York Dordrecht Heidelberg London

Library of Congress Control Number: 2011934353

© Springer Science+Business Media, LLC 2012

All rights reserved. This work may not be translated or copied in whole or in part without the written permission of the publisher (Springer Science+Business Media, LLC, 233 Spring Street, New York, NY 10013, USA), except for brief excerpts in connection with reviews or scholarly analysis. Use in connection with any form of information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed is forbidden.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

Printed on acid-free paper

Springer is part of Springer Science+Business Media (www.springer.com)

*This book is dedicated to my wife Pooja
and my daughter Prisha.*

Preface

I started my Industry career in year 2000 in the field of Chip Design. My work involved lot of research that gave me opportunity to write technical papers, participate in various conferences and share practical experiences. During this journey I got lot of positive feedback on my publications. Readers have often asked me forcing me to think if I should write a book compiling all the practical experiences. The book's aim is to highlight all the complex issues, tasks and skills that must be mastered by an IP designer to design an optimized and robust digital circuit to solve a problem. The techniques and methodologies prescribed in the book, if properly employed, can significantly reduce the time it takes to convert initial ideas and concepts into right-first-time silicon.

The book is intended for a wide audience. Though it may be used in an undergraduate or graduate course, book is mainly intended for those in semiconductor industries who are directly involved with chip design and requires deeper understand of the subject.

This book is distinguished from others by its primary focus on real problems rather than theoretical concepts with its emphasis on design techniques across various aspects of chip-design.

The book covers aspects of chip design in a consistent way, starting with basic concepts in Chap. 1 and gradually increasing the depth to reach advanced concepts, such as EMC design techniques or sophisticated low power techniques like DVFS (Dynamic Voltage and Frequency scaling).

Chapter 1 covers “*metastability*” to help user understand more clearly the issues related to metastability, how it can be quantified and necessary techniques to minimize its effort.

Chapter 2 covers general set of recommendations around “*clocks and resets*” intended for use by designers while designing a block or Intellectual Property (IP). The guidelines are independent of any CAD tool or silicon process and are applicable to any ASIC designs.

Chapter 3 goes beyond synchronous clock designs and covers asynchronous clocks or “*handling multiple clocks*” in design, problems faced and solutions in order to get a robust designs that works on multiple clocks.

Chapter 4 covers all about “*Clock Dividers*” that a typical SoC may require generating number of phase related clocks. Apart from synchronous division where required clocks are generated by dividing the master clock by a power of two, chapter also covers odd division (Divide by 3, 5 etc.) as well as non-integer dividers (Divide by 1.5, 2.5 etc.).

Chapter 5 covers all about “*Low Power Design techniques*”. In recent times, power consumption has become a significant design constraint with shrinking technology as well as to meet power targets for energy efficient applications. This Chapter describes various design methodologies and techniques at various levels of design abstraction to reduce dynamic and as well as static power consumption.

Chapter 6 covers the concept of “*Pipelining*”, the way it applies to processor design to increase the throughput in terms of calculations per clock cycle. The chapter extends the scope of pipelining beyond microprocessor to cover typical circuits so as to increase performance.

Chapter 7 covers “*Endianess issues*” in design that may include several third-party IPs with different Endianess and the way it can be handled in the design in an optimal way.

Chapter 8 covers several hardware as well as software “*Debouncing Techniques*” to eliminate unwanted noise or glitch in the circuit caused by an external input (usually some kind of switch).

Chapter 9 covers deep details on EMC/EMI issues, the way it applies to digital circuits and design guidelines that can be followed at various level of abstraction for “*better EMC performance*”.

Theoretical part has been intentionally kept to the minimum that is essentially required to understand the subject. The guidelines explained across various chapters are independent of any CAD tool or silicon process and are applicable to any ASIC designs and can help designers to plan and to execute a successful System on Chip (SoC) with a well-structured and synthesizable RTL code.

There are few chapters that include Verilog Hardware Description Language (HDL) code for beginner’s who are learning digital circuits, however the same can be skipped by more advanced engineers who are already exposed to the fundamentals.

Some of the more advanced chapters like “*Design Guidelines for EMC performance*” have been thoroughly researched and have taken months to write in a way to make topics more relevant to digital designers.

Every possible effort was made to make the book self-contained. Any feedback/comments are welcome on this aspect or any other related aspects. Comments can be sent to me at the following mails: mohit.arora@me.com or mohit.arora@freescale.com.

Acknowledgements

The original idea behind “*The Art of Hardware Architecture*” was to link my years of experience as a design architect with the extensive research I have conducted. However, achieving the final shape of this book would not have been possible without many contributions.

My sweet wife *Pooja* was so patient with my late nights, and I want to thank her for her faithful support & encouragement in writing this book. Most of the work occurred on weekends, nights, while on vacation, and other times inconvenient to my family. I like to thank my parents for allowing me to follow my ambitions throughout my childhood.

I am grateful to *Prashant Bhargava*, my colleague from Freescale Semiconductor for his careful reading of drafts of this book and his constructive suggestions for improvement based on his years of experience. He also helped in book editing, formatting as well contribution to some of the sections in Chap. 7.

Special thanks to *Rakesh Pandey* and *Sudhi Ranjan Proch* from Freescale Semiconductor for their early help and feedback on some of the sections of the book.

I thank *Charles Glaser*, Senior Editor from Springer for providing me opportunity to publish with Springer and entire publication team at *Springer* who helped turn this book into excellent professional manuscript.

Mohit Arora

Contents

1	The World of Metastability	1
1.1	Introduction.....	1
1.2	Theory of Metastability.....	1
1.3	Metastability Window.....	3
1.4	Calculating MTBF	3
1.5	Avoiding Metastability.....	5
1.5.1	Using a Multi-stage Synchronizer	6
1.5.2	Multi-stage Synchronizer Using Clock Boost Circuitry.....	6
1.6	Metastability Test Circuitry	7
1.7	Types of Synchronizers.....	8
1.8	Metastability/General Recommendations.....	10
2	Clocks and Resets	11
2.1	Introduction.....	11
2.2	Synchronous Designs.....	11
2.2.1	Avoid Using Ripple Counters	12
2.2.2	Gated Clocks.....	12
2.2.3	Double-Edged or Mixed Edge Clocking.....	13
2.2.4	Flip Flops Driving Asynchronous Reset of Another Flop.....	13
2.3	Recommended Design Techniques	14
2.3.1	Avoid Combinational Loops in Design.....	14
2.3.2	Avoid Delay Chains in Digital Logic.....	16
2.3.3	Avoid Using Asynchronous Based Pulse Generator.....	16
2.3.4	Avoid Using Latches.....	17
2.3.5	Avoid Using Double-Edged Clocking	20
2.4	Clocking Schemes.....	22
2.4.1	Internally Generated Clocks	22
2.4.2	Divided Clocks.....	24
2.4.3	Ripple Counters	25
2.4.4	Multiplexed Clocks.....	25
2.4.5	Synchronous Clock Enables and Gated Clocks.....	26

2.5	Clock Gating Methodology.....	28
2.5.1	Latch Free Clock Gating Circuit.....	28
2.5.2	Latch Based Clock Gating Circuit.....	30
2.5.3	Gating Signals.....	32
2.5.4	Data Path Re-ordering to Reduce Switching Propagation.....	32
2.6	Reset Design Strategy.....	32
2.6.1	Design with Synchronous Reset.....	33
2.6.2	Design with Asynchronous Reset.....	36
2.6.3	Flip Flops with Asynchronous Reset and Asynchronous Set.....	38
2.6.4	Asynchronous Reset Removal Problem.....	40
2.6.5	Reset Synchronizer.....	40
2.6.6	Reset Glitch Filtering.....	42
2.7	Controlling Clock Skew.....	42
2.7.1	Short Path Problem.....	43
2.7.2	Clock Skew and Short Path Analysis.....	44
2.7.3	Minimizing Clock Skew.....	46
	References.....	49
3	Handling Multiple Clocks.....	51
3.1	Introduction.....	51
3.2	Multiple Clock Domains.....	51
3.3	Problems with Multiple Clock Domains Design.....	51
3.3.1	Setup Time and Hold Time Violation.....	53
3.3.2	Metastability.....	53
3.4	Design Tips for Efficient Handling of a Design with Multiple Clocks.....	54
3.4.1	Clock Nomenclature.....	54
3.4.2	Design Partitioning.....	55
3.4.3	Clock Domain Crossing.....	55
3.5	Synchronous Clock Domain Crossing.....	58
3.5.1	Clocks with the Same Frequency and Zero Phase Difference.....	59
3.5.2	Clocks with the Same Frequency and Constant Phase Difference.....	59
3.5.3	Clocks with the Different Frequency and Variable Phase Difference.....	60
3.6	Handshake Signaling Method.....	64
3.6.1	Requirements for Handshake Signaling.....	65
3.6.2	Disadvantages of Handshake Signaling.....	66
3.7	Data Transfer Using Synchronous FIFO.....	66
3.7.1	Synchronous FIFO Architecture.....	67

3.7.2	Working of Synchronous FIFO.....	68
3.8	Asynchronous FIFO (or Dual Clock FIFO).....	69
3.8.1	Avoid Using Binary Counters for the Pointer Implementation	70
3.8.2	Use Gray Coding Instead of Binary for the Counters.....	71
3.8.3	Gray Code Implementation of FIFO Pointers.....	74
3.8.4	FIFO Full and FIFO Empty Generation.....	79
3.8.5	Dual Clock FIFO Design	82
	References.....	86
4	Clock Dividers	87
4.1	Introduction.....	87
4.2	Synchronous Divide by Integer Value	87
4.3	Odd Integer Division with 50% Duty Cycle.....	88
4.4	Non-integer Division (with a Non 50% Duty Cycle)	90
4.4.1	Divide by 1.5 with Non 50% Duty Cycle	90
4.4.2	Counter Implementation for Divide by 4.5 (Non 50% Duty Cycle)	91
4.5	Alternate Approach for Divide by N.....	92
4.5.1	LUT Implementation for Divide by 1.5	93
	Reference	93
5	Low Power Design.....	95
5.1	Introduction.....	95
5.2	Sources of Power Consumption.....	95
5.3	Power Reduction at Different Levels of Design Abstraction.....	96
5.4	System Level Power Reduction	98
5.4.1	System on Chip (SoC) Approach.....	98
5.4.2	Hardware/Software Partitioning	98
5.4.3	Low Power Software.....	101
5.4.4	Choice of Processor	102
5.5	Architecture Level Power Reduction	102
5.5.1	Advanced Clock Gating	103
5.5.2	Dynamic Voltage and Frequency Scaling (DVFS)	104
5.5.3	Cache Based Architecture.....	105
5.5.4	Log FFT Architecture	106
5.5.5	Asynchronous (Clockless) Design.....	106
5.5.6	Power Gating.....	108
5.5.7	Multi-threshold Voltage	111
5.5.8	Multi-supply Voltage.....	112
5.5.9	Gate Memory Power	112
5.6	Register Transfer Level (RTL) Power Reduction	113
5.6.1	State Machine Encoding and Decomposition.....	113
5.6.2	Binary Number Representation.....	114

5.6.3	Basic Gated Clock.....	115
5.6.4	One Hot Encoded Multiplexer.....	117
5.6.5	Removing Redundant Transactions	118
5.6.6	Resource Sharing	119
5.6.7	Using Ripple Counters for Low Power.....	121
5.6.8	Bus Inversion	124
5.6.9	High Activity Nets	124
5.6.10	Enabling-Disabling Logic Clouds.....	125
5.7	Transistor Level Power Reduction.....	126
5.7.1	Technology Level.....	126
5.7.2	Layout Optimization	127
5.7.3	Substrate Biasing	127
5.7.4	Reduce Oxide Thickness.....	127
5.7.5	Multi-oxide Devices.....	127
5.7.6	Minimizing Capacitance by Custom Design	128
	References.....	128
6	The Art of Pipelining	129
6.1	Introduction.....	129
6.2	Factors Affecting the Maximum Frequency of Clock	129
6.2.1	Clock Skew	131
6.2.2	Clock Jitter.....	131
6.3	Pipelining.....	133
6.4	Pipelining Explained – A Real Life Example.....	136
6.5	Performance Increase from Pipelining	137
6.6	Implementation of DLX Instruction	140
6.7	Effect of Pipelining on Throughput	144
6.8	Pipelining Principles.....	145
6.9	Pipelining Hazards.....	146
6.9.1	Structural Hazards	146
6.9.2	Data Hazards.....	147
6.9.3	Control Hazards	150
6.9.4	Other Hazards	151
6.10	Pipelining in ADC – An Example	152
	References.....	153
7	Handling Endianness.....	155
7.1	Introduction.....	155
7.2	Definition	155
7.3	Little-Endian or Big-Endian: Which Is better?.....	157
7.4	Issues Dealing with Endianness Mismatch.....	158
7.5	Accessing 32 Bit Memory	160
7.6	Dealing with Endianness Mismatch	161

7.6.1	Preserve Data Integrity (Data Invariance).....	161
7.6.2	Address Invariance.....	163
7.6.3	Software Byte Swapping.....	166
7.7	Endian Neutral code.....	167
7.8	Endian-Neutral Coding Guidelines	167
	References.....	168
8	Debouncing Techniques	169
8.1	Introduction.....	169
8.2	Behavior of a Switch.....	170
8.3	Switch Types.....	171
8.4	De-bouncing Techniques	172
8.4.1	RC De-bouncer	172
8.4.2	Hardware De-bouncers	176
8.4.3	Software De-bouncing	177
8.4.4	De-bouncing Guidelines	179
8.4.5	De-bouncing on Multiple Inputs.....	180
8.5	Existing Solutions	181
9	Design Guidelines for EMC Performance	183
9.1	Introduction.....	183
9.2	Definition	183
9.3	EMI Theory and Relationship with Current and Frequency.....	185
9.4	EMI Regulations, Standards and Certification.....	186
9.5	Factors Affecting IC Immunity Performance	187
9.5.1	Microcontroller as Noise Source	187
9.5.2	Other Factors Affecting EMC.....	188
9.5.3	Noise Carriers	189
9.6	Techniques to Reduce EMC/EMI	189
9.6.1	System Level Techniques.....	190
9.6.2	Board Level Techniques.....	192
9.6.3	Microcontroller Level Techniques	201
9.6.4	Software Level Techniques	205
9.6.5	Other Techniques	212
9.7	Summary	213
	References.....	214
	References.....	215
	Index.....	219

Chapter 1

The World of Metastability

1.1 Introduction

In a synchronous system, the data always has a fixed relationship with respect to the clock. When that relationship obeys the setup and hold requirements for the device, the output goes to a valid state within its specified propagation delay time. In synchronous systems, the input signals always meet the flip-flop's timing requirements; therefore, metastability does not occur. However, in an asynchronous system, the relationship between data and clock is not fixed; therefore, occasional violations of setup and hold times can occur. When this happens, the output may go to an intermediate level between its two valid states and remain there for an indefinite amount of time before resolving itself or it may simply be delayed before making a normal transition.

This Chapter is intended to help understand more clearly the issues relating to the metastability, how it is quantified, and how to minimize its effort.

1.2 Theory of Metastability

Metastability arises as a result of violation of setup and hold times of a flip flop. Every flip-flop that is used in any design has a specified setup and hold time, or the time during which the data input is not legally permitted to change before and after a rising clock edge, respectively. If the signal does change during this time window, the output will be unknown or “metastable”. This propagation of unwanted state is called Metastability. As a result the output of a flip-flop can produce a glitch or remain temporarily in metastable state, thus taking longer to return to stable state.

When a flip-flop is in a metastable state, the output hovers at a voltage level between high and low, causing the output transition to be delayed beyond the specified clock-to-output delay (t_{co}). The additional time beyond t_{co} that a metastable output takes to resolve to a stable state is called the settling time (t_{MET}). This has

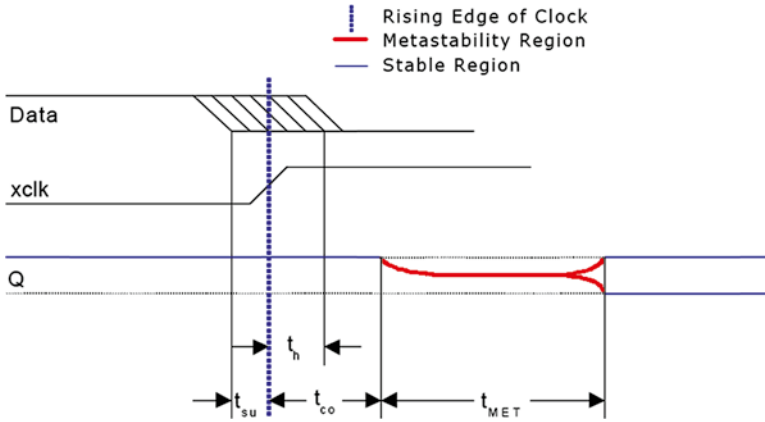


Fig. 1.1 Metastability timing parameters

been shown in Fig. 1.1. Not every transition that violates the setup or hold times results in a metastable output. The likelihood that a flip-flop enters a metastable state and the time required to return to stable state depends on the process technology used to manufacture the device and on the ambient conditions. Generally, flip-flops will return to a stable state within one or two clock cycles.

The operation of a Flip-Flop is analogous to a ball rolling over a frictionless hill, as shown in Fig. 1.2. Each side of the hill represents a stable state (i.e. high or low) and the top represents a metastable state. Suppose the ball is in a stable state (i.e. either 1 or 0) and a push (state transition) is given to the ball that is sufficient (no setup or hold time violations) enough to make the ball cross over to the other stable state, the ball crosses to the other stable state within the specified time.

However, if the push is less (i.e. violation of setup and hold time), the ball shall travel to the top of the hill (i.e. output metastable), stay there for some time and return to either stable state (i.e. output becomes stable eventually). It may also happen that the ball may rise partially and come back (i.e. output may produce some glitches). Either condition increases the delay from clock transition to a stable output.

Thus, in simple words, when a signal is changing in one clock domain (*src_data_out*) and is sampled in another clock domain (*dest_data_in*), then this causes the output to become metastable. This is known as Synchronization Failure (Shown in Fig. 1.3).

1.3 Metastability Window

Metastability Window is defined as the specific length of time, during which both the data and clock should not change. If both signals do occur, the output may go metastable. As shown in Fig. 1.4, the combination of the Setup and Hold time determine the width of the Metastability window.

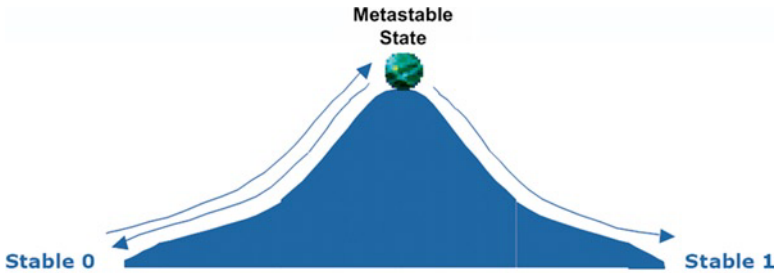


Fig. 1.2 Metastable behavior of flip flop

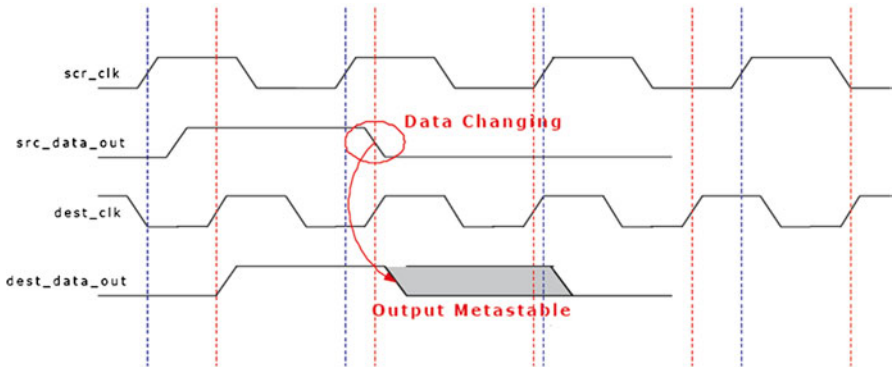


Fig. 1.3 Metastability in flip flop

The larger the window, the greater the chance the device will go Metastable. In most cases, newer logic families have smaller Metastability windows which reduce the chance of the device going Metastable.

1.4 Calculating MTBF

Mean (Average) Time Between Failures or MTBF of a system, is the reciprocal of the failure rate in the special case when the failure rate is constant. This gives the information on how often a particular Flip Flop will fail.

For a single-stage synchronizer with a given clock frequency and an asynchronous data edge that has a uniform probability density within the clock period, the rate of generation of metastable events can be calculated by taking the ratio of the setup and hold time window to the time between clock edges and multiplying by the data edge frequency.

Fig. 1.4 Metastability window

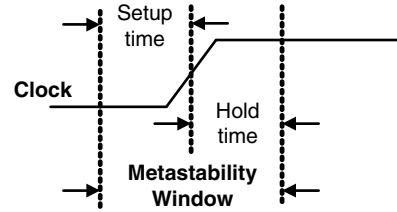
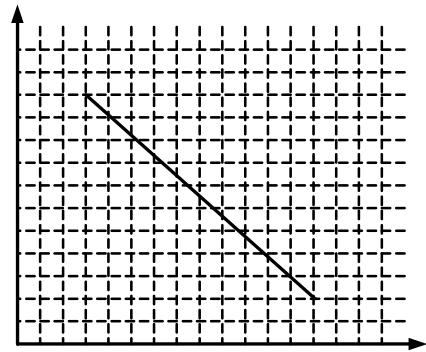


Fig. 1.5 Failure rate vs. time (log scale)



$$\frac{1}{\text{FailureRate}} = \text{MTBF}_1 = \frac{e(t_r / \tau)}{W \times f_c \times f_d} \quad (1.1)$$

where

t_r = resolve time allowed in excess of the normal propagation delay time of the device

τ = metastability (resolving) time constant for a flip-flop

W = Metastability Window

f_c = Clock frequency

f_d = Asynchronous data edge frequency

The constants W and τ are related to electrical characteristics of the device and may vary according to the process technology node. Therefore, different devices manufactured with the same process have similar values for W and τ .

If the failure rate of a device is measured at different resolve times and plotted, the result is an exponentially decaying curve. When plotted on a semi logarithmic scale, as shown in Fig. 1.5, this becomes a straight line the slope of which is equal to τ ; therefore, two data points on the line are sufficient to calculate the value of τ using Eq. 1.2.

$$\tau = \frac{t_{r2} - t_{r1}}{\ln(N1 / N2)} \quad (1.2)$$

where

t_{r1} = resolve time 1

t_{r2} = resolve time 2

$N1$ = numbers of failures at t_{r1}

$N2$ = numbers of failures at t_{r2}

Based on Eqs. 1.1 and 1.2, MTBF for a two-stage synchronizer can be calculated by Eq. 1.3 below

$$MTBF_2 = \frac{e(t_{r1} / \tau)}{W \times f_c \times f_d} \times e(t_{r2} / \tau) \quad (1.3)$$

where

t_{r1} = resolve time allowed for the first stage of synchronizer

t_{r2} = resolve time in access of normal propagation delay

The first term in the Eq. 1.3 calculates the MTBF of the first stage of the synchronizer, which in effect becomes the generation rate of the metastable events for the next stage. The second term then calculates the probability that the metastable event will be resolved based on the value of t_{r2} , the resolve time allowed external to the synchronizer. The product of the two terms gives the overall MTBF for the two-stage synchronizer.

In quantitative terms, using Eq. 1.3 above, if the Mean Time Between Failure (MTBF) of a particular Flip-Flop in the context of a given clock rate and the input transition rate is 40 s then MTBF of two such flip-flops used to synchronize the input would be $40 \times 40 = 26.6$ min.

1.5 Avoiding Metastability

As shown in Sect. 1.2, metastability occurs whenever setup or hold time is violated. So signals may violate the timing requirements under the following conditions:

- When the input signal is an asynchronous signal.
- When the clock skew/slew (rise/fall times) is higher than the tolerable limit.
- When signals cross the domains working at two different frequencies or with same frequency but different phase and skew.
- When the combinational delay is such that the Flip Flop data input changes in the Metastability Window.

Metastability can cause excessive propagation delays and subsequent system failures. All Flip Flops and latches exhibit metastability. The problem cannot be eliminated. But it is possible to make metastability less likely to occur.

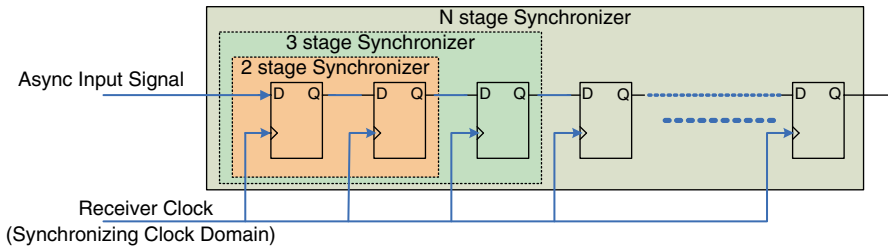


Fig. 1.6 N-stage synchronizer

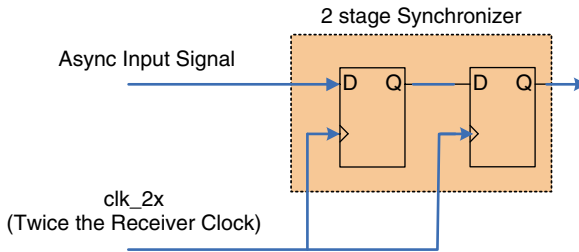


Fig. 1.7 Multi-stage synchronizer with clock boost circuitry

In the simplest case, designers can avoid metastability by making sure the clock period is long enough to allow for the resolution of quasi-stable states and for the delay of whatever logic may be in the path to the next flip-flop. This approach, while simple, is rarely practical given the performance requirements of most modern designs. The other approach is to use Synchronizers.

1.5.1 Using a Multi-stage Synchronizer

The most common way to avoid metastability is to add one or more synchronizing flip-flops at the signals that move from one clock domain to the other as shown in Fig. 1.6. This approach allows for an entire clock period (except for the setup time of the second flip-flop) for metastable events in the first synchronizing flip-flop to resolve itself. This does however; increase the latency in the synchronous logic's observation of input.

1.5.2 Multi-stage Synchronizer Using Clock Boost Circuitry

One limitation of the multiple-stage synchronizer is that it takes longer for the system to respond to an asynchronous input. A solution to this problem is to use the output of a clock doubler to clock the two synchronizing flip-flops. Altera's FPGA exhibit this technique as Clock Boost or Clock Doubler (Fig. 1.7).

This approach allows the system to respond to an asynchronous input within one system clock cycle, while still improving MTBF. Although the Clock Boost clock could decrease the MTBF, this effect is more than offset by the two synchronizing flip-flops.

Neither of these approaches can guarantee that metastability cannot pass through the synchronizer; they simply reduce the probability of occurrence of metastability.

1.6 Metastability Test Circuitry

Whenever a flip-flop samples an asynchronous input, a small probability exists that the flip-flop output will exhibit an unpredictable delay. This happens not only when the input transition violates setup and hold time specifications, but also when the transition actually occurs within a small timing window during which the flip-flop accepts the new input. Under these circumstances, the flip-flop can enter a metastable state.

The test circuit described in Fig. 1.8 is used to determine metastability characteristics of a Flip-Flop. Figure 1.8 shows an Asynchronous Input “*async_In*” to the Flip Flop “FF_A” triggered on positive edge of Clock “*clk*”. As shown both the Flops “FF_B” and “FF_C” are triggered on negative edge of the clock in order to capture the metastable event on “FF_A”.

As complementary signals are passed on the input of Flip Flops “FF_B” and “FF_C”, the output of the XNOR gate goes HIGH whenever a metastable event occurs on “FF_A”. This conditions is captured on output of Flip Flop “FF_D” indicating that a metastable event has been detected.

The timing for all the nodes in this test circuit is shown in Fig. 1.9.

Because the resolving flip-flops (“FF_B” and “FF_C”) are clocked by the falling clock edge, the required settling time can be controlled by changing the clock high time (Δt). The settling time t_{MET} can be determined with the equation below

$$t_{MET} = \Delta t - t_{ACN} \quad (1.4)$$

where t_{ACN} is the minimum clock period which is equal to t_{CQ} (clock to Output delay of FF_A) + setup time t_{su} of the resolving Flip Flop (FF_B or FF_C).

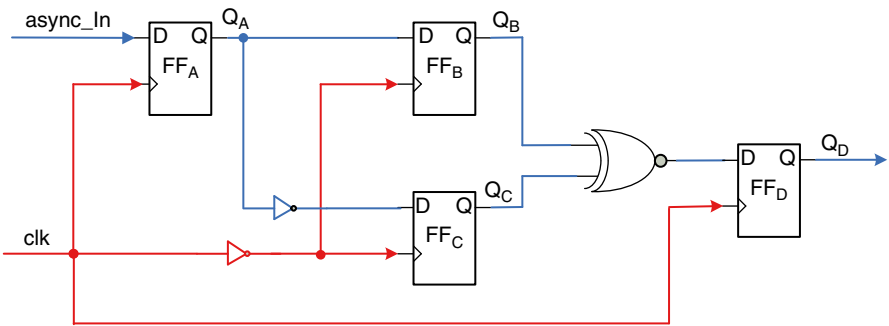


Fig. 1.8 Metastability test circuitry

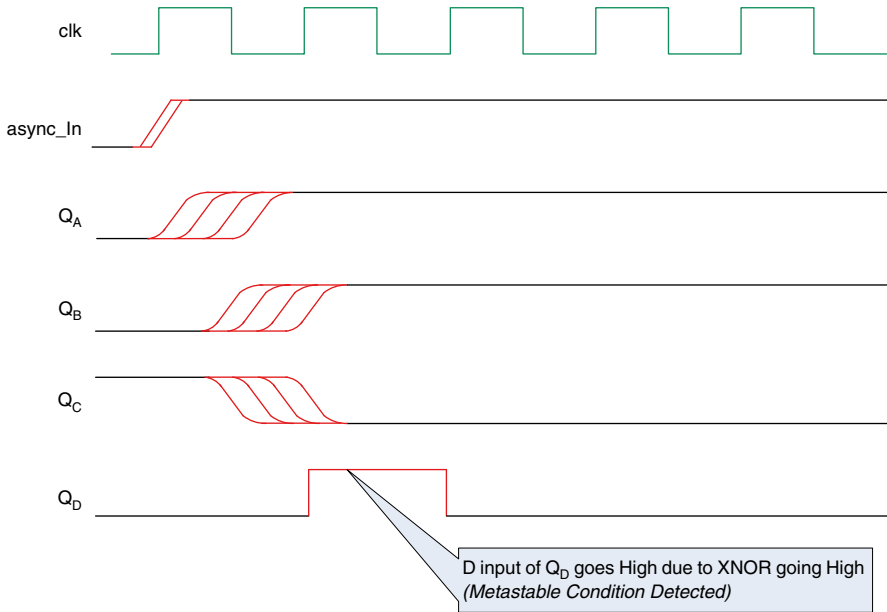


Fig. 1.9 Timings for metastability test circuitry

One of the methods to reduce the resolve time or settling time is to add jitter to the data centered across setup/hold.

1.7 Types of Synchronizers

As per Eq. 1.1, Mean Time Between Failures (MTBF) of a circuit with an asynchronous input is exponentially related to the time available for recovery from a metastable condition. Use synchronizers to create a time buffer for recovering from a metastable event.

Note that an asynchronous signal should never be synchronized by more than one synchronizer. (To do so would risk having the outputs of multiple synchronizers produce different signals). This section shows two-synchronizer schemes A and B.

Scheme A is normal scheme and works best when the width of the Asynchronous Input signal is greater than the clock period (Fig. 1.10).

Note that even if the asynchronous input reaches a stable condition outside the setup interval, it will still be clocked through with a latency of two clock cycles otherwise FF1 may enter metastability.

If metastability is resolved in less than one clock cycle, FF2 will have a stable input else deeper cascading is required as shown in Fig. 1.6.

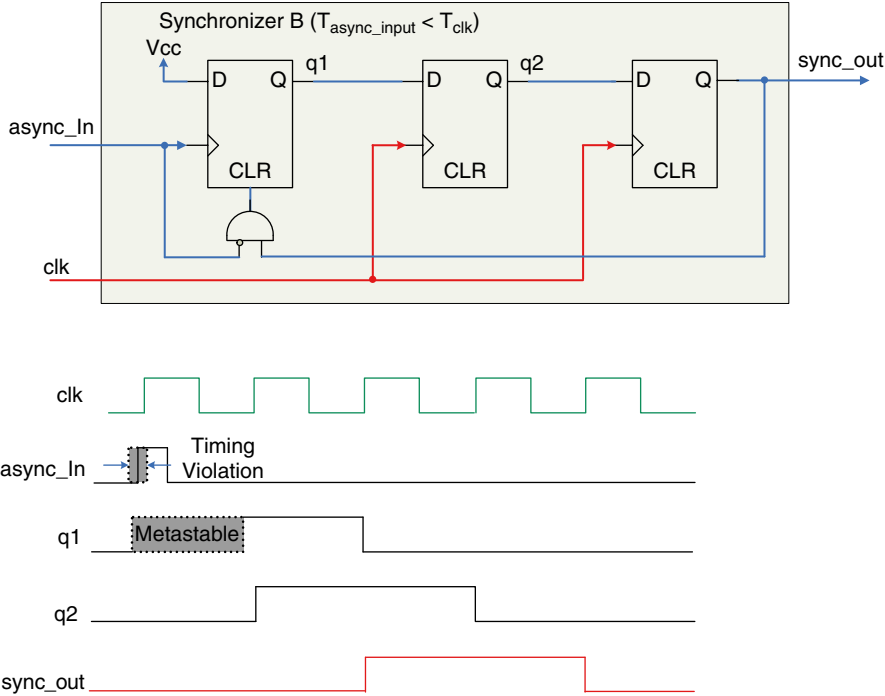


Fig. 1.10 Synchronizer Scheme A for a two-stage synchronizer

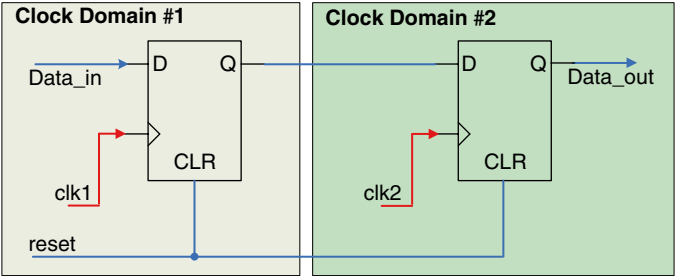


Fig. 1.11 Synchronizer Scheme B for a two-stage synchronizer

However, Scheme A does not work where the width of the Asynchronous Input is smaller than the clock period. In this case, scheme B works the best as shown in Fig. 1.11.

Note that incase of Synchronizer scheme B, D input of the first FF (Flip Flop) is connected to V_{CC} , while asynchronous input clocks the FF. The other two FF in the stage are clocked directly by system clock or clk . A short pulse will drive $q1$ High that will propagate to $sync_out$ after two “ clk ” edges.

So this defines our rule of thumb that is summarized as follows

1. Use synchronizers when a signal must cross a boundary between clock domains.
2. If $\text{Clk1} < \text{Clk2}$ use Synchronizer scheme A at the input of the clock domain 2 (as shown in Fig. 1.11) , otherwise use synchronizer scheme B.

1.8 Metastability/General Recommendations

Metastability cannot be avoided at the boundary between two systems that are asynchronous with respect to each other. However the probability that metastable states are encountered can be significantly reduced by the following recommendations:

- (a) Use Synchronizers.
- (b) Use Faster Flip Flops (narrower metastable window T_w).
- (c) Use metastable hardened Flip Flops (designed for very high bandwidth and reduced sampling times that are optimized for clock domain input circuitry).
- (d) Cascade flip-flops as Synchronizers (two or more) as shown in Fig. 1.6. A chain of N Flip Flops has a probability of P^N where P is the chance of metastable failure for one flip flop.
- (e) Reduce Sampling rate.
- (f) Avoid input signals with low dV/dt .

Chapter 2

Clocks and Resets

2.1 Introduction

The cost of designing ASICs is increasing every year. In addition to the non-recurring engineering (NRE) and mask costs, development costs are increasing due to ASIC design complexity. To overcome the risk of re-spins, high NRE costs, and to reduce time-to-market delays, it has become very important to design the first time working silicon.

This chapter constitutes a general set of recommendations intended for use by designers while designing a block or an IP (Intellectual Property). The guidelines are independent of any CAD tool or silicon process and are applicable to any ASIC designs and can help designers to plan and to execute a successful System on Chip (SoC) with a well-structured and synthesizable RTL code.

The current paradigm shift towards system level integration (SLI), incorporating multiple complex functional blocks and a variety of memories on a single circuit, gives rise to a new set of design requirements at integration level. The recommendations are principally aimed at the design of the blocks and memory interfaces which are to be integrated into the system-on-chip. However, the guidelines given here are fully consistent with the requirements of system level integration and will significantly ease the integration effort, and ensure that the individual blocks are easily reusable in other systems.

These guidelines can form as a basis of checklist that can be used as a signoff for each design prior to submission for fabrication.

2.2 Synchronous Designs

Synchronous designs are characterized by a single master clock and a single master set/reset driving all sequential elements in the design.

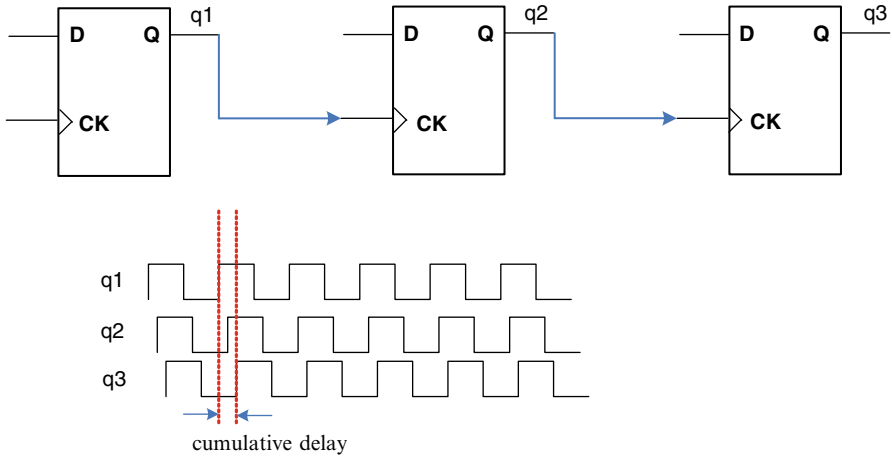


Fig. 2.1 Flip flop driving the clock input of another flip flop (ripple counter)

Experience has shown that the safest methodology for time domain control of an ASIC is synchronous design. Some of the problems with the circuits not being synchronous have been shown in this section.

2.2.1 Avoid Using Ripple Counters

Flip Flops driving the clock input of other flip flops is somewhat problematic. The clock input of the second flip-flop is skewed by the *clock-to-q* delay of the first flip-flop, and is not activated on every clock edge. This cumulative effect with more than two Flip Flops connected in a similar manner forms a Ripple counter as shown in Fig. 2.1. Note the cumulative delay gets added on with more number of flip flops and hence the same is not recommended. More details on the ripple counter are given in Sect. 5.6.7.

2.2.2 Gated Clocks

Gating in a clock line causes clock skew and can introduce spikes which trigger the flip-flop. This is particularly the case when there is a multiplexer in the clock line as shown in Fig. 2.2.

Simulating a gated clock design might work perfectly fine but the problem arises when such a design is synthesized.

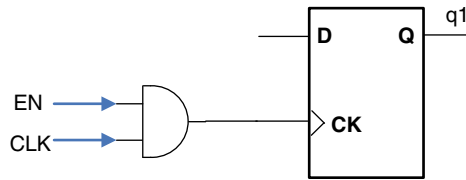


Fig. 2.2 Gated clock line

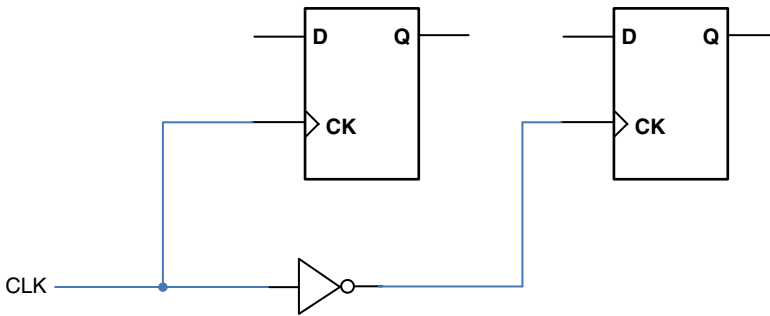


Fig. 2.3 Double-edged clocking

2.2.3 Double-Edged or Mixed Edge Clocking

As shown in Fig. 2.3, the two flip-flops are clocked on opposite edges of the clock signal. This makes synchronous resetting and test methodologies such as scan-path insertion difficult, and causes difficulties in determining critical signal paths.

2.2.4 Flip Flops Driving Asynchronous Reset of Another Flop

In Fig. 2.4, the second flip-flop can change state at a time other than the active clock edge, violating the principle of synchronous design. In addition, this circuit contains a potential race condition between the clock and reset of the second flip-flop.

The subsequent sections show the methods to avoid the above non-recommended circuits.

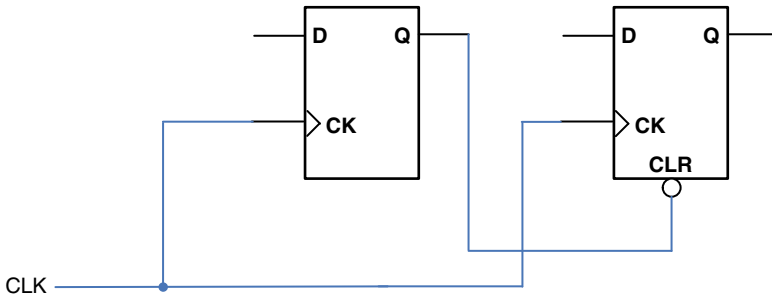


Fig. 2.4 Flip flop driving asynchronous reset of another flop

2.3 Recommended Design Techniques

When designing with HDL code, it is important to understand how a synthesis tool interprets different HDL coding styles and the results to expect. It is very important to think in terms of hardware as a particular design style (or rather coding style) can affect gate count and timing performance. This section discusses some of the basic techniques to ensure optimal synthesis results while avoiding several causes of unreliability and instability.

2.3.1 Avoid Combinational Loops in Design

Combinational loops are among the most common causes of instability and unreliability in digital designs. In a synchronous design, all feedback loops should include registers. Combinational loops violate synchronous design principles by establishing a direct feedback with no registers.

In terms of HDL language, combinational loops occur when the generation of a signal depends on itself through several combinational *always*¹ blocks or when the left-hand side of an arithmetic expression also appears on the right-hand side. Combo loops are a hazard to a design and synthesis tools will always give errors when combo loops are encountered, as these are not synthesizable.

The generation of combo loops can be understood from the following bubble diagram in Fig. 2.5. Each bubble represents a combo *always* block and the arrow going into it represents the signal being used in that *always* block while an arrow going out from the bubble represents the output signal generated by that output block. It is evident that the generation of signal 'a' depends on itself through signal 'd', thereby generating a combinational loop.

¹For simplicity, any HDL languages that this book refers to takes Verilog as an example.

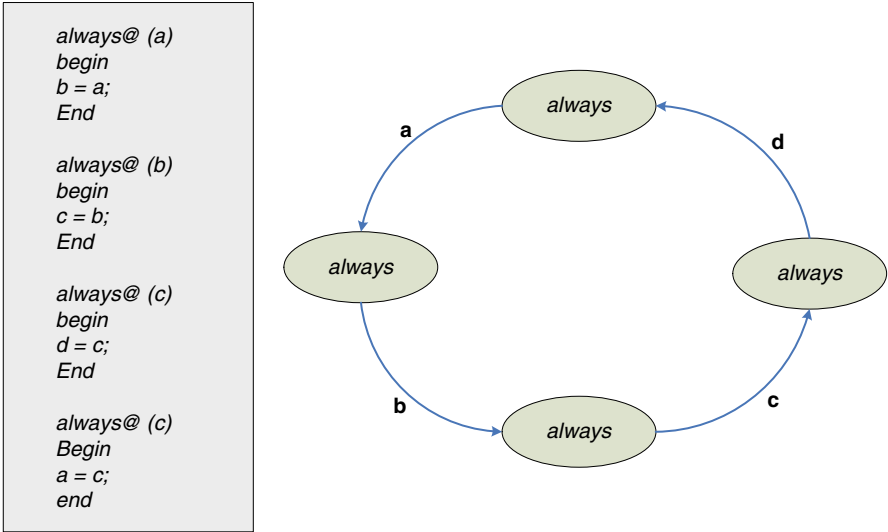


Fig. 2.5 Combinational loop example and bubble diagram

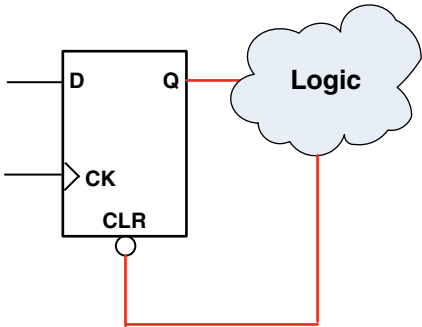


Fig. 2.6 Combinational loop through asynchronous control pins

The code and the bubble diagram are shown below [28]:

In order to remove combo loops, one must change the generation of one of the signals so the dependency of signals on each other is removed. Simple resolution to this problem is to introduce a Flip Flop or register in the combo loop to break this direct path.

Figure 2.6 shows another example where output of a register directly controls the asynchronous pin of the same register through combinational logic.

Combinational loops are inherently high-risk design structures. Combinational loop behavior generally depends on the relative propagation delays through the logic involved in the loop. Propagation delays can change based on various factors and the behavior of the loop may change. Combinational loops can cause endless

computation loops in many design tools. Most synthesis tools break open or disable combinatorial loop paths in order to proceed. The various tools used in the design flow may open a given loop a different manner, processing it in a way that may not be consistent with the original design intent.

2.3.2 Avoid Delay Chains in Digital Logic

Delay chains occur when two or more consecutive nodes with a single fan-in and a single fan-out are used to cause delay. Often inverters are chained together to add delay. Delay chains generally result from asynchronous design practices, and are sometimes used to resolve race conditions created by other combinational logic. In both FPGA and ASIC, delays can change with each place-and-route. Delay chains can cause various design problems, including an increase in a design's sensitivity to operating conditions, a decrease in a design's reliability, and difficulties when migrating to different device architecture. Avoid using delay chains in a design, rely on synchronous practices instead.

2.3.3 Avoid Using Asynchronous Based Pulse Generator

Often design requires generating a pulse based on some events. Designers sometimes use delay chains to generate either one pulse (pulse generators) or a series of pulses (multi-vibrators). There are two common methods for pulse generation; these techniques are purely asynchronous and should be avoided:

- A trigger signal feeds both inputs of a two-input AND or OR gate, but the design inverts or adds a delay chain to one of the inputs. The width of the pulse depends on the relative delays of the path that feeds the gate directly and the one that goes through the delay. This is the same mechanism responsible for the generation of glitches in combinational logic following a change of inputs. This technique artificially increases the width of the spike by using a delay chain.
- A register's output drives the same register's asynchronous reset signal through a delay chain. The register essentially resets itself asynchronously after a certain delay.

Asynchronously generated pulse widths often pose problem to the synthesis and place-and-route software. The actual pulse width can only be determined when routing and propagation delays are known, after placement and routing. So it is difficult to reliably determine the width of the pulse when creating HDL. The pulse may not be wide enough for the application in all PVT conditions, and the pulse width will change when migrating to a different technology node. In addition, static timing analysis cannot be used to verify the pulse width so verification is very difficult.

Multi-vibrators use the principle of the "glitch generator" to create pulses, in addition to a combinational loop that turns the circuit into an oscillator [25].

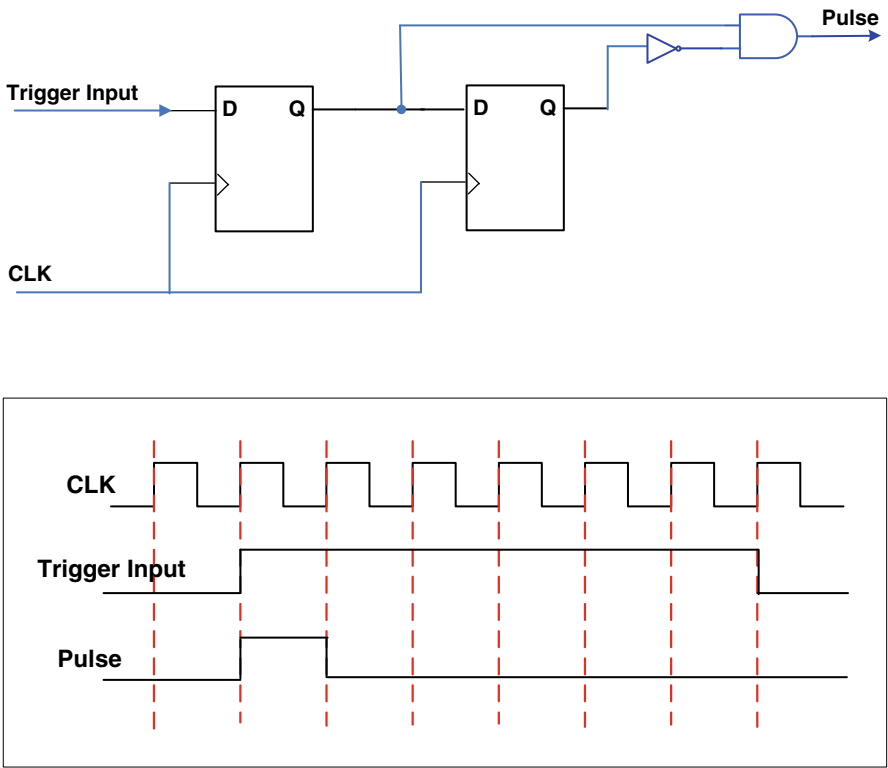


Fig. 2.7 Synchronous pulse generator circuit on start of trigger input

Structures that generate multiple pulses cause even more problems than pulse generators because of the number of pulses involved. In addition, when the structures generate multiples pulses, they also increase the frequency of the design.

A recommended Synchronous Pulse generator is shown in Fig. 2.7.

In the above synchronous pulse generator design, the pulse width is always equal to the clock period. This pulse generator is predictable, can be verified with timing analysis, and is easily migrated to other architectures and is technology independent.

Similar to Fig. 2.7, Fig. 2.8 shows the pulse generator at the end of trigger input.

2.3.4 Avoid Using Latches

In digital logic, latches hold the value of a signal until a new value is assigned. Latches should be avoided whereas possible in the design and flip-flops should be used instead.

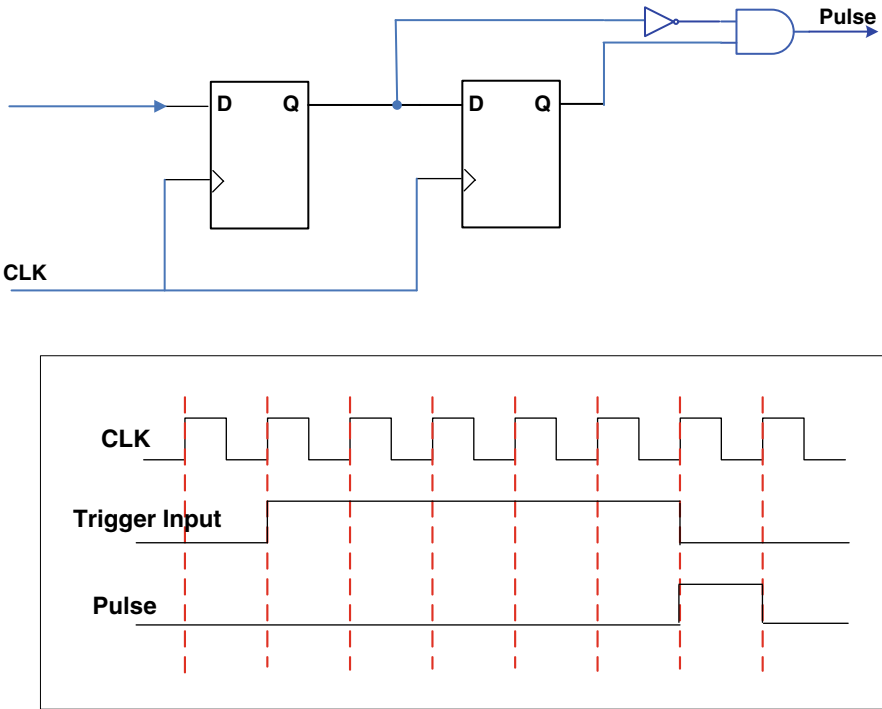


Fig. 2.8 Synchronous pulse generator circuit on end of trigger input

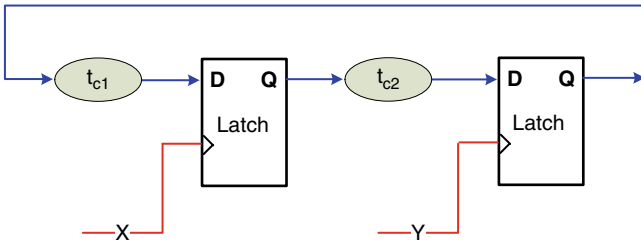


Fig. 2.9 Race conditions in latches

As shown in Fig. 2.9, if both the X and Y were to go high, and since these are level triggered, both the Latches would be enabled resulting in the circuit to oscillate.

Latches can cause various difficulties in the design. Although latches are memory elements like registers, they are fundamentally different. When a latch is in a feed-through mode, there is a direct path between the data input and the output. Glitches on the data input can pass to the output.

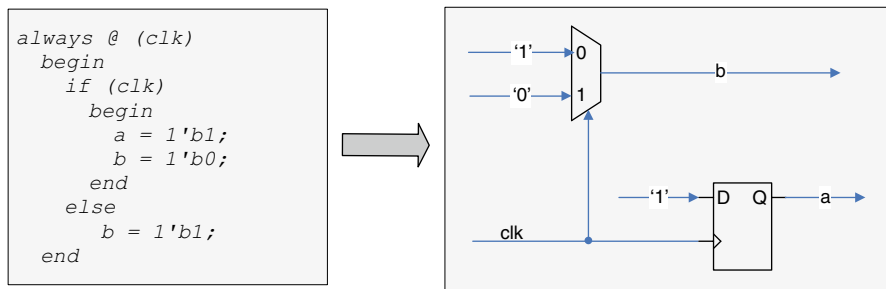


Fig. 2.10 Inferred latch due to incomplete 'if else' statement

Static timing analyzers typically make incorrect assumptions about latch transparency, and either find a false timing path through the input data pin or miss a critical path altogether. The timing for latches is also inherently ambiguous. When analyzing a design with a D latch, for example, the tool cannot determine whether you intended to transfer data to the output on the leading edge of the clock or on the trailing edge. In many cases, only the original designer knows the full intent of the design, which implies that another designer cannot easily migrate the same design or reuse the code.

Latches tend to make circuits less testable. Most design for test (DFT) and automatic test program generator (ATPG) tools do not handle latches very well.

Latches pose different challenge in FPGA designs as FPGA's are register-intensive; therefore, designing with latches uses more logic and leads to lower performance than designing with registers.

Synthesis tools occasionally infer a latch in a design when one is not intended. Inferred latches typically result from incomplete "if" or "case" statements. Omitting the final "else" clause in an "if" or "case" statement can also generate a latch. Figure 2.10 shows a similar example.

As shown in Fig. 2.10, 'b' will be synthesized as straight combinational logic while a latch will be inferred on signal 'a'.

A general rule for latch inferring is that if a variable is not assigned in all possible executions of an always statement (for example, when a variable is not assigned in all branches of an 'if' statement), then a latch is inferred.

Some FPGA architectures do not support latches. When such a design is synthesized, the synthesis tool creates a combinational feedback loop instead of a latch (as shown in Fig. 2.11).

Combinational feedback loops as shown above are capable of latching data but pose more problem than latches since they may violate setup, hold requirements which are difficult to be determined, whereas latches do not have any setup time, hold time violations since they are level triggered.

Note: The design should not contain any combinational feedback loops. They should be replaced by flip-flops or latches or be eliminated by fully enumerating RTL conditionals.

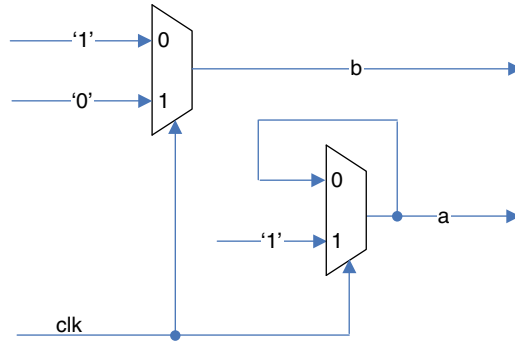


Fig. 2.11 Combinational loop implemented due to incomplete ‘if else’ statement

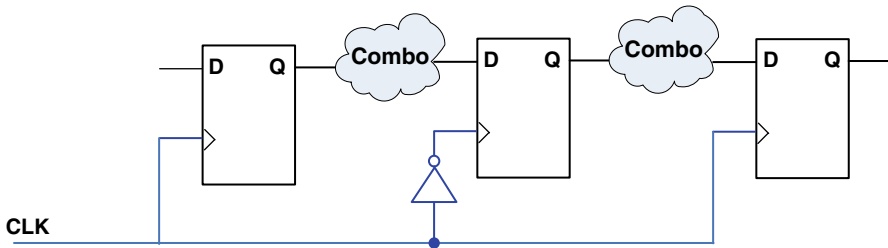


Fig. 2.12 Logic with double edged clocking

To conclude, this does not mean latches should never exist, we will see later how latches could be wonderful when it comes to cycle stealing or time borrowing to meet a critical path in a design.

2.3.5 Avoid Using Double-Edged Clocking

Double or Dual edged clocking is the method of data transfer on both the rising and falling edges of the clock, instead of just one or the other. The change allows for double the data throughput for a given clock speed.

Double edge output stage clocking is a useful way of increasing the maximum possible output speed from a design; however this violates the principle of Synchronous circuits and causes a number of problems.

Figure 2.12 shows a circuit triggered by both edges of clock.

Some of the problems encountered with Double Edged clocking are mentioned below:

- An asymmetrical clock duty cycle can cause setup and hold violations.
- It is difficult to determine critical signal paths.

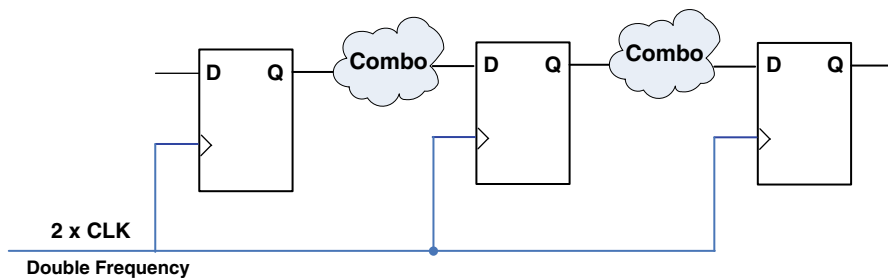


Fig. 2.13 Logic with single edged clocking

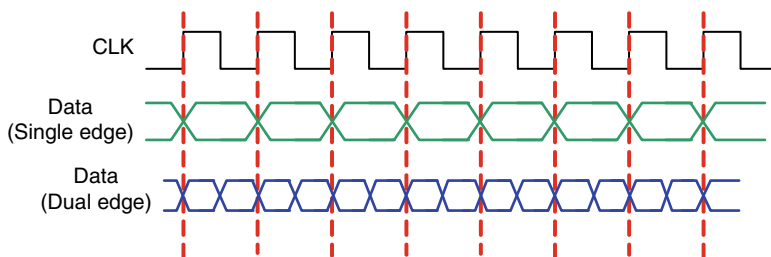


Fig. 2.14 Single/double edged data transfer

- Test methodologies such as scan-path insertion are difficult, as they rely on all flip-flops being activated on the same clock edge. If scan insertion is required in a circuit with double-edged clocking, multiplexers must be inserted in the clock lines to change to single-edged clocking in test mode.

Figure 2.13 shows the normal equivalent pipelined logic with single edge clocking. Note that this synchronous circuit requires a clock frequency that is double the one shown in Fig. 2.12.

Figure 2.14 shows the single transition and double transition clocked data transfer.

The green and blue signals represent data; the “hexagon” shapes are the traditional way of representing a signal that at any given time can be either a one or a zero.

In the circuit shown in Fig. 2.12, an asymmetrical clock duty cycle could cause setup and hold time violations, and a scan-path cannot easily be threaded through the flip-flops.

The above does not mean that circuits with dual edge clocking should never be used unless there is an intense desire for higher performance/speed that cannot be met with the equivalent synchronous circuits as the latter comes with an additional overhead of complexity in DFT and verification.

2.3.5.1 Advantages of Dual Edge Clocking

The one constant in the PC world is the desire for increased performance. This in turn means that most interfaces are, over time, modified to allow for faster clocking, which leads to improved throughput. Many newer technologies in the PC world have gone a step beyond just running the clock faster. They have also changed the overall signaling method of the interface or bus, so that data transfer occurs not once per clock cycle, but twice or more.

There are other advantages of circuit operating on dual edge rather than the same synchronous circuit being fed with double the clock frequency. Whatever extent possible, interface designers do regularly increase the speed of the system clock. However, as clock speeds get very high, problems are introduced on many interfaces. Most of these issues are related to the electrical characteristics of the signals themselves. Interference between signals increases with frequency and timing becomes more “tight”, increasing cost as the interface circuits must be made more precise to deal with the higher speeds.

The other advantage using double edged clocking is lower power consumptions as clock speeds are decreased by half and hence the system consumes less power than the equivalent synchronous circuits.

So to conclude system integrator should only use dual or double edged clocking unless the same desired performance cannot be met with the equivalent synchronous circuits.

2.4 Clocking Schemes

2.4.1 Internally Generated Clocks

A designer should avoid internally generated clocks, wherever possible, as they can cause functional and timing problems in the design, if not handled properly.

Clocks generated with combinational logic can introduce glitches that create functional problems and the delay due to the combinational logic can lead to timing problems. In a synchronous design, a glitch on the data inputs does not cause any issues and is automatically avoided as data is always captured on the edge of the clock and thus blocks the glitch. However, a glitch or a spike on the clock input (or an asynchronous input of a register) can have significant consequences.

Narrow glitches can violate the register’s minimum pulse width requirements. Setup and hold times may also be violated if the data input of the register is changing when a glitch reaches the clock input. Even if the design does not violate timing requirements, the register output can change value unexpectedly and cause functional hazards elsewhere in the design.

Figure 2.15 shows the effect of using a combinational logic to generate a clock on a synchronous counter. As shown in the timing diagram, due to the glitch on the clock edge, the counters increments twice in the clock cycle shown.

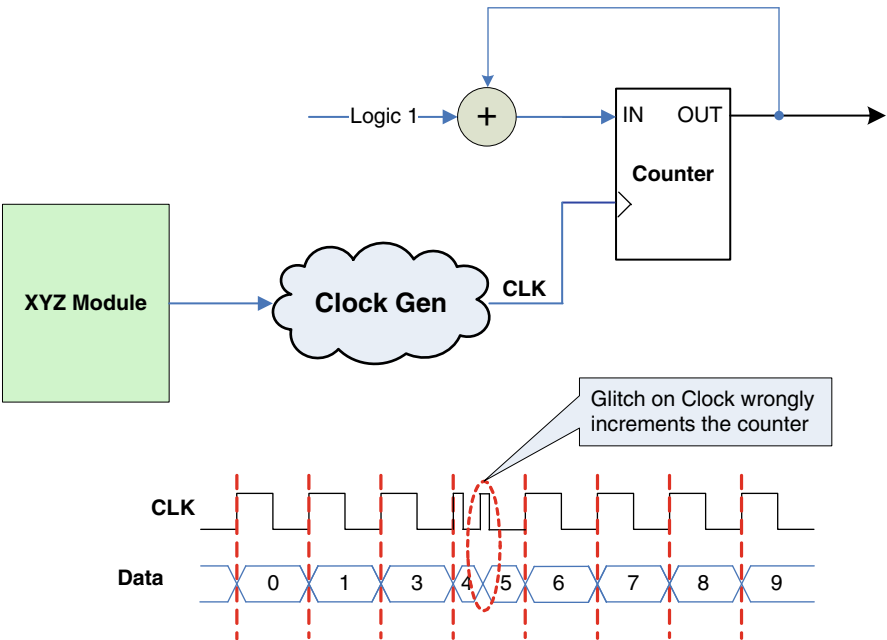


Fig. 2.15 Counter example for using combinational logic as a clock

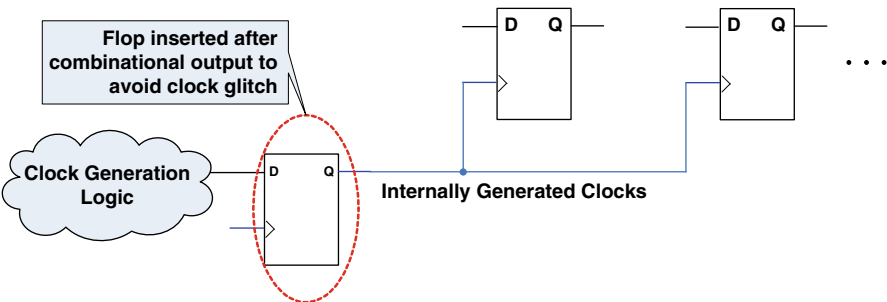


Fig. 2.16 Recommended clock generation technique

This extra counting may create functional issues in the design where instead of counting the desired count, counter counts an additional count due to the glitch on the clock.

Note: That for the sake of simplicity, it is assumed that the Counters Flops did not violate the setup/hold requirements on the data due to the glitch.

A simple guideline to the above problem is to always use a registered output of the combinational logic before using it as a clock signal. This registering ensures that the glitches generated by the combinational logic are blocked on the data input of the register (Fig. 2.16).

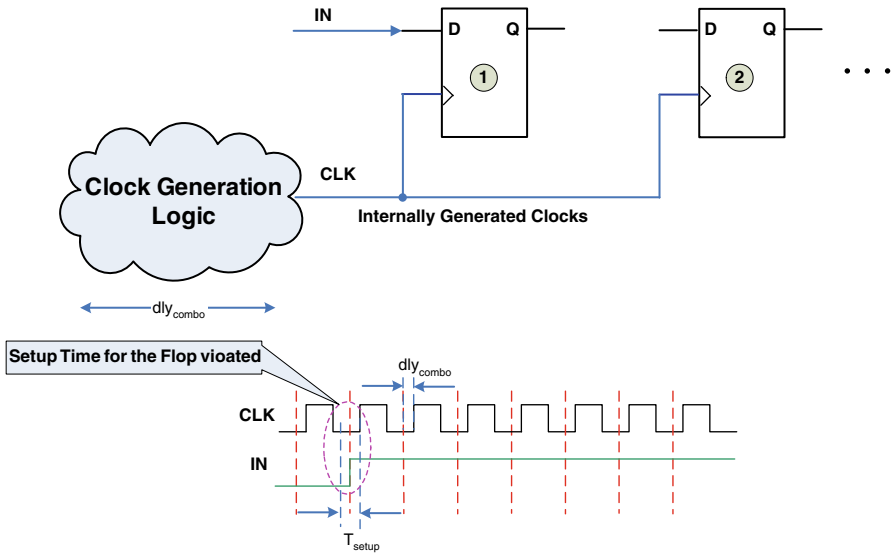


Fig. 2.17 Setup time violated due to skew of clock path

The combinational logic used to generate an internal clock also adds delays on the clock line. In some cases, logic delay on a clock line can result in a clock skew greater than the data path length between two registers. If the clock skew is greater than the data delay, the timing parameters of the register will be violated and the design will not function correctly.

Figure 2.17 shows a similar example where setup time on input “IN” is violated due to skew on the clock path.

Note: Data path delay is assumed to be zero for simplicity.

One solution to reduce the clock skew within the clock domain is by assigning the generated clock signal to one of the high-fanout and low-skew clock trees in the SoC. Using a low-skew clock tree can help reduce the overall clock skew for the signal.

2.4.2 Divided Clocks

Many designs require clocks created by dividing a master clock. Design should ensure that most of the clocks should come from the PLL. Using PLL circuitry will avoid many of the problems that can be introduced by asynchronous clock division logic. When using logic to divide a master clock, always use synchronous counters or state machines.

In addition, the design should ensure that registers always directly generate divided clock signals. Design should never decode the outputs of a counter or a state machine to generate clock signals; this type of implementation often causes glitches and spikes.

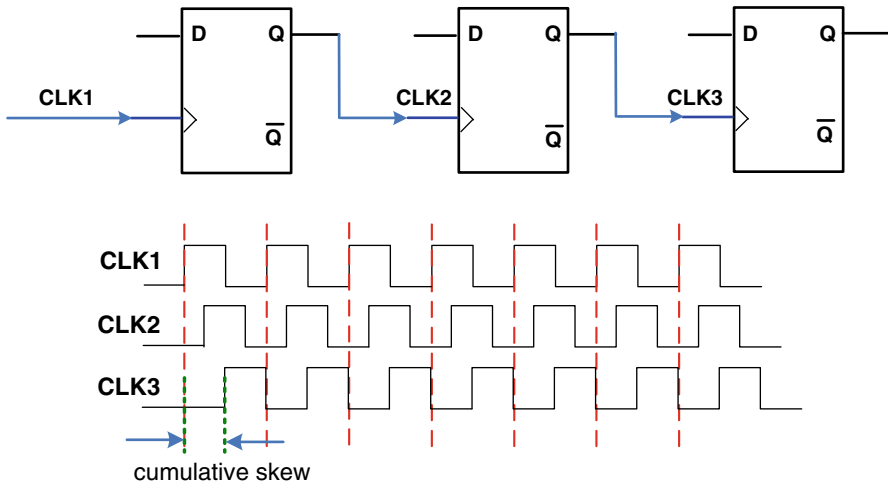


Fig. 2.18 Cascading effort in ripple counters

2.4.3 Ripple Counters

ASIC designers have often implemented ripple counters to divide clocks by a power of 2 because the counters use fewer gates than their synchronous counterparts. Ripple counters use cascaded registers, in which the output pin of each register feeds the clock pin of the register in the next stage (Fig. 2.18).

This cascading can cause problems because the counter creates a ripple clock at each stage. These ripple clocks pose another set of challenges for STA and synthesis tools. One should try to avoid these types of structures to ease verification effort.

Despite of all the challenges and problems with respect to using Ripple counters, these are quite handy in systems that eat power and can be good to reduce the peak power consumed by a logic or SoC.

Note: *Digital designers should consider using this technique in limited cases and under tight control.*

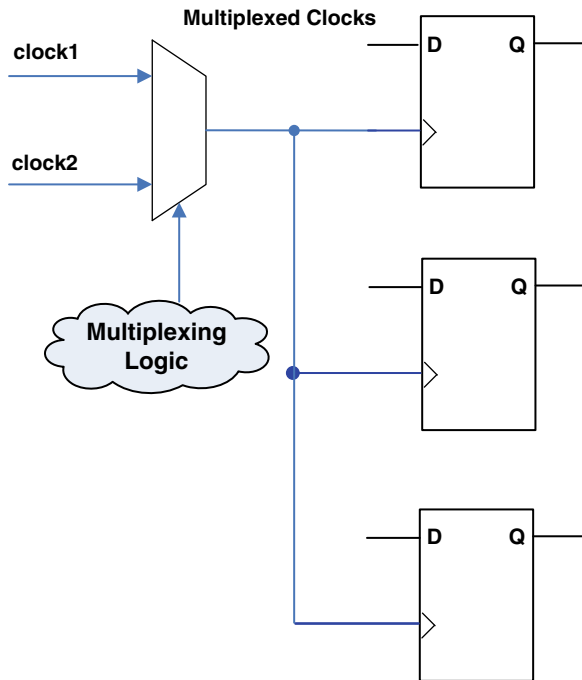
Refer Chap. 5 “Low power design” on more details analysis and techniques of using Ripple counters to save power consumption.

2.4.4 Multiplexed Clocks

Clock multiplexing can be used to operate the same logic function with different clock sources. Multiplexing logic of some kind selects a clock source as shown in Fig. 2.19.

For example, telecommunications applications that deal with multiple frequency standards often use multiplexed clocks.

Fig. 2.19 Multiplexing logic and clock sources



Adding multiplexing logic to the clock signal can lead to some of the problems discussed in the previous sections, but requirements for multiplexed clocks vary widely depending on the application.

Clock multiplexing is acceptable if the following criteria are met:

- The clock multiplexing logic does not change after initial configuration
- The design bypasses functional clock multiplexing logic to select a common clock for testing purposes
- Registers are always in reset when the clock switches
- A temporarily incorrect response following clock switching has no negative consequences

If the design switches clocks on the fly with no reset and the design cannot tolerate a temporarily incorrect response of the chip, then one must use a synchronous design so that there are no timing violations on the registers, no glitches on clock signals, and no race conditions or other logical problems.

2.4.5 Synchronous Clock Enables and Gated Clocks

Gated clocks turn a clock signal on and off using an enable signal that controls some sort of gating circuitry. As shown in Fig. 2.20, when a clock is turned off, the corresponding clock domain is shut down and becomes functionally inactive.

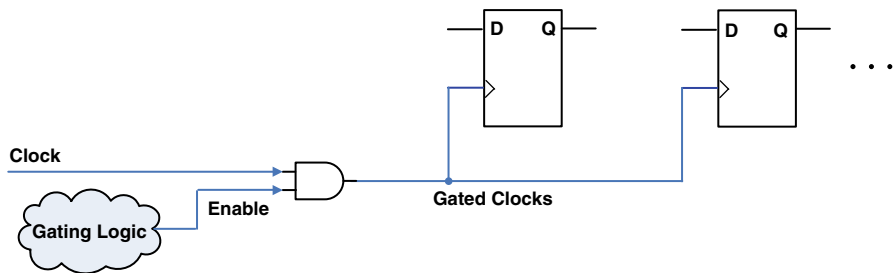


Fig. 2.20 Gated clock

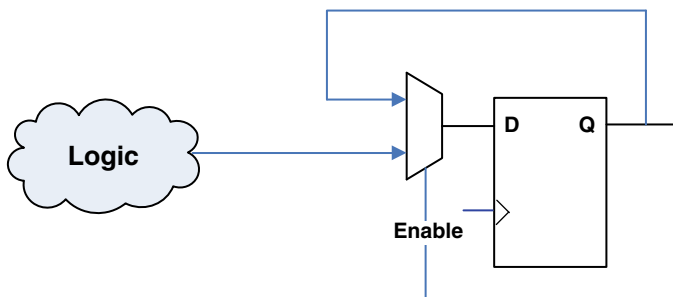


Fig. 2.21 Synchronous clock enable

Gated clocks can be a powerful technique to reduce power consumption. When a clock is gated both the clock network and the registers driven by it stop toggling, thereby eliminating their contributions to power consumption. However, gated clocks are not part of a synchronous scheme and therefore can significantly increase the effort required for design implementation and verification. Gated clocks contribute to clock skew and are also sensitive to glitches, which can cause design failure.

A clock domain can be turned off in a purely synchronous manner using a synchronous clock enable. However, when using a synchronous clock enable scheme, the clock tree keeps toggling and the internal circuitry of each Flip Flop remains active (although outputs do not change values), which does not reduce power consumption. A synchronous clock enable technique is shown in Fig. 2.21.

This Synchronous Clock Enable Clocking scheme does not reduce power consumption as much as gating the clock at the source because the clock network keeps toggling, but it will perform the same function as a gated clock by disabling a set of Flip Flops. As shown in Fig. 2.21, multiplexer in front of the data input of every Flip Flop either load new data or copy the output of the Flip Flop based on the Enable signal.

The next section is dedicated to efficient clock gating methodology that should be used where ever clocking gating is desired due to tight power specifications.

2.5 Clock Gating Methodology

In the traditional synchronous design style, the system clock is connected to the clock pin on every flip-flop in the design. This results in three major components of power consumption:

1. Power consumed by combinatorial logic whose values are changing on each clock edge (due to flops driving those combo cells).
2. Power consumed by flip-flops (this has non-zero value even if the inputs to the flip-flops, and therefore, the internal state of the flip-flops, is not changing).
3. Power consumed by the clock tree buffers in the design.

Gating the clock path substantially reduces the power consumed by a Flip Flop. Clock Gating can be done at the root of the clock tree, at the leaves, or somewhere in between.

Since the clock tree constitutes almost 50% of the whole chip power, it is always a good idea to generate and gate the clock at the root so that entire clock tree can be shut down instead of implementing the gating along the clock tree at the leaves.

Figure 2.22 shows an example of a clock gating for a three bit Counter.

The circuit is similar to the traditional implementation except that a clock gating element has been inserted into the clock network, which causes the flip-flops to be clocked only when the *INC* input is high. When the *INC* input is low, the flip-flops are not clocked and therefore retain the old data. This saves three multiplexers in front of the flip-flops which would have been there in case the gating was implemented by Synchronous Clock Enable as described in Fig. 2.21. This can result in significant area saving when wide banks of registers are being implemented.

2.5.1 Latch Free Clock Gating Circuit

The latch-free clock gating style uses a simple AND or OR gate (depending on the edge on which flip-flops are triggered) as shown in Fig. 2.23.

For the correct operation the circuit imposes a requirement that all enable signals be held constant from the active (rising) edge of the clock until the inactive (falling) edge of the clock to avoid truncating the generated clock pulse prematurely or generating multiple clock pulses (or glitches in clock) where one is required.

Figure 2.24 shows the case where generated clock is truncated prematurely when the above requirement is not satisfied.

This restriction makes the latch-free clock gating style inappropriate for our single-clock flip-flop based design.

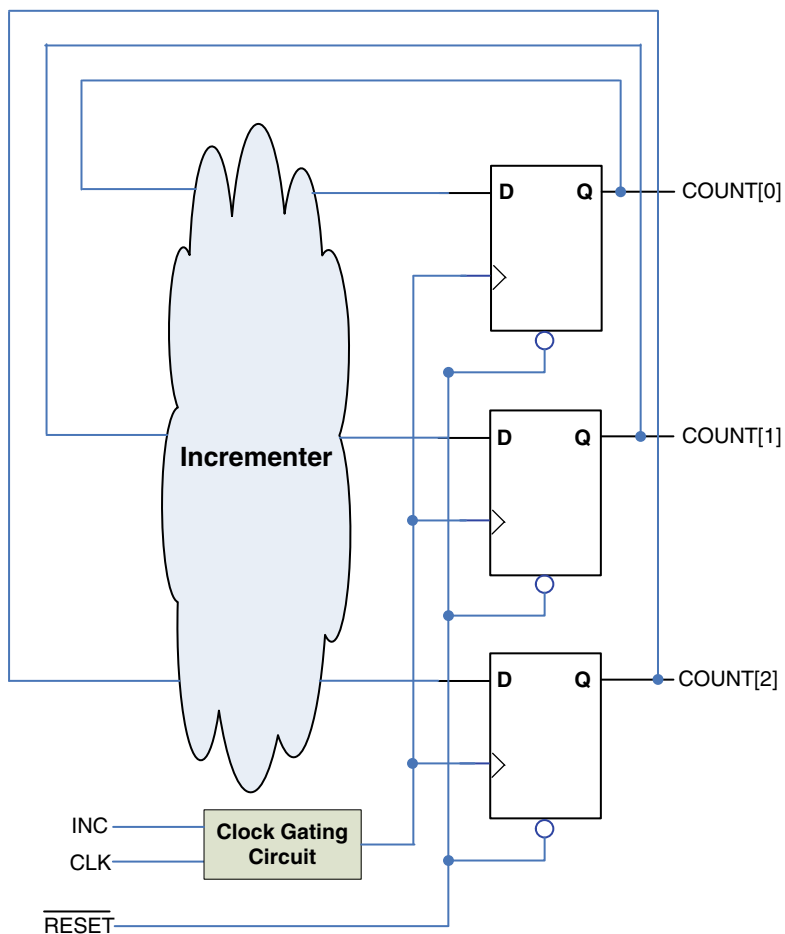


Fig. 2.22 Three bit counter with clock gating

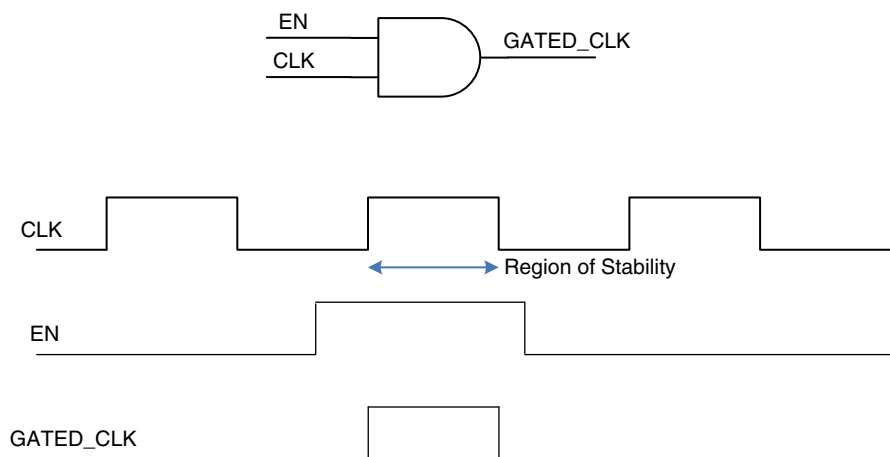


Fig. 2.23 Latch free clock gating circuit

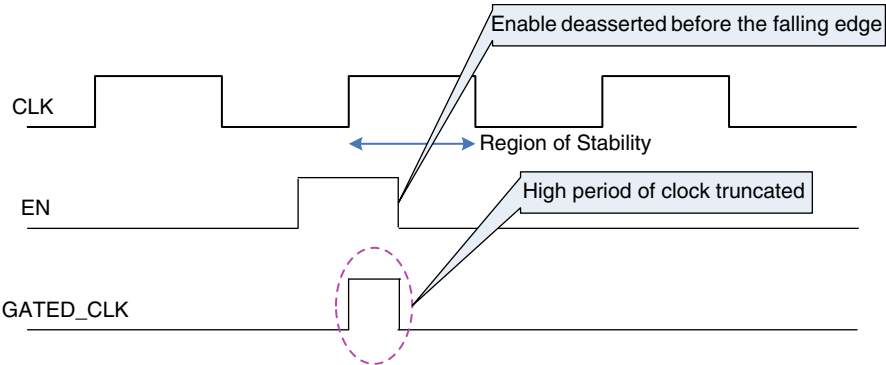


Fig. 2.24 Generated clock terminated prematurely

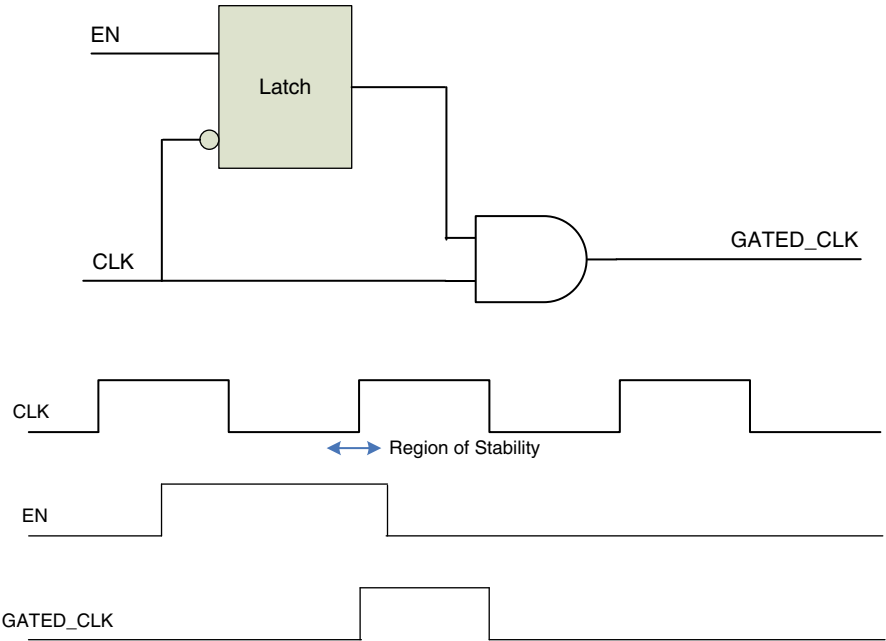


Fig. 2.25 Latch based clock gating circuit

2.5.2 Latch Based Clock Gating Circuit

The latch-based clock gating style adds a level-sensitive latch to the design to hold the enable signal from the active edge of the clock until the inactive edge of the clock, making it unnecessary for the circuit to itself enforce that requirement as shown in Fig. 2.25.

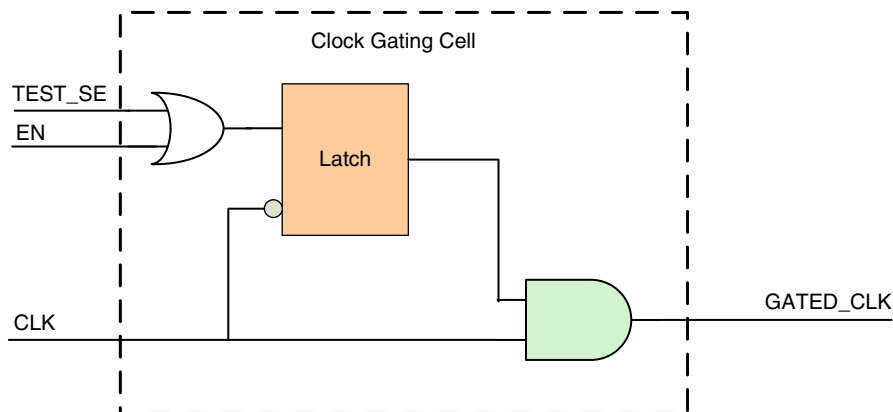


Fig. 2.26 Standard clock gating cell

Since the latch captures the state of the enable signal and holds it until the complete clock pulse has been generated, the enable signal need only be stable around the rising edge of the clock.

Using this technique, only one input of the gate that turns the clock on and off changes at a time, ensuring that the circuit is free from any glitches or spikes on the output.

Note: Use an AND gate to gate a clock that is active on the rising edge. For a clock that is active on the falling edge, use an OR gate to gate the clock and register the enable with a positive edge-triggered Latch.

When using this technique, special attention should be paid to the duty cycle of the clock and the delay through the logic that generates the enable signal, because the enable signal must be generated in half the clock cycle. This situation might cause problems if the logic that generates the enable command is particularly complex, or if the duty cycle of the clock is severely unbalanced. However, being careful with the duty cycle and logic delay may be acceptable compared with the problems created by other methods of gating clocks.

To ensure high manufacturing fault coverage, it is necessary to make sure the clock gating circuit is full controllable and observable to use within a scan methodology. A controllability signal which causes all flip-flops in the design to be clocked, regardless of the enable term value, can be added to allow the scan chain to shift information normally.

This signal can be ORed in with the enable signal before the latch and can be connected to either a test mode enable signal which is asserted throughout scan testing or to a scan enable signal which is asserted only during scan shifting.

The modified circuit is shown in Fig. 2.26. Most of the ASIC vendors do provide this “Clock Gating Cell” as a part of their standard library cell.

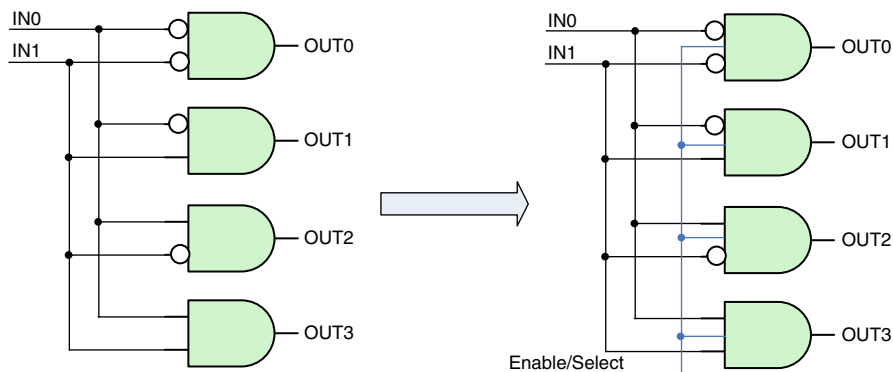


Fig. 2.27 Decoder with enable

2.5.3 Gating Signals

Effective power implementation can be achieved using gating signals for particular parts of the design. Similar to the concept of gating clock, signal gating reduces the transitions in clock free signals. The most common example is the decoder enable.

As part of an address decoding mechanism, signals used by other parts of the design may toggle as a reflection of activity in these parts. Switching activity on one input of the decoder will induce a large number of toggling gates. Controlling this with an enable or select signal prevents the propagation of their switching activity, even if the logic is slightly more complex (Fig. 2.27).

2.5.4 Data Path Re-ordering to Reduce Switching Propagation

Several data path elements, such as decoders or comparison operators, as well as “glitchy” logic may significantly contribute to power dissipation. The glitches, caused by late arrival signals or skews, propagate through other data path elements and logic until they reach a register. This propagation burns more power as the transitions traverse the logic levels. To reduce this wasted dissipation, designers need to rewrite the HDL code and shorten the propagation paths as much as possible. Figure 2.28 illustrates two implementations of the priority mux where the “glitchy” and “stable” conditions are ordered differently.

2.6 Reset Design Strategy

Many design issues must be considered before choosing a reset strategy for an ASIC design, such as whether to use synchronous or asynchronous resets, will every flip-flop receive a reset etc.

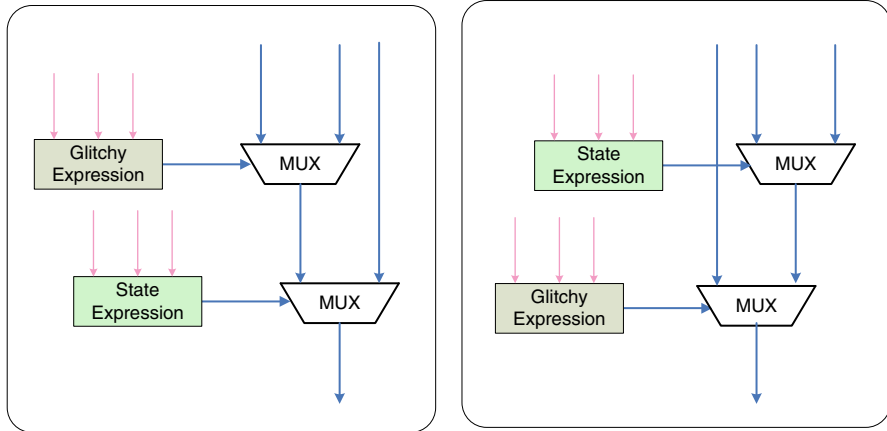


Fig. 2.28 Data path re-ordering to reduce switching propagation

The primary purpose of a reset is to force the SoC into a known state for stable operations. This would avoid the SoC to power on to a random state and get hanged. Once the SoC is built, the need for the SoC to have reset applied is determined by the system, the application of the SoC, and the design of the SoC. A good design guideline is to provide reset to every flip-flop in a SoC whether or not it is required by the system. In some cases, when pipelined flip-flops (shift register flip-flops) are used in high speed applications, reset might be eliminated from some flip-flops to achieve higher performance designs.

A design may choose to use either an Asynchronous or Synchronous reset or a mix of two. There are distinct advantages and disadvantages to use either synchronous or asynchronous resets and either method can be effectively used in actual designs. The designer must use an approach that is most appropriate for the design.

2.6.1 Design with Synchronous Reset

Synchronous resets are based on the premise that the reset signal will only affect or reset the state of the flip-flop on the active edge of a clock. In some simulators, based on the logic equations, the logic can block the reset from reaching the flip-flop. This is only a simulation issue, not a real hardware issue.

The reset could be a “late arriving signal” relative to the clock period, due to the high fanout of the reset tree. Even though the reset will be buffered from a reset buffer tree, it is wise to limit the amount of logic the reset must traverse once it reaches the local logic.

Figure 2.29 shows one of the RTL code for a loadable Flop with Synchronous Reset. Figure 2.30 shows the corresponding hardware implementation.

```

module load_syn_ff ( clk, in, out, load, rst_n);
input clk, in, load, rst_n;
output out;

always @(posedge clk)
    if (!rst_n)
        out <= 1'b0; // sync reset
    else if (load)
        out <= in; // sync

endmodule

```

Fig. 2.29 Verilog RTL code for loadable flop with synchronous reset

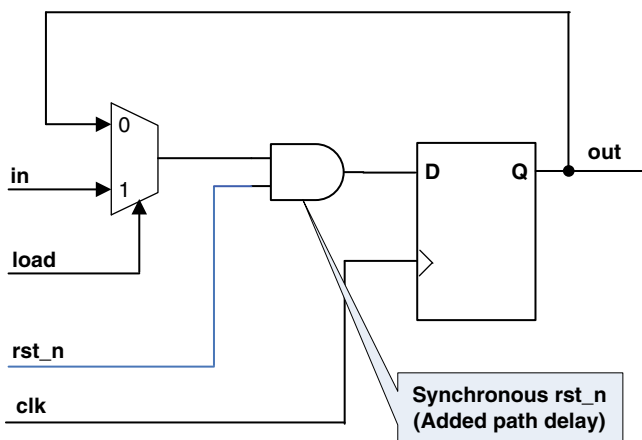


Fig. 2.30 Loadable flop with synchronous reset (hardware implementation)

One problem with synchronous resets is that the synthesis tool cannot easily distinguish the reset signal from any other data signal. The synthesis tool could alternatively have produced the circuit of Fig. 2.31.

Circuit shown in Fig. 2.31 is functionally identical to implementation shown in Fig. 2.30 with the only difference that reset AND gates are outside the MUX. Now, consider what happens at the start of a gate-level simulation. The inputs to both legs of the MUX can be forced to 0 by holding “*rst_n*” asserted low, however if “*load*” is unknown (X) and the MUX model is pessimistic, then the flops will stay unknown (X) rather than being reset. Note this is only a problem during simulation. The actual circuit would work correctly and reset the flop to 0.

Synthesis tools often provide compiler directives which tell the synthesis tool that a given signal is a synchronous reset (or set). The synthesis tool will “pull” this signal as close to the flop as possible to prevent this initialization problem from occurring.

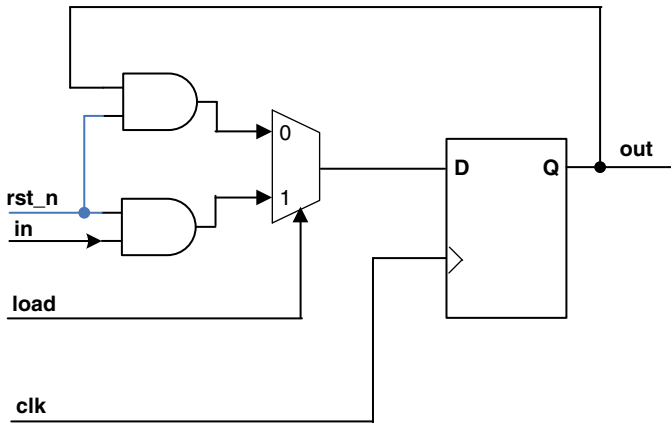


Fig. 2.31 Alternate implementation for loadable flop with synchronous reset

It would be recommended to add these directives to the RTL code from the start of project to avoid re-synthesizing the design late in the project schedule.

2.6.1.1 Advantages of Using Synchronous Resets

1. Synchronous resets generally insure that the circuit is 100% synchronous.
2. Synchronous reset logic will synthesize to smaller flip-flops, particularly if the reset is gated with the logic generating the Flop input.
3. Synchronous resets ensure that reset can only occur at an active clock edge. The clock works as a filter for small reset glitches.
4. In some designs, the reset must be generated by a set of internal conditions. A synchronous reset is recommended for these types of designs because it will filter the logic equation glitches between clocks.

2.6.1.2 Disadvantages of Using Synchronous Resets

Not all ASIC libraries have flip-flops with built-in synchronous resets. Since synchronous reset is just another data input, the reset logic can be easily synthesized outside the flop itself (as shown in Figs. 2.30 and 2.31).

1. Synchronous resets may need a pulse stretcher to guarantee a reset pulse width wide enough to ensure reset is present during an active edge of the clock. This is an issue that is important to consider when doing multi-clock design. A small counter can be used that will guarantee a reset pulse width of a certain number of cycles.
2. A potential problem exists if the reset is generated by combinational logic in the SoC or if the reset must traverse many levels of local combinational logic. During simulation, depending on how the reset is generated or how the reset is applied

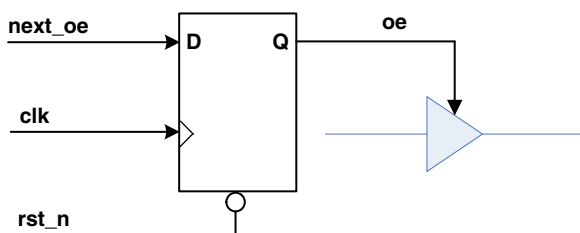


Fig. 2.32 Asynchronous reset for output enable

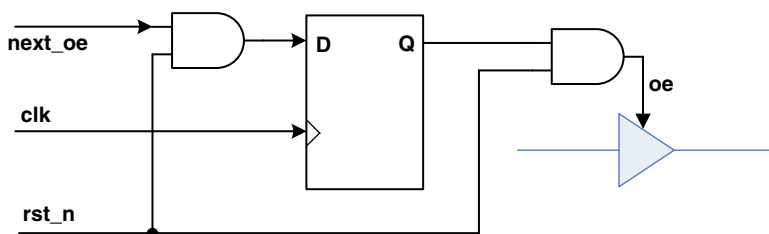


Fig. 2.33 Synchronous reset for output enable

to a functional block, the reset can be masked by X's. The problem is not so much what type of reset you have, but whether the reset signal is easily controlled by an external pin.

3. By its very nature, a synchronous reset will require a clock in order to reset the circuit. This may be a problem in some case where a gated clock is used to save power. Clock will be disabled at the same time during reset is asserted. Only an asynchronous reset will work in this situation, as the reset might be removed prior to the resumption of the clock.

The requirement of a clock to cause the reset condition is significant if the ASIC/FPGA has an internal tristate bus. In order to prevent bus contention on an internal tristate bus when a chip is powered up, the chip should have a power-on asynchronous reset as shown in Fig. 2.32.

A synchronous reset could be used; however you must also directly de-assert the tristate enable using the reset signal (Fig. 2.33). This synchronous technique has the advantage of a simpler timing analysis for the reset-to-HiZ path.

2.6.2 Design with Asynchronous Reset

Asynchronous reset flip-flops incorporate a reset pin into the flip-flop design. With an active low reset (normally used in designs), the flip-flop goes into the reset state when the signal attached to the flip-flop reset pin goes to a logic low level.

```

module load_asyn_ff ( clk, in, out, load, rst_n);
  input clk, in, load, rst_n;
  output out;

  always @(posedge clk or nedge rst_n)
    if (!rst_n)
      out <= 1'b0; // sync reset
    else if (load)
      out <= in; // sync
    else
      out <= out; // async

endmodule

```

Fig. 2.34 Verilog RTL code for loadable flop with asynchronous reset

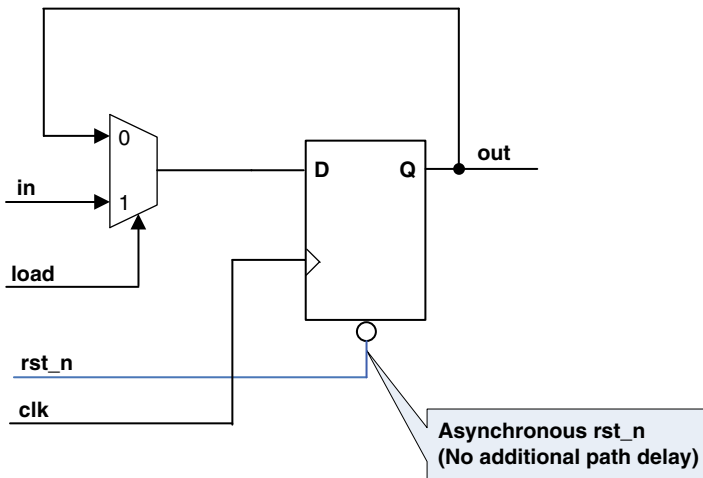


Fig. 2.35 Loadable flop with asynchronous reset (hardware implementation)

Figure 2.34 shows one of the RTL code for a loadable Flop with Asynchronous Reset. Figure 2.35 shows corresponding hardware implementation.

2.6.2.1 Advantages of Using Asynchronous Resets

1. The biggest advantage to using asynchronous resets is that, as long as the vendor library has asynchronously reset-able flip-flops, the data path is guaranteed to be clean. Designs that are pushing the limit for data path timing, cannot afford to have added gates and additional net delays in the data path due to logic inserted to handle synchronous resets. Using an asynchronous reset, the designer is guaranteed not to have the reset added to the data path (Fig. 2.35).

```

module dff_set_reset ( clk, in, out, rst_n, set_n);
  input clk, in, rst_n, set_n;
  output out;

  always @(posedge clk or nedge rst_n or negedge set_n)
    if (!rst_n)
      out <= 1'b0; // async reset
    else if (!set_n)
      out <= 1'b1; // async set
    else
      out <= in;
endmodule

```

Fig. 2.36 Verilog RTL for the flop with async reset and async set

2. The most obvious advantage favoring asynchronous resets is that the circuit can be reset with or without a clock present. Synthesis tool tend to infer the asynchronous reset automatically without the need to add any synthesis attributes.

2.6.2.2 Disadvantages of Using Asynchronous Resets

1. For DFT, if the asynchronous reset is not directly driven from an I/O pin, then the reset net from the reset driver must be disabled for DFT scanning and testing [30].
2. The biggest problem with asynchronous resets is that they are asynchronous, both at the assertion and at the de-assertion of the reset. The assertion is a non issue, the de-assertion is the issue. If the asynchronous reset is released at or near the active clock edge of a flip-flop, the output of the flip-flop could go metastable and thus the reset state of the SoC could be lost.
3. Another problem that an asynchronous reset can have, depending on its source, is spurious resets due to noise or glitches on the board or system reset. Often glitch filters needs to be designed to eliminate the effect of glitches on the reset circuit. If this is a real problem in a system, then one might think that using synchronous resets is the solution.
4. The reset tree must be timed for both synchronous and asynchronous resets to ensure that the release of the reset can occur within one clock period. The timing analysis for a reset tree must be performed after layout to ensure this timing requirement is met. One approach to eliminate this is to use distributed reset synchronizer flip-flop.

2.6.3 Flip Flops with Asynchronous Reset and Asynchronous Set

Most synchronous designs do not have flop-flops that contain both an asynchronous set and asynchronous reset, but at times such a flip-flop is required.

Figure 2.36 shows the Verilog RTL for the Asynchronous Set/Reset Flip Flop.

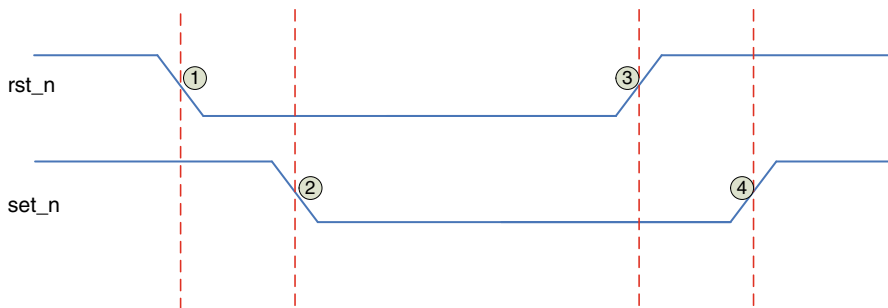


Fig. 2.37 Timing waveform for any asynchronous set/reset condition

```
// Add Compiler specific directive to
// ignore the following block during synthesis
always @(rst_n or set_n)
if (rst_n && !set_n) force q = 1;
else release q;
// End the compiler directive here
endmodule
```

Fig. 2.38 Simulation model for flop with asynchronous set/reset

Synthesis tool should be able to infer the correct flip flop with the asynchronous set/reset but this is not going to work in simulation. The simulation problem is due to the always block that is only entered on the active edge of the set, reset or clock signals.

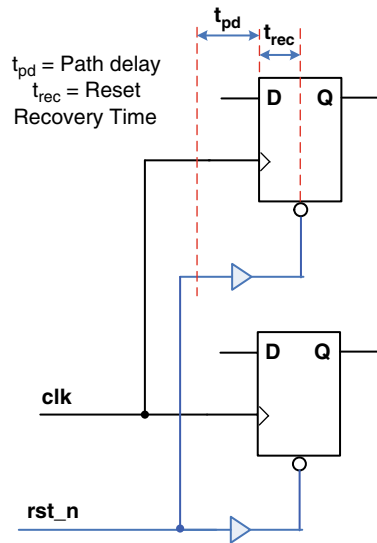
If the reset becomes active, followed then by the set going active, then if the reset goes inactive, the flip-flop should first go to a reset state, followed by going to a set state (Timing waveform shown in Fig. 2.37).

With both these inputs being asynchronous, the set should be active as soon as the reset is removed, but that will not be the case in Verilog since there is no way to trigger the always block until the next rising clock edge. Always block will be only triggered for 1 and 2 events shown in Fig. 2.37 and would skip the events 3 and 4.

For those rare designs where reset and set are both permitted to be asserted simultaneously and then reset is removed first, the fix to this simulation problem is to model the flip-flop using self-correcting code enclosed within the correct compiler directives and force the output to the correct value for this one condition. The best recommendation here is to avoid, as much as possible, the condition that requires a flip-flop that uses both asynchronous set and asynchronous reset.

The code shown in Fig. 2.38 shows the fix that will simulate correctly and guarantee a match between pre- and post-synthesis simulations.

Fig. 2.39 Asynchronous reset removal recovery time problem



2.6.4 Asynchronous Reset Removal Problem

Releasing the Asynchronous reset in the system could cause the chip to go into a metastable unknown state, thus avoiding the reset all together. Attention must be paid to the release of the reset so as to prevent the chip from going into a metastable unknown state when reset is released. When a synchronous reset is being used, then both the leading and trailing edges of the reset must be away from the active edge of the clock.

As shown in Fig. 2.39, there are two potential problems when an asynchronous reset signal is de-asserted asynchronous to the clock signal.

1. Violation of reset recovery time. Reset recovery time refers to the time between when reset is de-asserted and the time that the clock signal goes high again. Missing a recovery time can cause signal integrity or metastability problems with the registered data outputs.
2. Reset removal happening in different clock cycles for different sequential elements. When reset removal is asynchronous to the rising clock edge, slight differences in propagation delays in either or both the reset signal and the clock signal can cause some registers or flip-flops to exit the reset state before others.

2.6.5 Reset Synchronizer

Solution to asynchronous reset removal problem described in Sect. 2.6.4 is to use a Reset Synchronizer. This is the most commonly used technique to guarantee correct reset removal in the circuits using Asynchronous Resets. Without a reset synchronizer, the usefulness of the asynchronous reset in the final system is void even if the reset works during simulation.

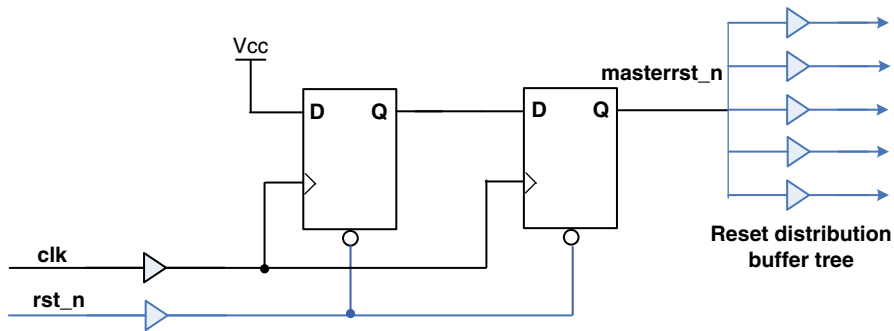


Fig. 2.40 Reset synchronizer block diagram

The reset synchronizer logic of Fig. 2.40 is designed to take advantage of the best of both asynchronous and synchronous reset styles.

An external reset signal asynchronously resets a pair of flip-flops, which in turn drive the master reset signal asynchronously through the reset buffer tree to the rest of the flip-flops in the design. The entire design will be asynchronously reset.

Reset removal is accomplished by de-asserting the reset signal, which then permits the d-input of the first master reset flip-flop (which is tied high) to be clocked through a reset synchronizer. It typically takes two rising clock edges after reset removal to synchronize removal of the master reset.

Two flip-flops are required to synchronize the reset signal to the clock pulse where the second flip-flop is used to remove any metastability that might be caused by the reset signal being removed asynchronously and too close to the rising clock edge.

Also note that there are no metastability problems on the second flip-flop when reset is removed. The first flip-flop of the reset synchronizer does have potential metastability problems because the input is tied high, the output has been asynchronously reset to a 0 and the reset could be removed within the specified reset recovery time of the flip-flop (the reset may go high too close to the rising edge of the clock input to the same flip-flop). This is why the second flip-flop is required.

The second flip-flop of the reset synchronizer is not subjected to recovery time metastability because the input and output of the flip-flop are both low when reset is removed. There is no logic differential between the input and output of the flip-flop so there is no chance that the output would oscillate between two different logic values.

The following equation calculates the total reset distribution time

$$T_{rst_dis} = t_{clk-q} + t_{pd} + t_{rec}$$

where

t_{clk-q} = Clock to Q propagation delay of the second flip flop in the reset synchronizer

t_{pd} = Total delay through the reset distribution tree

t_{rec} = Recovery time of the destination flip flop

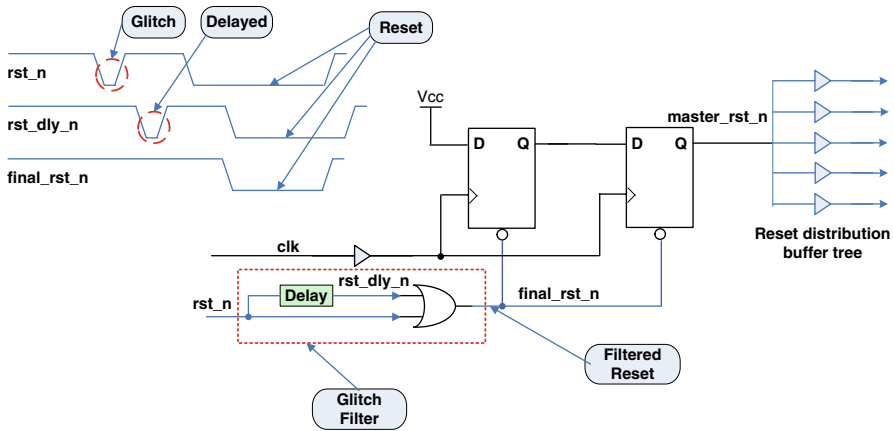


Fig. 2.41 Reset glitch filtering

2.6.6 Reset Glitch Filtering

Asynchronous Reset are susceptible to glitches, that means any input wide enough to meet the minimum reset pulse width for a flip-flop will cause the flip-flop to reset. If the reset line is subject to glitches, this can be a real problem. A design may not have a very high frequency sampling clock to detect small glitch on the reset; this section presents an approach that will work to filter out glitches [30]. This solution requires a digital delay to filter out small glitches. The reset input pad should also be a Schmidt triggered pad to help with glitch filtering. Figure 2.41 shows the reset glitch filter circuit and the timing diagram.

In order to add the delay, some vendors provide a delay hard macro that can be hand instantiated. If such a delay macro is not available, the designer could manually instantiate the delay into the synthesized design after optimization. A second approach is to instantiate a slow buffer in a module and then instantiated that module multiple times to get the desired delay. Many variations could expand on this concept.

Since this approach uses delay lines, one of the disadvantages is that this delay would vary with temperature, voltage and process. Care must be taken to make sure that the delay meets the design requirements across all PVT corners.

2.7 Controlling Clock Skew

Difference in clock signal arrival times across the chip is called clock skew. It is a fundamental design principle that timing must satisfy register setup and hold time requirements. Both data propagation delay and clock skew are parts of these

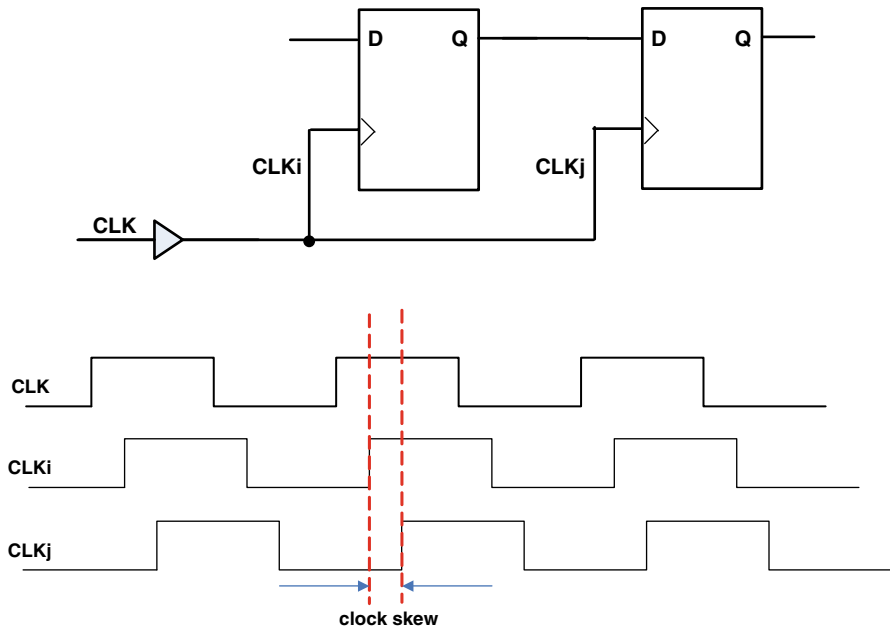


Fig. 2.42 Clock skew in two sequentially adjacent flip-flops

calculations. Clocking sequentially-adjacent registers on the same edge of a high-skew clock can potentially cause timing violations or even functional failures. Probably this is one of the largest sources of design failure in an ASIC.

Figure 2.42 shows an example of clock skew for two sequentially adjacent flip-flops.

Given two sequentially-adjacent flops, F_i and F_j , and an equi-potential clock distribution network, the clock skew between these two flops is defined as

$$T_{\text{skew}_{ij}} = T_{c_i} - T_{c_j}$$

where T_{c_i} and T_{c_j} are the clock delays from the clock source to the Flops F_i and F_j , respectively.

2.7.1 Short Path Problem

The problem of short data paths in the presence of clock skew is very similar to hold-time violations in flip-flops. The problem arises when the data propagation delay between two adjacent flip-flops is less than the clock skew.

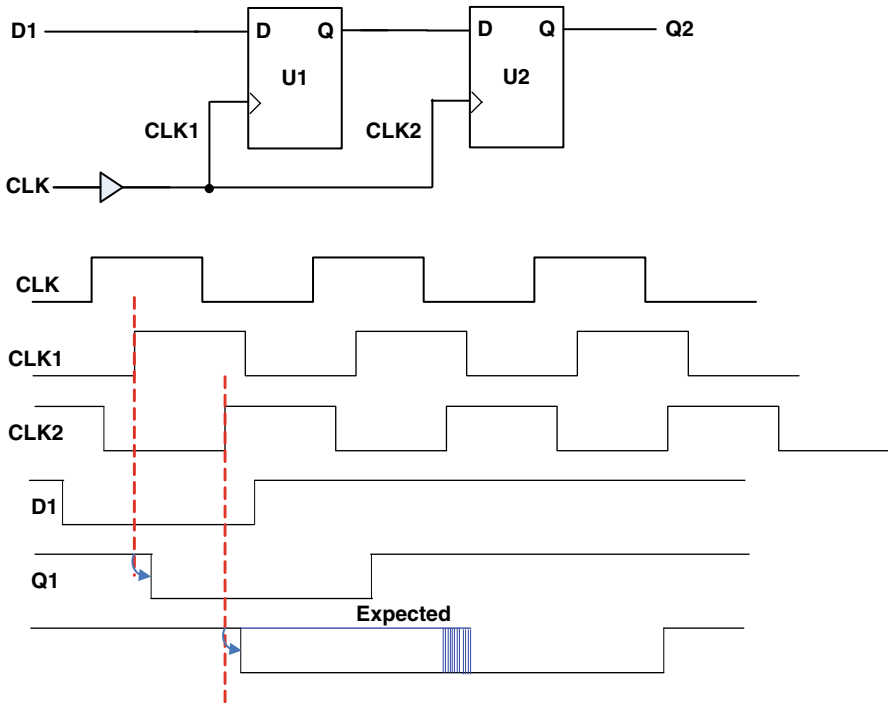


Fig. 2.43 Circuit with a short path problem

Figure 2.43 shows a circuit with timings to illustrate a short-path problem.

Since the same clock edge arrives at the second flip-flop later than the new data, the second flip-flop output switches at the same edge as the first flip-flop and with the same data as the first flip-flop. This will cause U2 to shift the same data on the same edge as U1, resulting in a functional error.

2.7.2 Clock Skew and Short Path Analysis

As mentioned earlier, clock skew and short-path problems emerge when the data propagation path delay between two sequentially adjacent flip-flops is less than the clock skew between the two. Figure 2.44 shows the general diagram of the delay blocks in a sample circuit [33].

The delays in Fig. 2.44 are as follows:

- T_{cq1} : The clock to out delay of the first flip-flop
- T_{rdq1} : The propagation delay from the output of the first flip-flop to the input of the second one
- T_{ck2} : The clock arrival time at the second flip-flop minus the clock arrival time at the first flip-flop

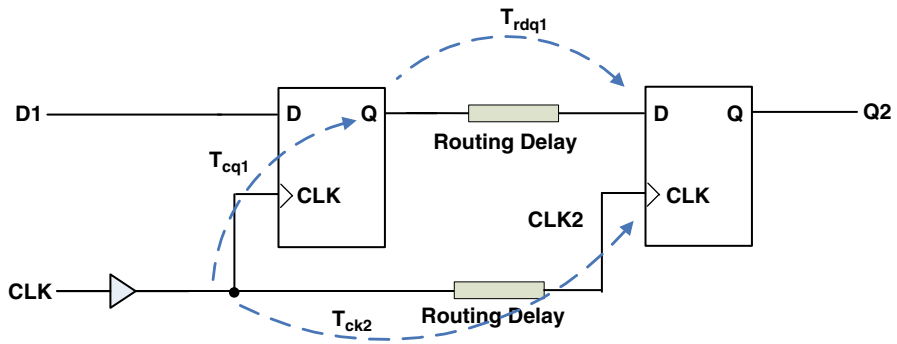


Fig. 2.44 General delay blocks in a simple circuit

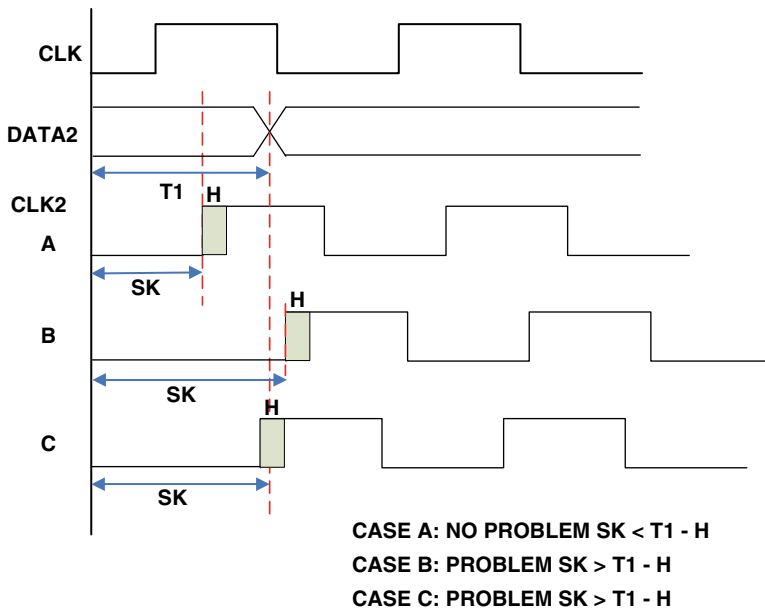


Fig. 2.45 Illustration of short path problem

The short-path problem will definitely emerge in this circuit if

$$T_{ck2} > T_{cq1} + T_{rdq1} - T_{HOLD2}$$

where T_{HOLD2} is the hold-time requirement of the sink flip-flop.

The regions are illustrated in Fig. 2.45.

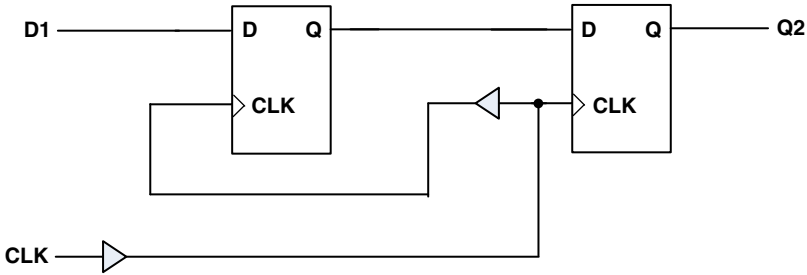


Fig. 2.46 Clock reversing methodology

Therefore, in order to identify the paths with the problem, the user needs to extract the clock skew (e.g. T_{ck2}) and the short-path delays (e.g. $T_{cq1} + T_{rdq1} - T_{HOLD2}$).

2.7.3 Minimizing Clock Skew

Reducing the Clock skew to the minimum is the best approach to reduce the risk of short-path problems. Maintaining the clock skew at a value less than the smallest Flop-to-Flop delay in the design will improve the robustness of the design against any short-path problems.

The following sections are a few well-known design techniques to make designs more robust against clock skew.

2.7.3.1 Adding Delay in Data Path

As Shown in Fig. 2.44, by increasing the Routing Delay in the data path (T_{rdq1}) that eventually increases the total delay of the data path to a value greater than the clock skew, will eliminate the short path problem.

The amount of the inserted delay in the data path should be large enough so that the data path delay becomes sufficiently greater than the clock skew.

2.7.3.2 Clock Reversing

Clock reversing is another approach to get around the problem of short data paths and clock skew. In this technique Clock is applied in the reverse direction with respect to data so that clock skew is automatically eliminated.

The receiving Flop will clock in the transmitting (source) value before the transmitting register receives its clock edge. Figure 2.46 shows a simple example of implementing the clock reversing approach.

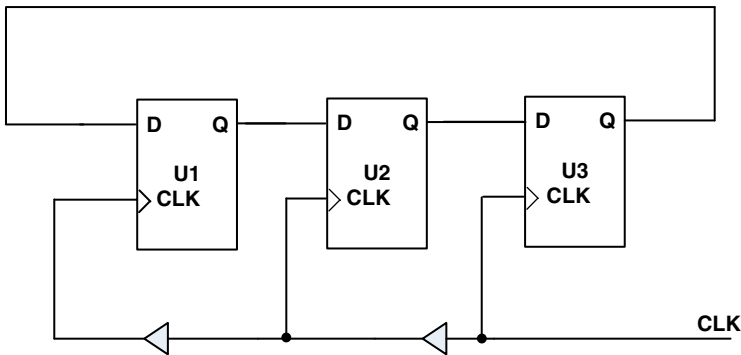


Fig. 2.47 Clock reversing in a circular structure

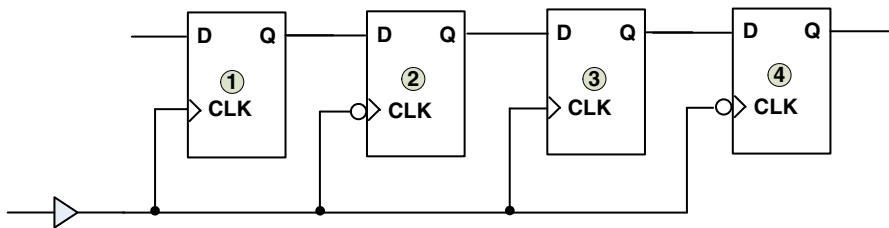


Fig. 2.48 Alternate edge clocking

As shown when sufficient delay is inserted, the receiving Flop will receive the active-clock edge before the source Flop. This improves the Hold time at the expense of Setup Time.

The clock reversing method will not be effective in circular structures such as Johnson counters and Linear Feedback Shift Registers (LFSRs), because it is not possible to define the Sink Flop explicitly. Figure 2.47 shows an example of a circular structure with clock reversing interconnection. As shown, short-path problem exists between flip-flops U1 and U3.

2.7.3.3 Alternate Phase Clocking

One of the known methodologies to avoid clock skew issues is alternate-phase clocking. The following sections mentions few design techniques of alternate phase clocking.

Clocking on Alternate Edges

In this method, sequentially adjacent Flops are clocked on the opposite edges of the clock as shown in Fig. 2.48.

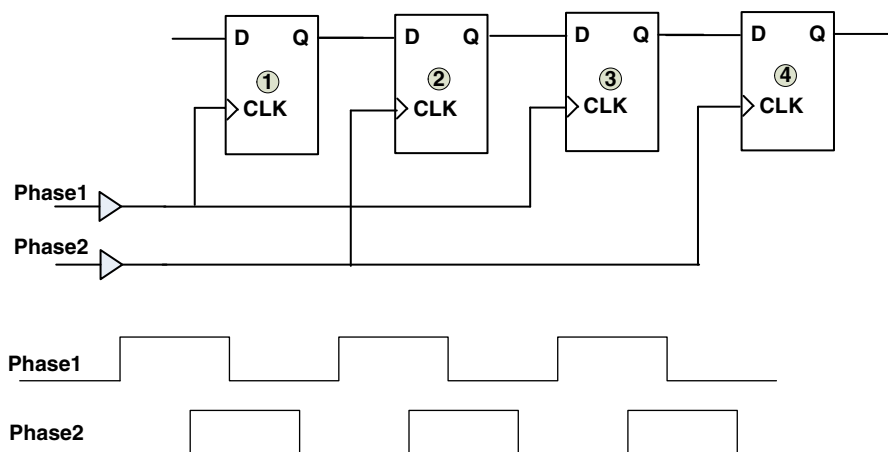


Fig. 2.49 Alternate phase clocking

As shown this method provides a short path-clock skew margin of about one half clock cycle for clock skew.

Clocking on Alternate Phases

Figure 2.49 shows a set of adjacent Flops, which are alternately clocked on two different phases of the same clock. In this case, between each two adjacent Flops, there is a safety margin approximately equal to the phase difference of the two phases.

The user should note that the usage of alternate-phase clocking may require completely different clock constraints on the original clock signal. For example, in the case of clocking on alternate edges, the new constraint on the clock frequency will be half the original frequency since the adjacent Flops are clocked on opposite edges of the same clock cycle.

Ripple Clocking Structure

In a ripple structure, each Flop output drives the next Flop clock port just like the way a Ripple counter is implemented. Here the sink Flop will not clock unless the source Flop toggled as shown in Fig. 2.50.

As shown in Fig. 2.50, the output of each counter flop drives the clock port of the next Flop instead of its data input port. This will eliminate the clock skew since the Flops do not toggle on the same clock. The first Flop is clocked on the positive edge of the CLK signal and the second- and third-stage Flops are clocked on the positive edge of the output of the previous Flop.

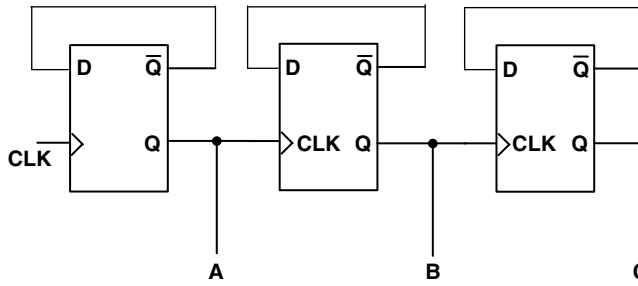


Fig. 2.50 Three bit ripple down-counter

Different techniques as mentioned above may be used to minimize clock skew and avoid short path problems depending on the design complexity and methodology being used.

2.7.3.4 Balancing Trace Length

Techniques described in the previous section are more on design techniques that may be planned much before the final project phase. Of course alternative to the above, Designers may choose to balance the trace length for low skew clock drivers. Apart from merely providing equal traces on all clock nets, the same termination strategy should be used on each trace by placing the same load at the end of the line. This would make sure trace lengths are properly balanced.

Below are some of the guidelines that should be followed:

1. Pay close attention to the specifications for input-to-output delay on the drivers.
2. Use the same drivers at every level of the clock hierarchy.
3. Balance the nominal trace delays at each level.
4. Use the same termination strategy on each line.
5. Balance the loading on each line, even if that means adding dummy capacitors to one branch to balance out loads on the other branches.

References

1. Mohit Arora, Prashant Bhargava, Amit Srivastava, *Optimization and Design Tips for FPGA/ASIC(How to make the best designs)*, DCM Technologies, SNUG India, 2002
2. Application Note, ASIC design guidelines, Atmel Corporation, 1999
3. Cummings CE, Sunburst Design, Inc.; Mills D, LCDM Engineering (2002) Synchronous resets? Asynchronous resets? I am so confused! How will I ever know which to use? SNUG, San Jose
4. Application Note, Clock skew and short paths timing, Actel Corporation, 2004

Chapter 3

Handling Multiple Clocks

3.1 Introduction

Designs involving single clocks are easy and simple to implement. But in actual practice, there are few practical designs that function on just one clock. This chapter deals with multiple clock designs, problems faced therein and solutions in order to get a robust design that works on multiple clocks.

A single clock design or rather synchronous design is shown in Fig. 3.1. In a single clock domain, there is a single clock that goes through the entire design. Such designs are easy to implement, pose less problems of Metastability, Setup and Hold time violations as compared to multiple clock designs.

3.2 Multiple Clock Domains

One of the challenges faced by an engineer is to develop designs with multiple clocks. Designs with multiple clocks (Fig. 3.2) can have any or all of the following type of clock relationships:

- Clocks with different frequencies.
- Clocks with same frequency but different phases between them.

The two relationships between these clocks are shown in Fig. 3.3.

3.3 Problems with Multiple Clock Domains Design

Multiple Clock designs are subjected to

- (a) Setup Time and Hold Time Violations
- (b) Metastability

Fig. 3.1 Single clock domain

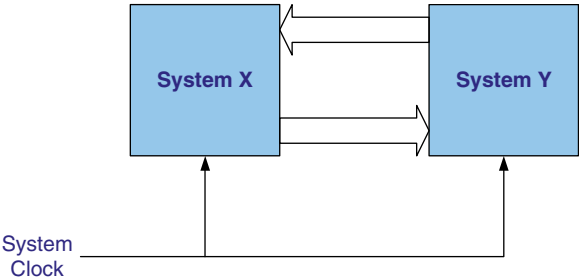
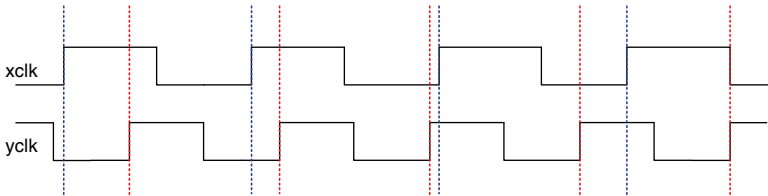
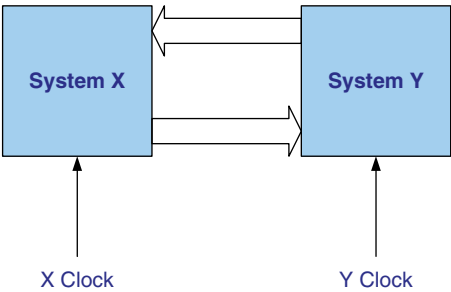
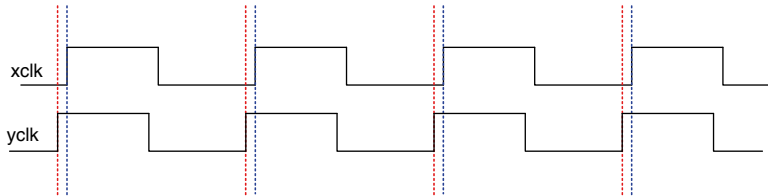


Fig. 3.2 Multiple clock domains



Clocks with different frequencies



Clock having same frequency but different phase

Fig. 3.3 Relationship between clocks (in multiple clock domains)

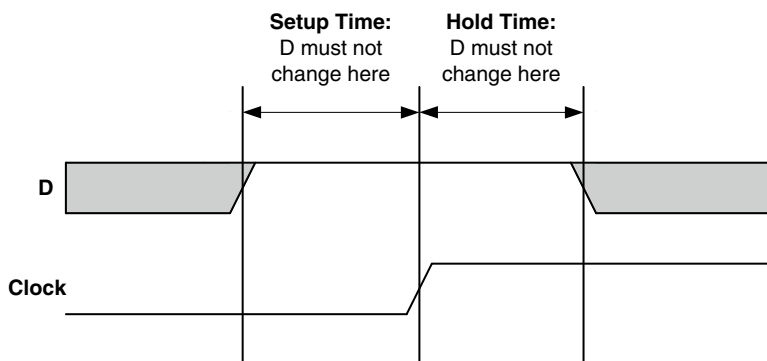


Fig. 3.4 Setup and Hold Times (w.r.t. To Clock Edge)

3.3.1 Setup Time and Hold Time Violation

Setup Time: The time required for data input to remain stable prior to arrival of clock pulse.

Hold Time: The time required for data input to remain stable after the arrival of clock pulse.

Figure 3.4 shows setup time and Hold time with respect to the rising edge of clock.

Setup Time requires that input should become stable before the rising edge of the clock while, Hold Time requires that input remains stable after the arrival of clock pulse. This can be easily achieved in single clock domains. However, in multiple clock domains, it may happen that the output from one clock domain may be changing when the rising edge of the second clock domain comes. This will cause the output of flops in the second clock domain to become metastable, thereby leading to wrong results.

Consider a dual clock system shown in Fig. 3.2. The timing for transfer of signals between domains is shown in Fig. 3.5. As shown, *xclk_output1* (belonging to the *xclk* domain) changes near the rising edge of *yclk*. When this signal is sampled by *yclk* domain, the *xclk_output1* signal is in transition. This causes violation of setup and hold time with respect to *yclk*. Thus the signals in *yclk* domain that depend on *xclk_output1* will go into metastable state and give wrong results. However, *xclk_output2* (belonging to the *xclk* domain) is stable at the rising edge of *yclk*. There is no violation of setup and hold time. Thus the signals in *yclk* domain that depend on the *xclk_output2* signal will provide a correct output.

3.3.2 Metastability

Metastability problems due to multiple clock domains have been detailed in Chap 1.

Next few sections describe some of the design tips and solutions to make a robust multiple clock design.

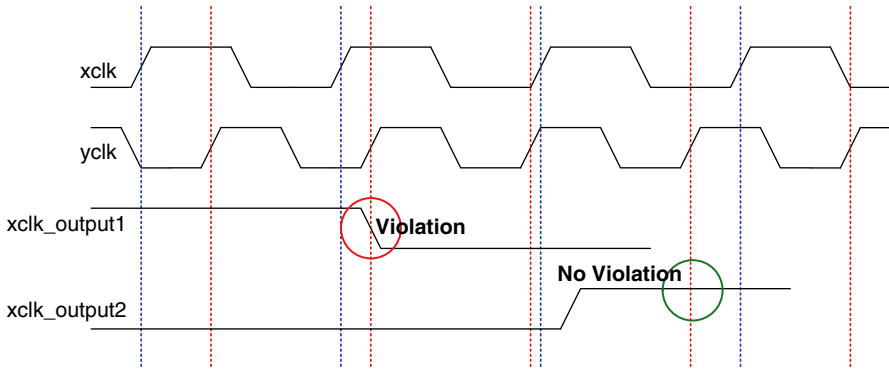


Fig. 3.5 Violation of setup and hold time

3.4 Design Tips for Efficient Handling of a Design with Multiple Clocks

When working on a design with multiple clocks, it is beneficial that one follows certain guidelines to help during simulation and synthesis. Some common guidelines are:

- Clock Nomenclature
- Design Partitioning

3.4.1 Clock Nomenclature

For easy handling of clock signals by synthesis scripts, it is necessary that there should be a certain clock naming procedure that is used all over the design. For example, system clock be named as *sys_clk*, transmitter clock be named as *tx_clk* and receiver clock as *rx_clk*, etc. This would help during scripting to refer to all clock signals using wildcards. Similarly, signals belonging to a particular clock domain can be prefixed by its clock name. For example, signals clocked by system clock can start as *sys_rom_addr*, *sys_rom_data*.

Using this nomenclature any engineer on the team can identify to which domain a particular signal belongs and decide whether to use the signal directly or pass it through a synchronizer.

This naming procedure can significantly reduce confusion and provide easy interfacing among modules thereby increasing the efficiency of the team.

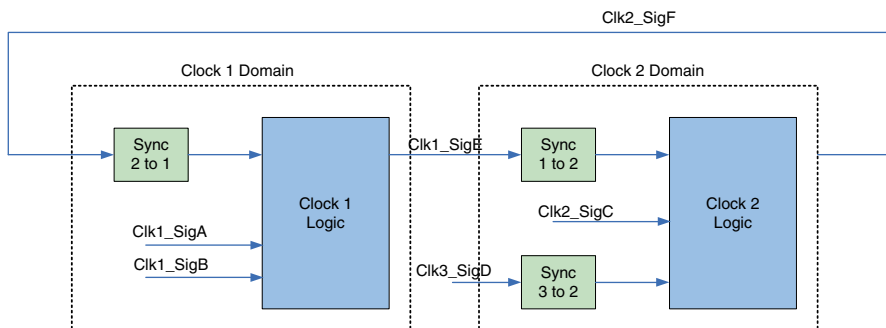


Fig. 3.6 Design partitioning

3.4.2 Design Partitioning

This is another technique for efficient designing of modules having multiple clocks. According to this guideline:

1. One module should work on one clock only.
2. A synchronizer module (module that performs the function of transferring signals from one domain to another) be made for all signals that cross from one clock domain to another, so that all inputs are synchronized to the same clock before entering the module.
3. The synchronizer module should be as small as possible.

The advantage of partitioning a design is that static timing analysis becomes easy as all signals entering or leaving a clock domain are made synchronous to the clock used in that module. The design becomes completely synchronous. Also, no timing analysis is required on the synchronization modules. However, it should be ensured that hold time requirements are met.

As shown in Fig. 3.6, the entire logic is separated into three clock domains viz Clock1, Clock2 and Clock3 domains. The names to the signal have been given as per the nomenclature explained in Sect. 3.4.1. Any signal going from one clock domain to another passes through an external synchronization module. This synchronization module converts the clock domain of the signal to the clock domain used by the module. Thus, as shown in Fig. 3.6, Sync 1 to 2 module converts the signals coming from clock1 domain to clock2 domain.

3.4.3 Clock Domain Crossing

The transfer of signals between clock domains can be categorized into two groups, namely:

- Transfer of Control Signals
- Transfer of Data Signals

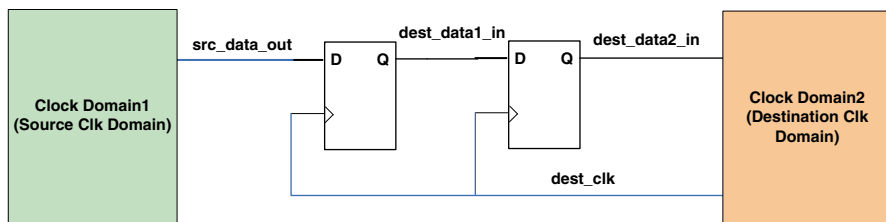


Fig. 3.7 Two stage synchronization circuit

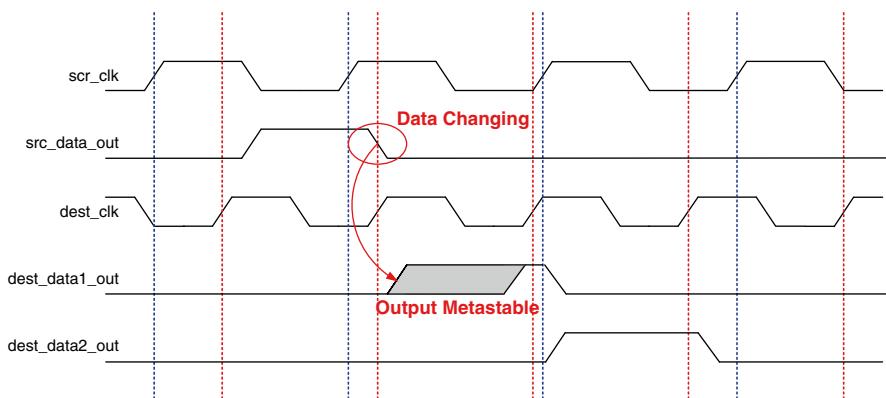


Fig. 3.8 Timings for the two-stage synchronizer circuit

3.4.3.1 Transfer of Control Signals (Synchronization)

If an asynchronous signal is directly fed to several parallel flip-flops in a design, the probability that a metastable event will occur greatly increases because there are more flip-flops that could become metastable. To avoid such a condition of metastability, the output of the synchronizing flip-flop is used rather than the asynchronous signal.

To reduce the effects of metastability, designers most commonly use a multiple stage synchronizer in which two or more flip-flops are cascaded to form a synchronizing circuit shown in Fig. 3.7.

If the first flop of a synchronizer produces a metastable output, the metastability may get resolved before it is sampled by the second flip flop. This method does not guarantee that the output of the second flip-flop will go metastable but it does decrease the probability of metastability. Adding more flos to the synchronizer will further reduce the probability of metastability.

One drawback or more aptly called “necessary evil” of the synchronizer circuit is that it adds up clocks to the total latency of the circuit.

The timings of the synchronizer circuit have been shown in Fig. 3.8.

The asynchronous output (*src_data_out*) from the Source Clock Domain working on *src_clk* is fed to the first synchronizer flip-flop. The signal *dest_data1_in*

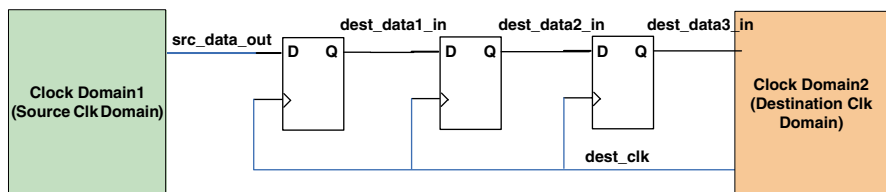


Fig. 3.9 Three stage synchronization circuit

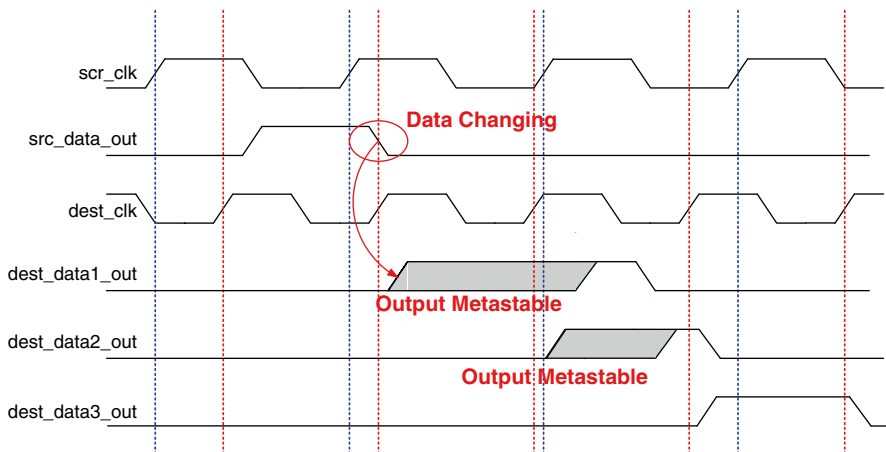


Fig. 3.10 Timings for the three-stage synchronizer circuit

(output of first synchronizing flip-flop) goes metastable but resolves to a stable state before it is sampled by the second flip-flop in the synchronizer circuit. Thus the signal *dest_data2_in* (output of the second synchronizer flip-flop) is synchronized to the *dest_clk* used by the Destination Clock Domain.

In some cases, it may happen that the output of the first synchronizer flip-flop takes longer than one clock to resolve from a metastable state to a stable state which means output of the second synchronizer flip-flop also goes metastable. In such cases, it is safe to use a three-stage synchronizer circuit.

A three-stage synchronizer circuit is shown in Fig. 3.9.

A three-stage synchronizer comprises of three cascaded flip-flops. As the second flip-flop output goes metastable, it resolves to a stable state before it is sampled by the third flip-flop.

Timing of the three-stage synchronizer circuit has been shown in Fig. 3.10.

The asynchronous output (*src_data_out*) from the source module working on *src_clk* is fed to the first synchronizer flip-flop. The signal *dest_data1_in* (output of first synchronizing flip-flop) goes metastable but resolves to a stable state after more than one clock duration. During the time the second flip-flop samples the

output of the first flip-flop, the signal *dest_data2_in* (output of the second synchronizer flip-flop) also becomes metastable. As shown this signal resolves to a stable state before it is sampled by the third flip-flop. Thus the asynchronous output of the *src_clk* domain is now synchronized to the *dest_clk* domain.

However, a two-stage synchronizer circuit is sufficient to avoid metastability in most of the multiple clock designs. A three-stage synchronizer is required in designs where the clock frequencies are very high.

3.4.3.2 Transfer of Data Signals

Multiple clock design often requires data transfer from one clock domain to other clock domain. Below are the two commonly used methods for Data Synchronization between two clock domains.

- Handshake signaling method.
- Asynchronous FIFO

The above two techniques have been described in detail in Sects. 3.6 and 3.8.

3.5 Synchronous Clock Domain Crossing

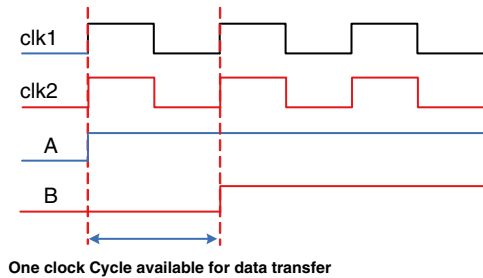
Before we move on to the next section on various methods to transfer data between two asynchronous clock domains, let's look at the various types of synchronous clock domain crossings in this section.

Clocks originating from the same clock-root and having a known phase and frequency relationship between them are known as synchronous clocks. A clock crossing between such clocks is known as a synchronous clock domain crossing. It can be divided into several categories based on the phase and frequency relationship of the source and destination clocks as follows:

- Clocks with the same frequency and zero phase difference
- Clocks with the same frequency and constant phase difference
- Clocks with different frequency and variable phase difference
 - Integer multiple clocks
 - Rational multiple clocks

The following section assumes the same phase and frequency jitter between the two clocks and paths between them are assumed to be balanced with the same specifications of clock latency and skew. It is also assumed that the clocks begin with a zero phase difference between them and the “clock to Q” delay of the flops is zero.

Fig. 3.11 Clocks with the same frequency and phase



3.5.1 Clocks with the Same Frequency and Zero Phase Difference

This scenario refers to two identical clocks “clk1” and “clk2” having the same frequency and zero phase difference. Clocks “clk1” and “clk2” being identical and generated from the same root clock, the data transfer from “clk1” to “clk2” would not be a clock domain crossing. For all practical purposes, this is the case of a single clock design and is considered here for completeness.

One complete clock cycle of “clk1” (or “clk2”) is available for data capture whenever data is transferred from clock “clk1” to “clk2”, as shown in Fig. 3.11.

As long as the combinational logic delay between the source and destination flops is such that the setup and hold time of the circuit can be met, the data will be transferred correctly. The only requirement here is that the design should be STA (static timing analysis) clean. In that case, there will be no problem of metastability, data loss or data incoherency.

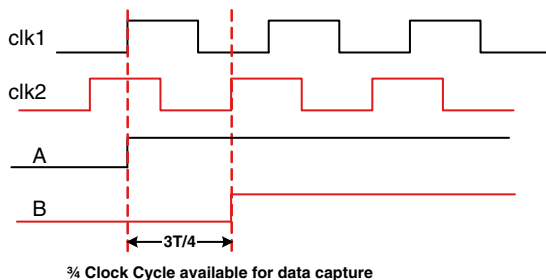
3.5.2 Clocks with the Same Frequency and Constant Phase Difference

These are the clocks having the same time period but a constant phase difference. A typical example is the use of a clock and its inverted clock. Another example is a clock that is phase shifted from its parent clock, for example by $T/4$ where T is the time period of the clocks.

As shown in Fig. 3.11, clocks “clk1” and “clk2” have the same frequency but are phase shifted where “clk1” is leading “clk2” by $3 T/4$ time units as shown in Fig. 3.12.

Whenever data is transferred from clock “clk1” to “clk2”, there is tighter constraint on the combinational logic delay due to smaller setup/hold margins. If the logic delay at the crossing is such that the setup and hold time requirements can be met at the capture edge, data will be transferred properly and there will be no metastability. In all such cases, there is no need for a synchronizer. The only requirement here is that the design should be STA clean.

Fig. 3.12 Clocks with same frequency but phase shifted



This condition is commonly created in STA to meet the timings. By adding skew between the launch and capture edge (i.e. clock will have same frequency but different phase) can help in meeting the timing requirements if the combo logic has more delay.

3.5.3 Clocks with the Different Frequency and Variable Phase Difference

Such clocks have a different frequency and a variable phase difference. There can be two sub-categories here, one where the time period of one clock is an integer multiple of the other and a second where the time period of one clock is a non-integer (rational) multiple of the other. In both cases, the phase difference between the active edges of clocks is variable. These two cases are described in detail below.

3.5.3.1 Clocks with Integral Multiple Frequencies

In this case, the frequency of one clock is an integer multiple of the other and the phase difference between their active edges is variable. Here the minimum possible phase difference between the active edges of the two clocks would always be equal to the time period of the fast clock.

In Fig. 3.13 clock “*clk1*” is three times faster than clock “*clk2*”. Assuming T is the time period of clock “*clk1*”, the time available for data capture by clock “*clk2*” could be T , $2T$ or $3T$ depending on which edge of clock “*clk1*” the data is launched. Hence, the worst case delay of any path should meet the setup time with respect to the edge with a phase difference of T . The worst case hold check would be made with respect to the edge with zero phase difference.

In all such cases, one complete cycle of the faster clock would always be available for data capture, hence it would always be possible to meet the setup and hold requirements. As a result there would be no metastability or data incoherency and hence a synchronizer would not be needed.

Since the data here is being launched on faster clock and captured on slower clock, to avoid data loss, the source data should be held constant for at least one

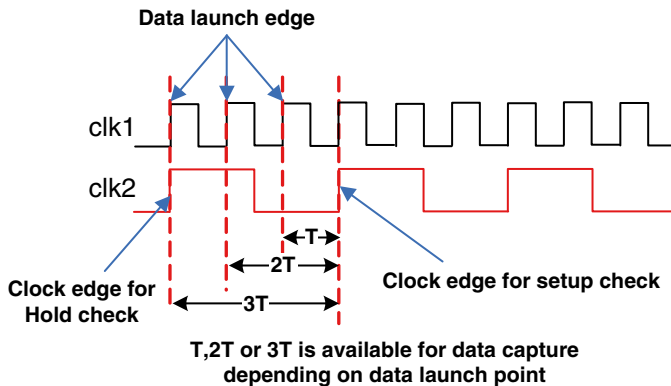


Fig. 3.13 Clocks with integral multiple frequencies

cycle of the destination clock. This can be ensured by using some control circuit, for example, a simple finite state machine (FSM). With reference to Fig. 3.13, if the source data is changed once in every three cycles of the source clock, there would be no data loss.

3.5.3.2 Clocks with Non-integral Multiple Frequencies

In this case, the frequency of one clock is a non-integer multiple of the other clock and the phase difference between the active clock edges is variable.

Unlike the situation where one clock is an integer multiple of the other, here the minimum phase difference between the two clocks can be small enough to cause metastability. Whether or not a metastability problem will occur depends on the value of the rational multiple, and the design technology. Three different cases are being considered here.

In the first case, there is a sufficient phase difference between the active edges of the source and destination clocks such that there will be no metastability.

In the second case, the active clock edges of the two clocks can come very close together, close enough to cause metastability problem. However, in this case the frequency multiple is such that, once the clock edges come close together, there would be sufficient margin in the next cycle to capture data properly without any setup or hold violation.

In the third case, the clock edges of the two clocks can be close enough for many consecutive cycles. This is similar to the behavior of asynchronous clocks except that here the clock-root for both the clocks is the same and hence the phase difference between the clocks can be calculated.

Note that in all the examples given below, some delay values are used and it is assumed that a phase margin of less than or equal to 1.5 ns between the clock edges can cause metastability. This is just a placeholder value and in real designs, it would be a function of many things including technology used, flip-flop characteristics, etc.

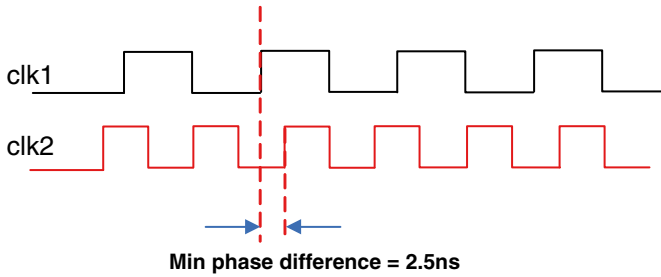


Fig. 3.14 Clock edges far apart thus avoiding metastability

Example 1

This is the case when the active clock edges of both the clocks will never come very close together, and in all cases there would be a sufficient margin to meet the setup and hold requirements of the circuit.

Consider a clock "*clk*" from which two clocks "*clk1*" and "*clk2*" are derived with a frequency of divide-by-3 and divide-by-2 respectively with respect to clock "*clk*". Here clock "*clk1*" is 1.5 times slower than clock "*clk2*". As shown in Fig. 3.14, the time period of clock "*clk1*" is 15 ns and of "*clk2*" is 10 ns. The least possible phase difference between the two clock edges is 2.5 ns which should be sufficient to meet setup and hold time requirements.

However, additional combinational logic should be avoided at the crossing due to the very small phase difference. For any additional logic added, it must meet the setup and hold time requirements to avoid any metastability and thus the synchronizer requirement.

Further, there would be no data loss for a slow to fast crossing but logic needs to be added to ensure that the signal is sampled once in the fast clock domain. However, in case of a fast to slow clock crossing, there can be data loss. In order to prevent this, the source data needs to be held constant for at least one cycle of the destination clock so that at least one active edge of the destination clock arrives between two consecutive transitions on the source data.

Example 2

In this case, the active clock edges of both the clocks can come very close together intermittently. In other words, the clock edges come close together once with sufficient margin between the edges for the next few cycles (to capture data properly) before they come close again. Here the word "close" implies close enough to cause metastability.

In Fig. 3.15, clocks "*clk1*" and "*clk2*" have time periods 10 and 7 ns respectively. Notice, that the minimum phase difference between the two clocks is 0.5 ns, which is very small. So, there are chances of metastability and a synchronizer would be required.

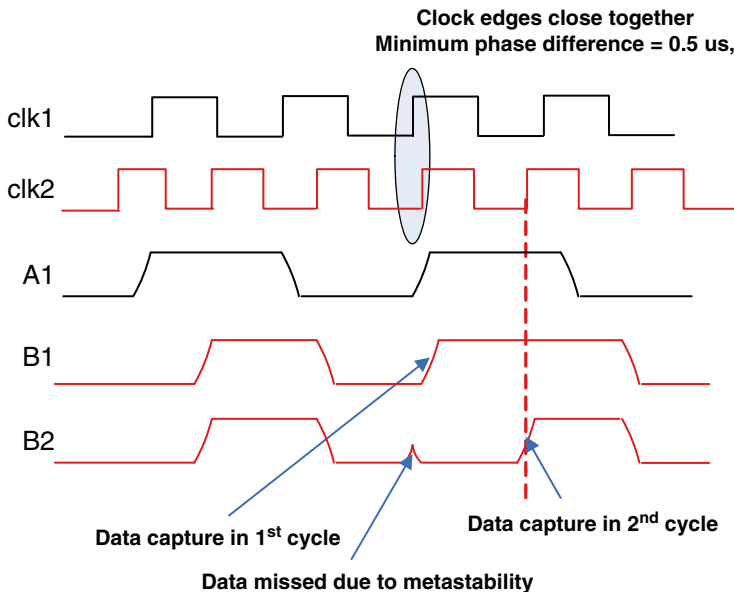


Fig. 3.15 Clock edges come together intermittently

Due to metastability, the data will not be captured correctly in the destination domain when the clock edges are very close together. However, in this case, note that once the clock edges come very close together, in the next cycle there is a sufficient margin so that the data can be captured properly by the destination clock. This is shown by signal “B2” in Fig. 3.15. While the expected output would be “B1”, the actual waveform could look like “B2”. Note that there is still no data loss in this case but may have some data incoherency issues.

For a fast to slow crossing, data loss can occur, and in order to prevent this, the source data should be held constant for a minimum of one destination clock cycle. Again, this can be done by the use of a simple FSM.

Example 3

This is the case when the phase difference between the clocks can be very small at times and can remain like that for several cycles. This is very similar to asynchronous clocks except that the variable phase differences will be known and will repeat periodically.

In Fig. 3.16, clocks “clk1” and “clk2” have time periods 10 and 9 ns respectively. It can be seen that the active clock edges of both the clocks come very close together for four consecutive cycles. In the first two cycles there is a possibility of a setup violation (as the source clock is leading the destination clock) and in the next two cycles there is a possibility of hold violation (as the destination clock is leading the source clock).

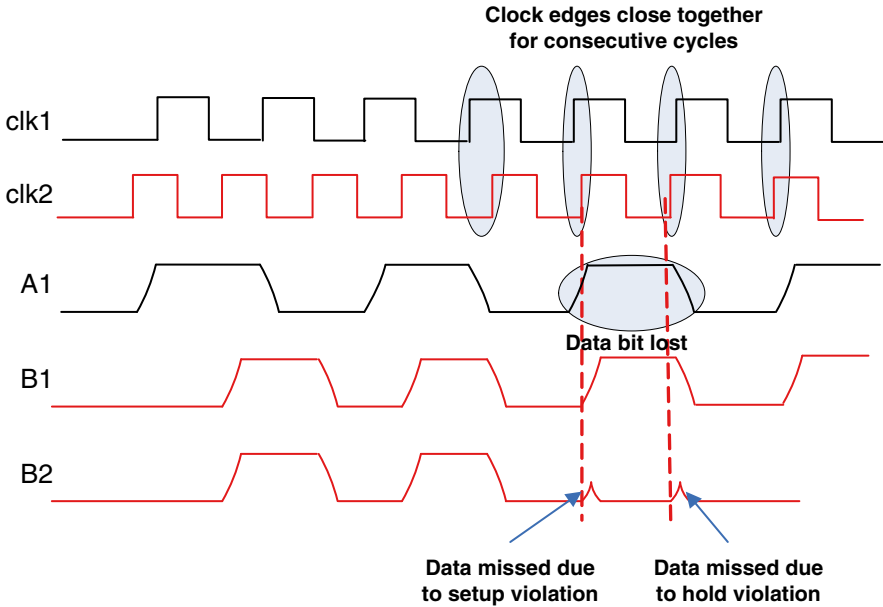


Fig. 3.16 Clock edges close together for consecutive cycles

In this case, there will be an issue of metastability and hence synchronization needs to be done. Apart from metastability there can be an issue of data loss also, even though it is a slow to fast clock domain crossing. As can be seen from Fig. 3.16, “B1” is the expected output if there would have been no metastability. However, the actual output can be “B2”. Here the data value ‘1’ is lost, because in the first cycle the value ‘1’ is not captured due to setup violation and in the second cycle the new value ‘0’ is incorrectly captured due to hold violation.

In order to prevent data loss, the data needs to be held constant for a minimum of two cycles of the destination clock. This is applicable for both fast to slow as well as slow to fast clock domain crossings. This can be done by controlling the source data generation using a simple FSM. However, the data incoherency issue can still be there.

In such cases, standard techniques like handshake and FIFO are more useful to control data transfer as they will also take care of the data incoherency issue.

3.6 Handshake Signaling Method

Using Handshake signaling is one of the oldest methods used to pass on the data from one domain to other.

Figure 3.17 shows two-clock domain divided into two separate systems.

“System X” sends data to “System Y” based on the handshake signals “xack” and “yreq”.

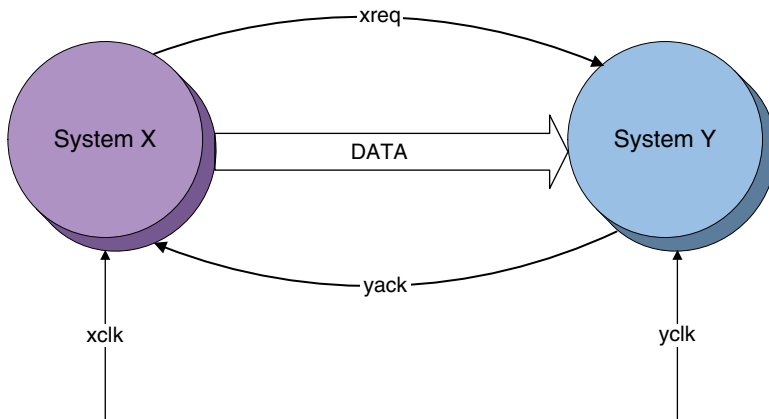


Fig. 3.17 Two clock domains divided into two separate systems

Below is the sequence for transfer of data with handshake signaling:

1. Transmitter “System X” places the data on the data bus and asserts “*xreq*” (request) signal indicating valid data on the data bus of the receiver “System Y”.
2. “*xreq*” signal is synchronized with the receiver clock domain “*yclk*”.
3. Receiver latches the data on the data bus on recognition of synchronized “*xreq*” signal “*yreq2*”.
4. Receiver asserts the Acknowledge “*yack*” signal, indicating that it has accepted the data.
5. Receiver Acknowledge signal “*yack*” is synchronized to the transmitter clock “*xclk*”.
6. Transmitter places the next data on the data bus when it recognizes the synchronized acknowledge signal “*xack2*”.

Timing for the handshake signaling sequence is shown in Fig. 3.18.

As shown in Fig. 3.18, it takes five clocks to transfer single data from Transmitter to the Receiver safely.

3.6.1 Requirements for Handshake Signaling

Data should be stable for atleast two rising edge clocks in the Transmitting clock domain.

Width of the Request signal “*xreq*” should be more than two rising edge clocks else this signal won’t get captured if passed from faster clock domain to slower clock domain.

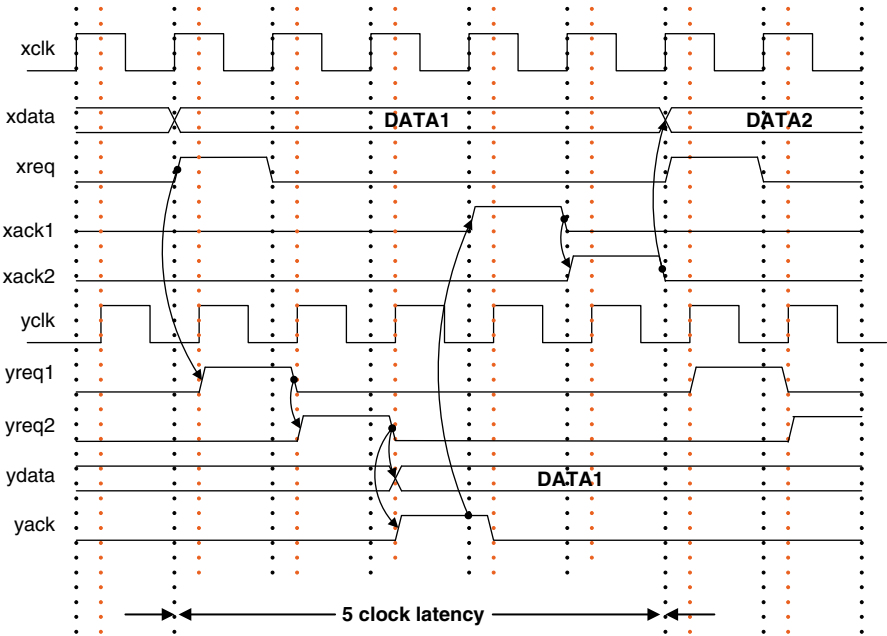


Fig. 3.18 Timing for handshaking method of data transfer

3.6.2 Disadvantages of Handshake Signaling

“Latency” for a single data transfer from one clock domain to other is much more than using a FIFO (described in later sections) used for the same data transfer.

3.7 Data Transfer Using Synchronous FIFO

During a system design, there are several components that work on different frequencies, for example like the processor, peripherals, etc. and they might, at times, have their own clock crystal. First-In-First-Out (FIFO) queues play an important role in the exchange of data between such devices. FIFOs are simple memories that are used to queue up data transmitted over communication buses.

Thus, FIFOs are usually used for data transfer across different clock domains. This section describes a simple Synchronous FIFO Architecture where reading and writing is done on the same clock. Subsequent sections describes in detail concept and design of an Asynchronous FIFO where reading and writing done on different clock frequencies.

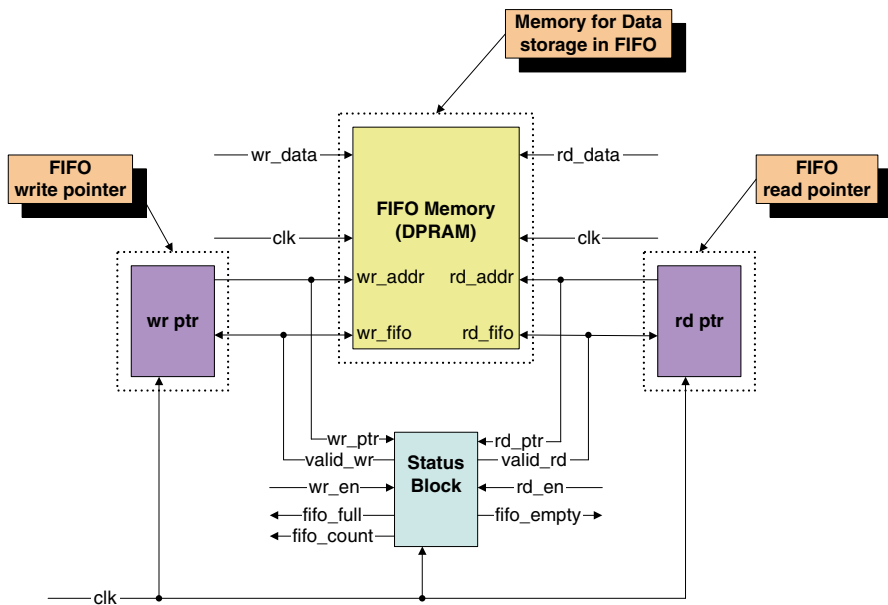


Fig. 3.19 Synchronous FIFO architecture

3.7.1 Synchronous FIFO Architecture

Figure 3.19 shows a general architecture of a Synchronized FIFO. DPRAM (Dual port RAM) is used as a FIFO memory to have an independent read and write [45].

The read and write ports have separate read and write addresses generated by two read and write pointers. The write pointer points to the location that will be written next and the read pointer to the location that will be read next. A valid write enable increments the write pointer and a valid read enable increments the read pointer.

Figure 3.19 shows a “**Status Block**” that generates the “*fifo_empty*” and “*fifo_full*” signals. If “*fifo_full*” is asserted it means that there is no room for more data to be written into the FIFO. Similarly “*fifo_empty*” indicates that there is no data available in the FIFO to be read by the external world. This block also indicates the number of empty or full locations in the FIFO at any point of time by performing some logic on the two pointers.

The **Dual Port Memory (DPRAM)** shown in Fig. 3.13 can have either synchronous reads or asynchronous reads. For synchronous read, an explicit read signal is supposed to be provided before the data at the output of the FIFO is valid. For asynchronous reads, DPRAM does not have registered outputs; valid data is available as soon as it is written (data is read first and then the pointer is incremented).

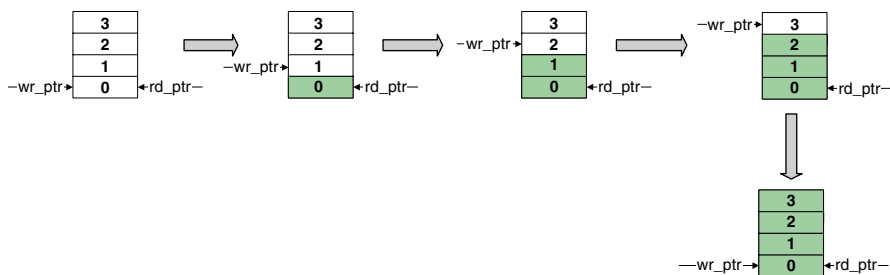


Fig. 3.20 FIFO full condition

3.7.2 Working of Synchronous FIFO

On reset both the read and write pointers are initialized to zero. Signal “*fifo_empty*” is asserted and “*fifo_full*” remains “low” during this time. Further reads from the FIFO are blocked when the FIFO is empty so only write operations are possible. Subsequent writes on the FIFO increments the write pointer and deasserts the “*fifo_empty*” signal. A point is reached where there is no room for more data and the write pointer becomes equal to $\text{RAM_SIZE} - 1$. At this time a write causes the write pointer to again roll back to zero, making “*fifo_full*” signal high.

To conclude FIFO is either full or empty when read-pointer equals to the write-pointer and so it is necessary to distinguish between these two conditions.

3.7.2.1 FIFO Full and Empty Generation

Figure 3.20 shows the FIFO full generation for a four deep synchronous FIFO.

All the transitions shown in Fig. 3.20 are in subsequent clocks. As shown in figure, FIFO becomes full when a write causes both the pointers to become equal in the next clock. This makes the following condition for assertion of “*fifo_full*” signal.

$$\text{fifo_full} = (\text{read_pointer} == (\text{write_pointer} + 1)) \text{ AND "write"} \quad (3.1)$$

Also shown below the sample Verilog code for the “*fifo_full*” logic.

```
always @ (posedge clk or nededge reset_n)
begin: fifo_full_gen
  if (~reset_n)
    fifo_full <= 1'b0;
  else if (wr_fifo && rd_fifo)
    ;//do nothing
  else if (rd_fifo)
    fifo_full <= 1'b0;
  else if (wr_fifo && (rd_ptr = wr_ptr + 1'b1))
    fifo_full <= 1'b1;
end
```

Similarly FIFO becomes empty when a read causes both the pointers to become equal in the next clock. This makes the following condition for assertion of “*fifo_empty*” signal.

$$\text{fifo_empty} = (\text{write_pointer} == (\text{read_pointer} + 1)) \text{ AND "read"} \quad (3.2)$$

Also shown below the sample Verilog code for the “*fifo_empty*” logic.

```
always @ (posedge clk or negedge reset_n)
begin: full_gen
  if (~reset_n)
    fifo_full <= 1'b1;
  else if ( wr_fifo && rd_fifo)
    ; //do nothing
  else if (wr_fifo)
    fifo_empty <= 1'b0;
  else if (rd_fifo && (rw_ptr = rd_ptr + 1'b1 ))
    fifo_empty <= 1'b1;
end
```

3.7.2.2 An Alternative Approach

An alternative approach for generating the “*fifo_full*” and “*fifo_empty*” conditions are by maintaining a counter that constantly indicates the number of full or empty locations left in the FIFO.

Width of the counter needs to be equal to the depth of the FIFO so as to store the maximum value. Counter is initialized to a value of zero on reset. Any subsequent writes increments the counter by one and any subsequent read decrements the counter by one.

Now FIFO empty condition can easily be generated when the counter values reaches “zero” and FIFO full condition when counter’s value equal the size of the FIFO.

Alternate approach that is mentioned in this section, though simple, is not efficient as compared to the one mentioned in Sect. 3.7.2.1 since it requires additional hardware (comparators) for the generation of FIFO empty and FIFO full conditions. As the depth of the FIFO increases, so does the width of the counter; thus requiring higher order comparators for FIFO empty and FIFO full condition generation. This finally lowers the maximum frequency of operation of the FIFO.

3.8 Asynchronous FIFO (or Dual Clock FIFO)

Asynchronous FIFO is used to transfer data across two asynchronous clock domains.

Figure 3.21 shows two systems “*System X*” and “*System Y*” where data from “*System X*” is supposed to be transferred to “*System Y*”, both systems working at different clock domains.

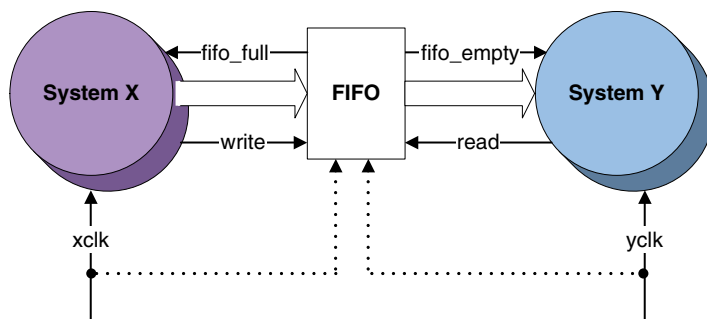


Fig. 3.21 Data transfer with asynchronous FIFO

“System X” writes the data on “xclk” clock into FIFO and is read out by “System Y” on “yclk” clock.

“FIFO full” and “FIFO empty” signals take care of the underflow and overflow conditions.

Overflow condition is taken care by “FIFO full” signal, i.e. Data is not written into the FIFO if “FIFO Full” signal is asserted else Data will be overwritten.

Underflow condition is taken care by “FIFO empty” signal, i.e. Data is not read from the FIFO if “FIFO empty” signal is asserted else junk Data would be read.

Unlike handshake signaling, Asynchronous FIFO is used in case of performance critical designs where clock latency is a factor rather than system resources.

As mentioned in Sect. 3.7, simple Synchronous FIFO can be implemented using Dual Port RAM with separate ports for read and write operations where reading and writing is done on the same clock. The same concept can be extended for designing Asynchronous FIFO with special care taken for FIFO empty and FIFO Full signal generation to avoid metastability conditions.

3.8.1 Avoid Using Binary Counters for the Pointer Implementation

Take the case of write pointer. Write pointer is always incremented on write clock whenever there is a valid write request to a FIFO. Similarly Read pointer is incremented on read clock whenever there is a valid read request. Now for FIFO Full signal generation, write pointer needs to be compared with read pointer and since both the pointers are synchronous to their respective clock but asynchronous to each other will result in wrong sampling of the pointer values for comparison if binary counters are used for the pointer implementation. This is illustrated as shown below.

Say, the binary counter is changing from FFF to 000. In this case all the bits will change at the same time. Metastability can be avoided by synchronizing the counter, but this may still get sampled values that are widely off the mark, so synchronizing the counters is not the final solution.

Table 3.1 Counter encoding in Gray

Gray/reflected code	Decimal equivalent
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

Possible transitions from FFF to 000:

- FFF → 000
- FFF → 001
- FFF → 010
- FFF → 011
- FFF → 100
- FFF → 101
- FFF → 110
- FFF → 111

If the synchronizing clock edge comes in the middle of the transition from FFF to 000, it is possible that any of the three bit binary value be sampled and synchronized in new clock domain.

Since the generation of FIFO full and FIFO empty flags depends on these pointer values, incorrect value of these pointers will result in wrong triggering of the flags. There might be a case where FIFO full flag not getting triggered even when actually FIFO is full resulting in data getting lost or FIFO empty flag not getting triggered resulting in junk data being read.

Note: Looking at the above case, it's highly recommended to avoid using binary counter for read and write pointers implementation.

3.8.2 Use Gray Coding Instead of Binary for the Counters

One way of implementing FIFO pointers is to make them count in Gray-code, as shown in Table 3.1.

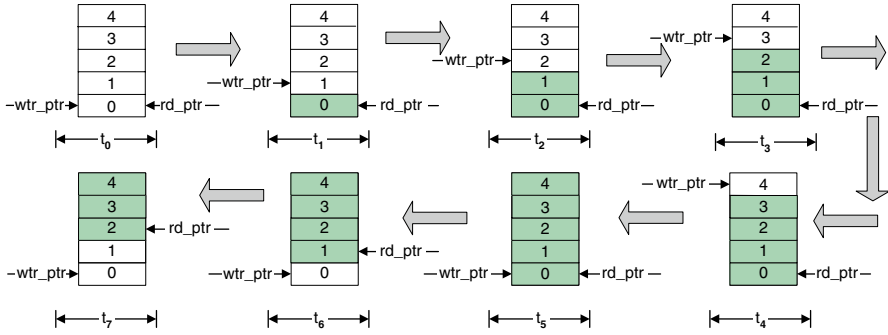


Fig. 3.22 Effect of synchronization on FIFO full logic

The advantage of the Gray code over pure binary numbers is that a number in Gray code changes by one bit as it proceeds from one number to the next.

To obtain a different Gray code, one can start with any bit combination and proceed to obtain the next bit combination by changing only one bit from 0 to 1 or 1 to 0 in any desired random fashion, as long as two numbers do not have identical code assignments. The Gray code is also known as *reflected* code.

Since gray code is a unit distance code, every next value differs from previous in one bit position, will result in a maximum of a single bit error/transition. For example if the counter changes from “1010” to “1011”, the sampling logic will either read “1010” (old value) or “1011” (new incremented value) but no other value.

Note: Synchronizing gray counter will rarely result in sampled counter value getting metastable and secondly the value sampled will have at most one bit error.

3.8.2.1 Effect of Synchronization of Pointers

Further accesses to the FIFO should be blocked incase of FIFO Full condition. To calculate the FIFO full condition, the read and write pointer that are incremented on their respective clocks have to be compared. The read pointer (Gray coded) needs to be synchronized to the write clock. Let take this with an example.

As shown in Fig. 3.22, initially read and write pointers are zero at t_0 with FIFO empty. As subsequent write takes place on FIFO, write pointer gets incremented. A stage is reached when write pointer equals read pointer and FIFO becomes FULL. This happens at t_5 as shown in Fig. 3.23.

Now incase a read takes place at t_6 , since a typical synchronizer circuit consists of atleast two flip flops, synchronizing read pointer on write clock will result in changed read pointer reflected after two write clocks. This results in blocking additional writes on the FIFO for additional cycles but is harmless. It would have been a problem if writes were not blocked when the FIFO was actually full.

Similarly further read access to FIFO should be blocked when the FIFO is Empty.

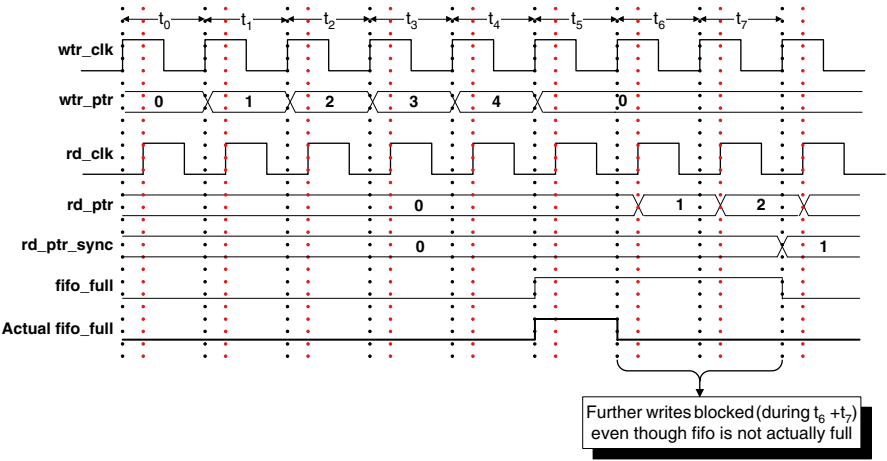


Fig. 3.23 FIFO full timings

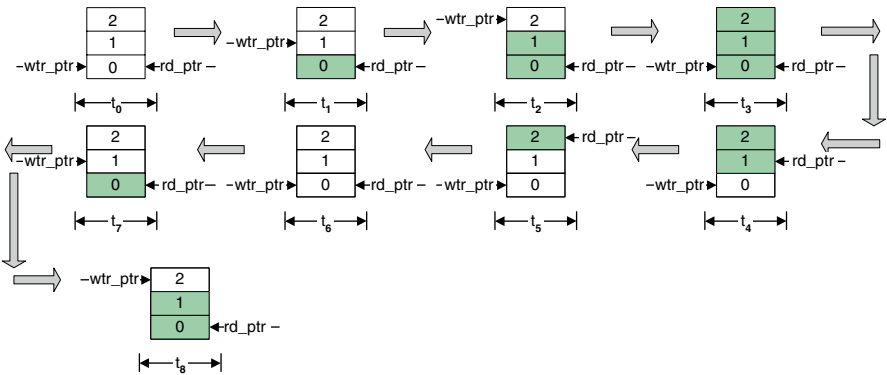


Fig. 3.24 Effect of synchronization on FIFO empty logic

For the FIFO Empty calculation, write pointer is synchronized to the read clock and compared against the read pointer. Due to this, read side sees delayed writes (two clock delayed signal), and would still indicate FIFO empty even though it actually has some data. This will result in reads getting blocked till the writes becomes visible to the read side.

As shown in Fig. 3.24, initially read and write pointers are zero at t_0 with FIFO empty. As subsequent write takes place on FIFO, write pointer is incremented. A stage is reached when write-pointer equals to read-pointer and FIFO becomes FULL. This happens at t_3 as shown in Fig. 3.25.

Subsequent reads starts at t_4 and again FIFO becomes empty at t_6 . FIFO is again written back at t_7 and t_8 , now since a typical synchronizer circuit consists of at least two flip flops, synchronizing write pointer on read clock will result in changed write pointer reflected after two read clocks. This results in blocking additional reads on

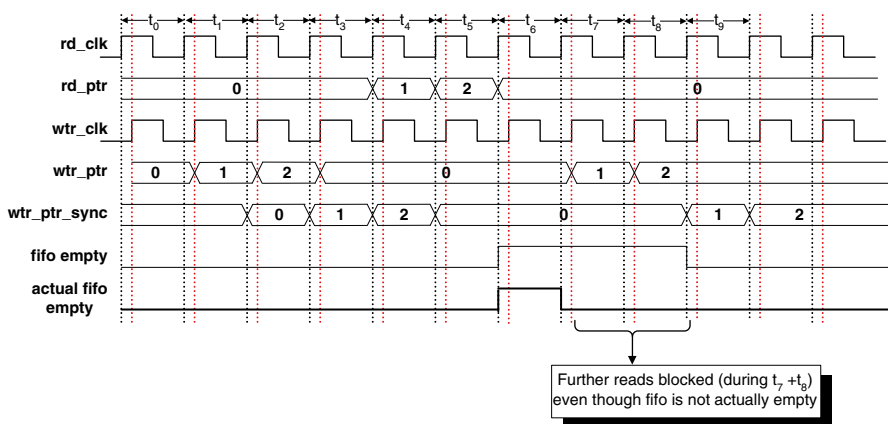


Fig. 3.25 FIFO empty illusion

FIFO and is harmless. It would have been a problem if reads were not blocked when the FIFO was actually empty.

Note: Reporting to the write side that FIFO is full when it is not is fine, and so is reporting to the read side that the FIFO is empty when it is not. Even if the synchronized values of the pointer (synchronized read pointer during write and synchronized write pointer during read) remains metastable for a small period of time, the effect would be to block writes/reads causing the FIFO to hang for a while, but not causing any errors.

3.8.3 Gray Code Implementation of FIFO Pointers

Both Read and Write pointer values need to be correctly sampled for perfect generation of FIFO empty and FIFO full conditions. The best way for passing pointers between clock domains is to use a gray code counter for pointers implementation, since they would eliminate most of the errors if the synchronized clock signal comes in the middle of the counter transition [45].

Designing a gray code counter seems quite complex but is indeed simple. All that is supposed to be done is the following:

- STEP I : Convert the Gray value to Binary value.
- STEP II : Increment the Binary value depending on some condition.
- STEP III: Convert the Binary value back to Gray.
- STEP IV: Store the final Gray value of the counter in a register.

Figure 3.26 shows the generalized Gray counter.

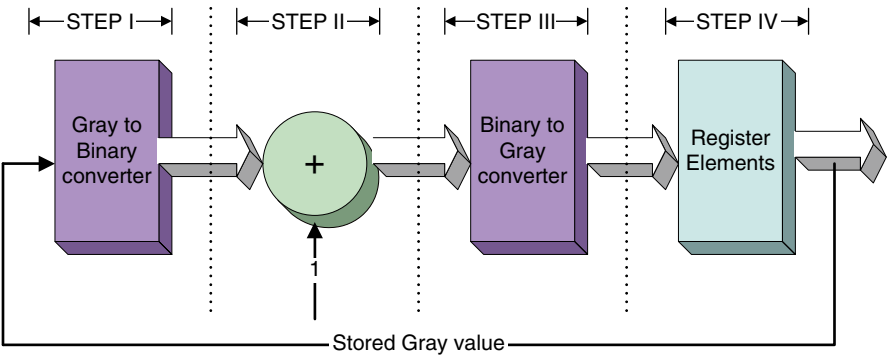


Fig. 3.26 Gray counter using binary adder

Table 3.2 Counter increment in gray/binary

Gray value	Binary value	Equivalent decimal value
0000	0000	0
0001	0001	1
0011	0010	2
0010	0011	3
0110	0100	4
0111	0101	5
0101	0110	6
0100	0111	7
1100	1000	8
1101	1001	9
1111	1010	10
1110	1011	11
1010	1100	12
1011	1101	13
1001	1110	14
1000	1111	15

3.8.3.1 Gray to Binary Converter

Table 3.2 shows the four-bit counter values when counted in Gray and binary. Subsequent rows in a particular column show the transition values of the counter when incremented on clock.

The equation for the Gray to Binary conversion:

$$\text{bin}_{n-1} = \text{gray}_{n-1} \tag{3.3}$$

$$\text{bin}_i = \text{gray}_i \oplus \text{bin}_{i+1} \tag{3.4}$$

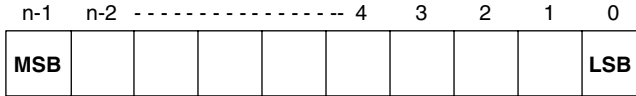


Fig. 3.27 Bit numbering of the counter

where $i < n - 1$, in an n bit counter value.

Figure 3.27 shows the bit numbering of the counter.

Let us take a simple example of converting gray value “1010” into its binary equivalent.

Taking $n - 1 = 3$

Substituting the value of $i = 3$ in the Eq. 3.3 above we have

$$\text{bin}_3 = \text{gray}_3 = \text{gray}[3] = 1$$

Substituting the value of $i = 2$ in the Eq. 3.4 above we have

$$\text{bin}_2 = \text{gray}_2 \oplus \text{bin}_3 = \text{gray}_2 \oplus \text{gray}_3 = \text{gray}[2] \oplus \text{gray}[3] = 1$$

Substituting the value of $i = 1$ in the Eq. 3.4 above we have

$$\begin{aligned} \text{bin}_1 &= \text{gray}_1 \oplus \text{bin}_2 = \text{gray}_1 \oplus \text{gray}_2 \oplus \text{gray}_3 \\ &= \text{gray}[1] \oplus \text{gray}[2] \oplus \text{gray}[3] = 0 \end{aligned}$$

Substituting the value of $i = 0$ in the Eq. 3.4 above we have

$$\begin{aligned} \text{bin}_0 &= \text{gray}_0 \oplus \text{bin}_1 = \text{gray}_0 \oplus \text{gray}_1 \oplus \text{gray}_2 \oplus \text{gray}_3 \\ &= \text{gray}[0] \oplus \text{gray}[1] \oplus \text{gray}[2] \oplus \text{gray}[3] = 0 \end{aligned}$$

So, we have the following four equations:

$$\text{bin}[0] = \text{gray}[0] \oplus \text{gray}[1] \oplus \text{gray}[2] \oplus \text{gray}[3] \quad (3.5)$$

$$\text{bin}[1] = \text{gray}[1] \oplus \text{gray}[2] \oplus \text{gray}[3] \quad (3.6)$$

$$\text{bin}[2] = \text{gray}[2] \oplus \text{gray}[3] \quad (3.7)$$

$$\text{bin}[3] = \text{gray}[3] \quad (3.8)$$

Based on the above equations, final binary equivalent value for Gray value of “1010” is “1100”.

So from above equations it's clear that $\text{bin}[3]$ can be generated by right shifting gray value by 3, $\text{bin}[2]$ by right shifting gray value by 2, $\text{bin}[1]$ by right shifting gray value by 1 and $\text{bin}[0]$ by right shifting gray value by 0 [43].

Below is the Verilog code of the above gray to binary converter.

```
module gray_to_bin (bin , gray);
    parameter SIZE = 4;
    input [SIZE] - 1:10] bin;
    output [SIZE] - 1:10] gray;
    reg [SIZE] - 1:10] bin;
    integer i;
    always @ (gray)
        for ( i = 0; i <= SIZE; i = i + 1 )
            bin[i] = ^(gray >> i);
endmodule
```

3.8.3.2 Binary to Gray Converter

Following are the equations for Binary to Gray conversion:

$$\text{gray}_{n-1} = \text{bin}_{n-1} \quad (3.9)$$

$$\text{gray}_i = \text{bin}_i \oplus \text{bin}_{i+1} \text{ where } i < n - 1 \quad (3.10)$$

Let us take a simple example of converting binary value “1100” back into its gray equivalent.

Taking $n - 1 = 3$

Substituting the value of $i = 3$ in the Eq. 3.9 above we have

$$\text{gray}_3 = \text{bin}_3 = \text{bin}[3] = 1$$

Substituting the value of $i = 2$ in the Eq. 3.10 above we have

$$\text{gray}_2 = \text{bin}_2 \oplus \text{bin}_3 = \text{bin}[2] \oplus \text{bin}[3] = 0$$

Substituting the value of $i = 1$ in the Eq. 3.10 above we have

$$\text{gray}_1 = \text{bin}_1 \oplus \text{bin}_2 = \text{bin}[1] \oplus \text{bin}[2] = 1$$

Substituting the value of $i = 0$ in the Eq. 3.10 above we have

$$\text{gray}_0 = \text{bin}_0 \oplus \text{bin}_1 = \text{bin}[0] \oplus \text{bin}[1] = 0$$

This gives the same gray value “1010” of the given binary value “1100”.

Based on above example, we have the following four equations:

$$\text{gray}[0] = \text{bin}[0] \oplus \text{bin}[1] \quad (3.11)$$

$$\text{gray}[1] = \text{bin}[1] \oplus \text{bin}[2] \quad (3.12)$$

$$\text{gray}[2] = \text{bin}[2] \oplus \text{bin}[3] \quad (3.13)$$

$$\text{gray}[3] = \text{bin}[3] \quad (3.14)$$

As inferred from Eqs. 3.11–3.14, the equivalent gray value can be obtained by performing bit wise exclusive or operation between the binary value and its right shift version as shown below:

bin[3]	bin[2]	bin[1]	bin[0]	→ binary value: <i>bin</i>
0	bin[3]	bin[2]	bin[1]	→ right shift (<i>bin</i>)

gray[3]	gray[2]	gray[1]	gray[0]	→ equivalent gray value
---------	---------	---------	---------	-------------------------

Below is the Verilog code of the above binary to gray converter

```
module bin_to_gray (bin, gray);
    parameter SIZE = 4;
    input [SIZE-1:0] bin;
    output [SIZE-1:0] gray;
    assign gray = (bin >> 1) ^ bin;
endmodule
```

3.8.3.3 Gray code counter implementation

It is a combination of all the four steps shown in the Fig. 3.26 (gray to binary converter, adder, binary to gray converter and finally sets of register elements to store the gray value).

Below is the Verilog code for the Gray code counter:

```
module gray_counter (clk, gray, inr, reset_n)
    parameter SIZE = 4;
    input clk, inr, reset_n;
    output [SIZE - 1] gray;
    reg [SIZE] - 1 gray_temp, gray, bin_temp, bin;
    integer i;

    always @ (gray or inr)
    begin:gray_bin_gray
        for (i = 0; i < SIZE ; i = i + 1)
            bin[i] = ^(gray >> i);           // gray to binary conversion
        bin_temp = bin + inr;                 // addition in binary
        gray_temp = bin_temp >> 1 ^ bin_temp; // binary to gray conversion
    end

endmodule
```

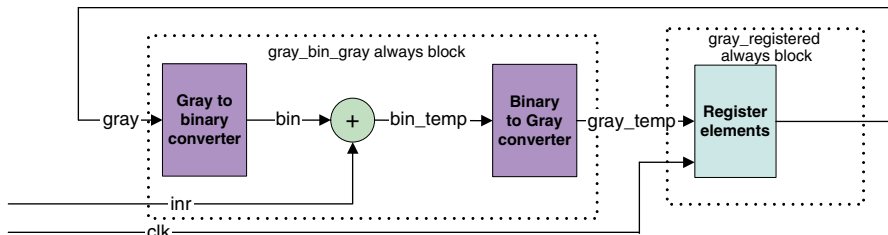


Fig. 3.28 Gray counter logic

The always block below show the registering of the converted gray value.

```
always @ (posedge clk or negedge reset_n)
begin:gray_registered
if (~reset_n)
    gray <= {SIZE {1'b0}};
else
    gray <= gray_temp;
end
```

Figure 3.28 shows the logical diagram for the above gray counter code.

3.8.4 FIFO Full and FIFO Empty Generation

N bit pointer can address 2^N locations in a FIFO. Since FIFO may be either empty or full when both pointers are equal, an extra bit is required to differentiate between these two conditions.

The FIFO is full when the most significant bits of the binary versions of the pointers differ and the remaining N bits are equal.

The FIFO is empty when the binary versions of the pointers are exactly equal in all bit positions. The following section shows this with an example:

Consider an eight deep FIFO. Three bits are required to address all its eight locations with an additional bit to distinguish between FIFO full and FIFO empty condition. Initially both *rd_ptr_bin* and *wr_ptr_bin* are "0000" and the FIFO is empty. Now after eight subsequent writes to FIFO we have the following values of read and write pointer:

$$\begin{aligned} rd_ptr_bin &= "0000" \\ wr_ptr_bin &= "1000" \end{aligned}$$

That is the FIFO full condition as shown in the Fig. 3.29 [45].

Fig. 3.29 FIFO full condition

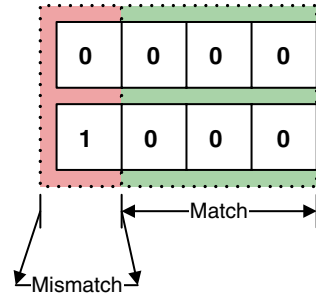
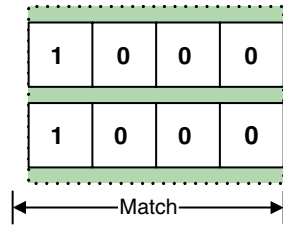


Fig. 3.30 FIFO empty condition



Now on subsequent eight reads, we have the following values for the read and write pointer:

$$\begin{aligned} rd_ptr_bin &= "1000" \\ wr_ptr_bin &= "1000" \end{aligned}$$

That is the FIFO empty condition as shown in the Fig. 3.30.

Figure 3.31 shows the block diagram showing FIFO empty and FIFO full generation:

In this case, maximum frequency of operation will depend on how fast gray code counters works since it requires chain of XOR gates.

As Read/Write pointer's value is stored in gray and all the comparisons, incrementing of pointers etc. is done in binary, it makes the implementation and debugging quite simpler. As shown in the Fig. 3.31, it requires four gray to binary converters, which can be avoided if the comparison etc. for calculation of FIFO empty and FIFO full generation are done directly in gray. This is somewhat complicated and it requires some additional logic. Let's see how this alternative approach works in our next section.

3.8.4.1 An Alternative approach for FIFO Full and FIFO Empty Generation

This approach requires creating two gray code counters, one of n bit and the other of $n - 1$ bit. The two counters can be created by a single n bit counter and then modifying its second MSB to generate $(n - 1)$ bit gray code counter with the same LSBs as of the n bit counter [43].

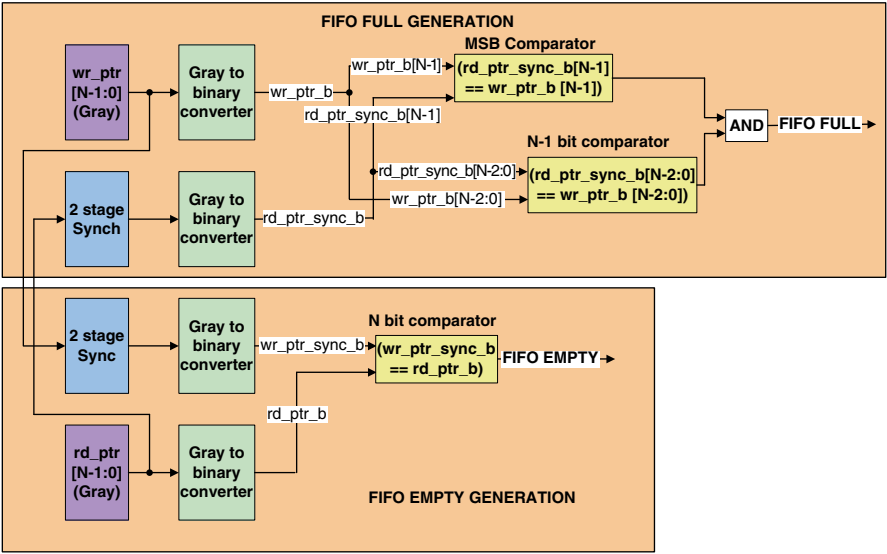


Fig. 3.31 FIFO full and empty signals generation hardware

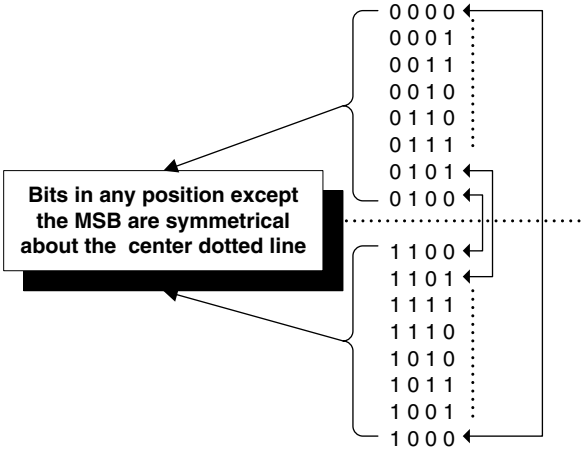


Fig. 3.32 Four-bit gray counter

Before we start up with the main logic let’s look up some more about Gray code counters.

Figure 3.32 shows four-bit Gray code counter.

As shown above in the figure, bits in any column except the MSB are symmetrical about the sequence mid-point. Thus the second half of the four-bit Gray code is a mirror image of the first half with the MSB inverted.

Fig. 3.33 Conversion of four-bit gray code to three-bit gray code

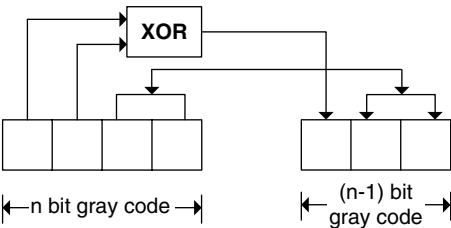


Table 3.3 Conversion of four-bit gray code to three-bit gray code

Four-bit gray code	Three-bit converted gray code
0000	000
0001	001
0011	011
0010	010
0110	110
0111	111
0101	101
0100	100
1100	000
1101	001
1111	011
1110	010
1010	110
1011	111
1001	101
1000	100

Now $(n - 1)$ bit Gray code can easily be generated by XORing the two MSBs of the n -bit Gray code to generate the MSB for the $(n - 1)$ bit gray code. Rest of the $(n - 2)$ bits can be simply using the $(n - 2)$ bits of the n -bit counter. Figure 3.33 shows the conversion of four-bit to three-bit Gray code (Table 3.3).

The usage for this dual n -bit Gray code counter for FIFO Empty/Full generation logic would be described in the next section on FIFO Design.

3.8.5 Dual Clock FIFO Design

Figure 3.34 shows the block diagram for the FIFO using a Dual port Memory as storage elements [45].

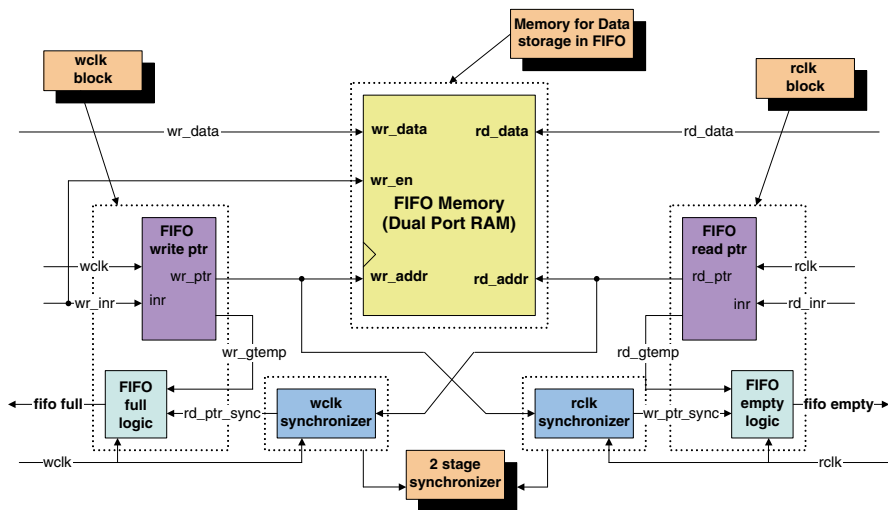


Fig. 3.34 Dual clock FIFO design

3.8.5.1 FIFO Empty Condition Generation

FIFO empty flag would be generated in the read clock domain immediately when the FIFO becomes empty that is when the read pointer matches up with the synchronized write pointer.

Note that both the read pointer and the synchronized write pointer are directly compared in gray unlike the previous implementation shown in Sect. 3.8.4. This saves four gray to binary converters that would have required if the pointers were first converted into their binary equivalent before comparison.

Similar to the previous implementation, the pointers are one bit larger than needed to address the FIFO memory. The synchronized write pointer (***wr_ptr_sync***) is compared against the ***rd_gtemp*** (the next gray code that will be registered in the ***rd_ptr***).

Below is the Verilog code for the above logic.

```
always @ (posedge rclk or negedge reset_n)
begin: fifo_empty_gen
  if (~reset_n)
    fifo_empty <= 1'b1;
  else
    fifo_empty <= (rd_gtemp == wr_ptr_sync);
end
```

Note: FIFO empty output generated is registered.

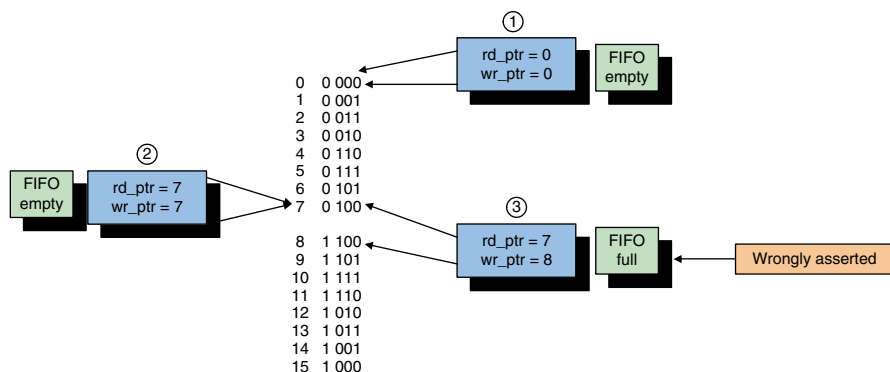


Fig. 3.35 FIFO full and FIFO empty conditions

3.8.5.2 FIFO Full Condition Generation

FIFO full flag would be generated in the write clock immediately when the FIFO becomes full that is when the write pointer matches up with the synchronized read pointer.

Note that both the write pointer and the synchronized read pointer are directly compared in gray.

Similar to the previous implementation, the pointers are one bit larger than needed to address the FIFO memory. The logic to generate this condition is different from previous implementation since pointers comparison is directly done in gray instead of binary.

Let's take this with an example.

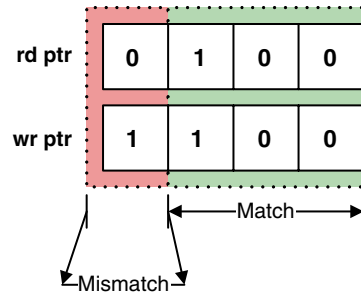
Figure 3.35 shows the steps performed on an eight depth FIFO.

- STEP 1: Initially FIFO is empty with “rd ptr” = “wr ptr” = 0 as shown as Fig. 3.35.
- STEP 2: Subsequent writes takes places on FIFO till the FIFO becomes full with “rd ptr” = 0 and “wr ptr” = 7. Now subsequent eight reads takes place with “rd ptr” = “wr ptr” = 7 and FIFO becomes empty (since all the bits of read and write pointer are equal) as shown in Fig. 3.35.
- STEP 3: A single write at this time would result in “rd ptr” = 7 and “write ptr” = 8. In case the same logic is used as in previous implementation (Sect. 3.8.4) using binary comparison, FIFO would again be indicated as Full, even though it is not (Fig. 3.36).

This condition can be easily taken care by using dual n-bit gray code counter.

The correct method to perform the full comparison is accomplished by synchronizing the “rd ptr” into the write clock domain. The MSBs are compared and should differ in case the write pointer has wrapped one more time than the synchronized read pointer. If the synchronized read pointer's MSB is high, the second MSB of the

Fig. 3.36 FIFO full condition



synchronized read pointer (*rd_ptr_sync*) is inverted before doing a comparison against a $(n - 1)$ bit write pointer.

So the FIFO full flag is asserted when all the following three condition below become true:

1. MSB of the synchronized read pointer (*rd_ptr_sync*) should differ from the MSB of the next gray code value of the write pointer (*wr_gtemp*) that will be registered in the *wr_ptr*.
2. Second MSB of the next gray code count in the write clock domain (*wr_gtemp*) should be equal to second MSB of the read pointer that has been synchronized into the write clock domain (*rd_ptr_sync*).
3. All the left out LSB's of the two pointers should match.

Note: The second MSB in (2) above is calculated by XORing the first two MSBs of the pointer. (Doing an exclusive-or operation of the two MSBs causes the second MSB to be inverted if the MSB is high).

Below is the Verilog code for the above logic.

```

wire rd_2nd_msb = rd_ptr_sync [SIZE] ^ rd_ptr_sync [SIZE - 1];
wire wr_2nd_msb = wr_gtemp [SIZE] ^ wr_gtemp [SIZE-1];
always @ (posedge wclk or negedge reset_n)
begin: fifo_full_gen
  if (~reset_n)
    fifo_full <= 1'b0;
  else
    fifo_full <= ((wr_gtemp [SIZE] != rd_ptr_sync[SIZE]) &&
      (rd_2nd_msb == wr_2nd_msb) &&
      (wr_gtemp[SIZE -2:0] == rd_ptr_sync[SIZE
        -2:0]));
end

```

References

1. Arora M, Bhargava P, Gupta S (2002) Handling multiple clocks (problems & remedies in designs involving multiple clocks). DCM Technologies, SNUG India
2. Cummings CE (2001) Synthesis and scripting techniques for designing multi-asynchronous clock designs. In: SNUG 2001 (Synopsys Users Group Conference), San Jose. User Papers

Chapter 4

Clock Dividers

4.1 Introduction

Typically most SoCs require a number of phase-related clocks to various components in the design. For the synchronous even division, the required clocks are generated by dividing the master clock by a power of 2. However, sometimes, it is desirable to divide a frequency by an odd or even fractional divisor. In these cases, no synchronous method exists without generating a higher frequency master clock.

Though dividing a clock by an even number always generates 50% duty cycle output, sometimes it is necessary to generate a 50% duty cycle frequency even when the input clock is divided by an odd or non-integer number.

This chapter provides guidelines and details to implement these unusual clock dividers.

The chapter starts up with simple dividers where the clock is divided by an odd number (Divide by 3, 5 etc) and then later expands it into non-integer dividers (Divide by 1.5, 2.5 etc). The circuits described are simple, efficient, cheaper and faster than any external PLL alternatives.

4.2 Synchronous Divide by Integer Value

A synchronous divide by integer can be easily specified using a Moore state machine.

Figure 4.1 shows a “Divide by 7” using Moore machine.

The logic though simple would not yield a 50% duty cycle output.

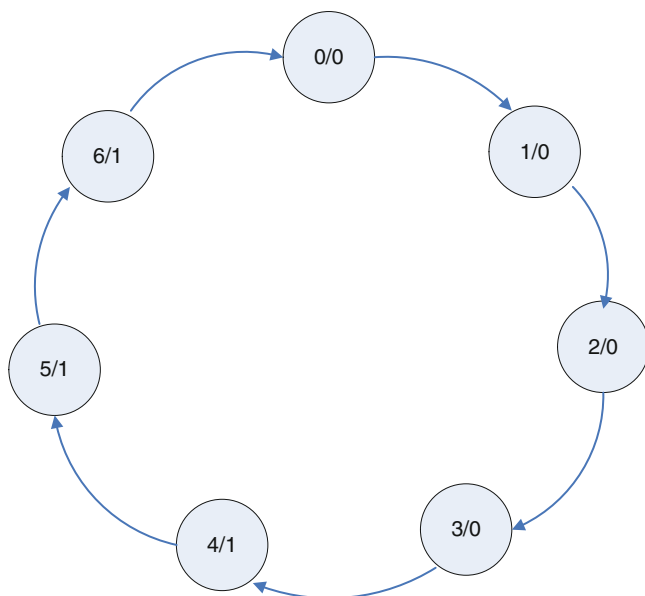


Fig. 4.1 Divide by 7 using Moore machine

4.3 Odd Integer Division with 50% Duty Cycle

Conceptually, the easiest way to create an odd divider with a 50% duty cycle is to generate two clocks at half the desired output frequency with a quadrature-phase relationship (constant 90° phase difference between the two clocks).

The output frequency can then be generated by exclusive-ORing the two waveforms together.

Because of the constant 90° phase offset, only one transition occurs at a time on the input of the exclusive-OR gate, effectively eliminating any glitches on the output waveform.

Let's see how it works by taking an example where the reference clock is divided by 3.

Below are the sequential steps listed for division by an odd integer [96]:

STEP I: Create a counter that counts from 0 to $(N - 1)$ and always clocks on the rising edge of the input clock where N is the natural number by which the input reference clock is supposed to be divided ($N \neq \text{Even}$).

For Divide by 3: i.e. counts from 0 to 2 ... $N=3$

For Divide by 5: i.e. counts from 0 to 4 ... $N=5$

For Divide by 7: i.e. counts from 0 to 6 ... $N=7$

Note: The counter is incremented on every rising edge of the input clock (*ref_clk*) and the counter is reset to ZERO when the terminal count of counter reaches to $(N - 1)$.

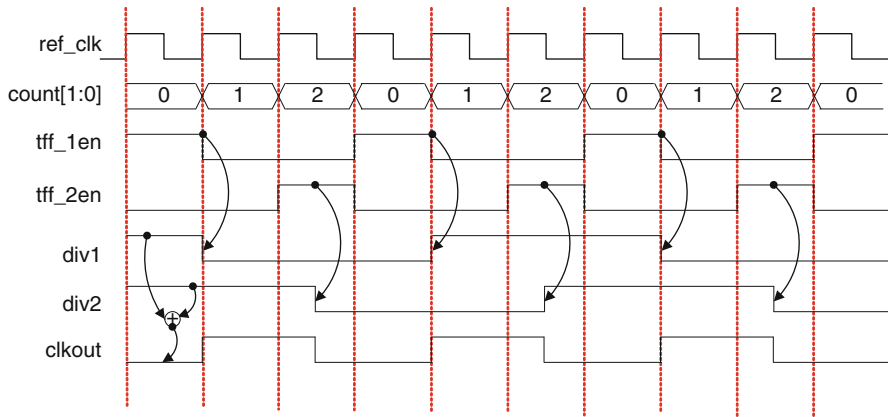


Fig. 4.2 Timing diagram for divide by 3 ($N=3$) with 50% duty cycle

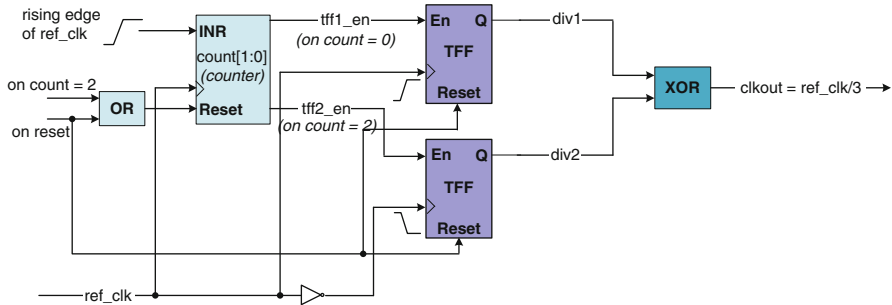


Fig. 4.3 Divide by 3 using T flip-flop with 50% duty cycle output

STEP II: Take two toggle flip-flops and generate their enables as follows:

tff1_en: TFF1 enabled when the counter value = 0

tff2_en: TFF2 enabled when the counter value = $(N+1)/2$. (2 for Divide by 3, 3 for Divide by 5 counter and likewise) as shown in Fig. 4.2

STEP III: Generate the following signals.

div1: output of TFF1 → triggered on rising edge of input clock (ref_clk)

div2: output of TFF2 → triggered on falling edge of input clock (ref_clk)

Note: The output “div1” and “div2” of two T Flip flops generate the divide-by-2N waveforms as shown in Fig. 4.2.

STEP IV: Generate the final output clock “clkout” (Divide by N) by XORing the “div1” and “div2” waveforms.

Figure 4.3 shows the logic for the Divide by 3 clock divider circuit.

4.4 Non-integer Division (with a Non 50% Duty Cycle)

It's a common practice to use a divide-by-N circuit to create a free-running clock based on another clock source. Designing such a circuit where N is a non-integer is not as difficult as it seems. Let's consider what it means to Divide by 1.5. It simply means that, every three reference clock include two symmetrical pulses.

Following section shows a simple example where the clock is divided by 1.5 with a non 50% duty cycle.

4.4.1 Divide by 1.5 with Non 50% Duty Cycle

The multiplexer in Fig. 4.4 selects the input clock when “clkout” is HIGH, otherwise it selects the inverted version of the input clock “ref_clk”.

Figure 4.5 shows the timing diagram for the divide-by-1.5 circuit shown in Fig. 4.4.

Note: *The above circuit will work perfectly in simulation but fail in synthesis due to the mux incorporated, which introduce unequal delays when the select line of the mux toggles. The mux not be able to change the output instantly and may produce glitches in the generated output clock. The probability of failure increases with increase in reference clock (ref_clk) frequency.*

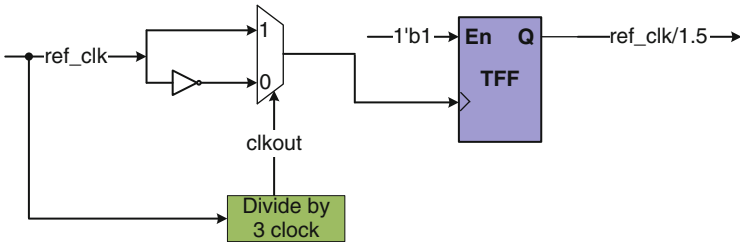


Fig. 4.4 Divide by 1.5 using T flip-flop (duty cycle non 50%)

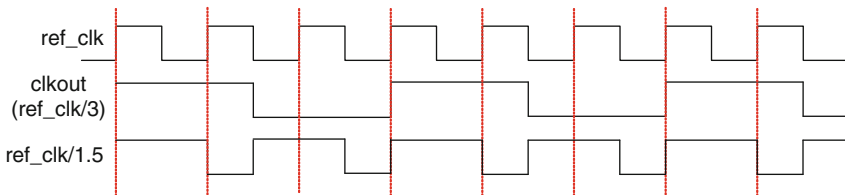


Fig. 4.5 Timing diagram for divide by 1.5 using toggle flip-flop (with non 50% duty cycle)

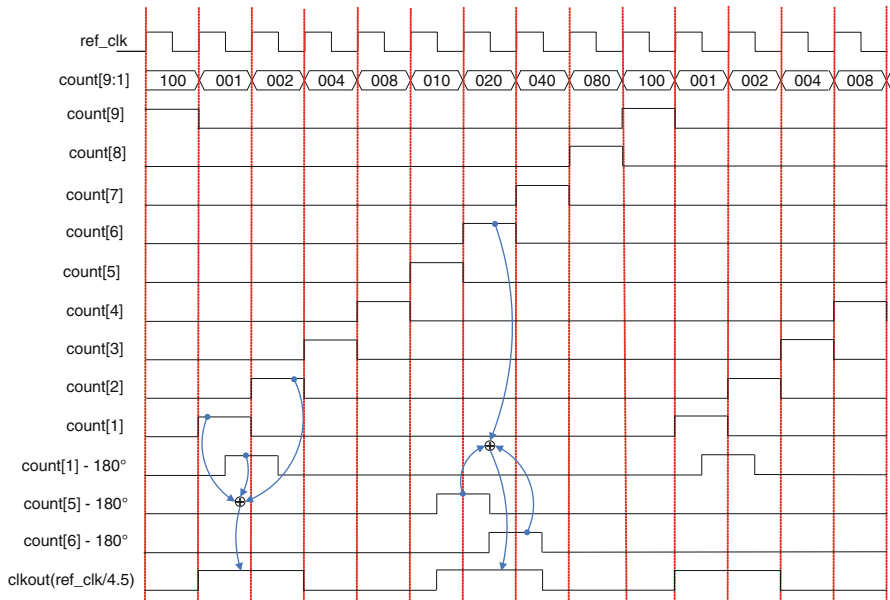


Fig. 4.6 Counter implementation for divide by 4.5 (duty cycle non 50%)

4.4.2 Counter Implementation for Divide by 4.5 (Non 50% Duty Cycle)

This section describes an alternate approach where the circuit for divide by a non-integer is more optimized with generated output clock perfectly glitch-free.

Let's take an example with a divide by 4.5. It means every nine reference clocks would include two symmetrical pulses.

Below are the sequential steps listed for division by a non-integer:

- STEP I: Take a nine-bit shift register and initialize it to 000000001 upon reset where the shift register is left-rotated on every rising clock edge.
- STEP II: To generate the first pulse, first bit must be shifted by half a clock period and then ORed with the first and second bit.
- STEP III: To generate the second pulse, the fifth and sixth bits must be shifted by half a clock period and then ORed with the original sixth bit.

Note: All this shifting is necessary to ensure a glitch-free output waveform.

Duty cycle for the above generated clock is 40% with a glitch-free output.

Figure 4.6 shows the timing diagram for divide-by-4.5 circuit

Below is sample Verilog code for the divide-by-4.5 logic.

```

/* Counter implementation
   reset value : 9'b000000001 */
always @ ( posedge ref_clk or negedge p_n_reset)
    if (!p_n_reset)
        count [9:1] <= 9'b000000001;
    else
        count [9:1] <= count [9:1] << 1;
/* count bit 1st, 5th and 6th phase shifter by 180 deg */
always @ (negedge ref_clk or negedge p_n_reset)
    if (!p_n_reset)
        begin
            ps_count1 <= 1'b0;
            ps_count5 <= 1'b0;
            ps_count6 <= 1'b0;
        end
    else
        begin
            ps_count1 <= count[1];
            ps_count5 <= count[5];
            ps_count6 <= count[6];
        end
// Genration of final output clock = (ref_clk / 4.5)
assign clkout = (ps_count 5 | ps_count 6 | count [6]) |
                 (count [0] | count [1] | ps_count1);

```

4.5 Alternate Approach for Divide by N

Each circuit assumes a 50% duty cycle of the incoming clock otherwise the fractional divider output will jitter and the integer divider will have unequal duty cycle.

All the circuits use combinatorial feedback around a LUT (look up table) that works perfectly and the output clock generated is glitch free.

Let's start up with a simple Divide-by-1.5 circuit again.

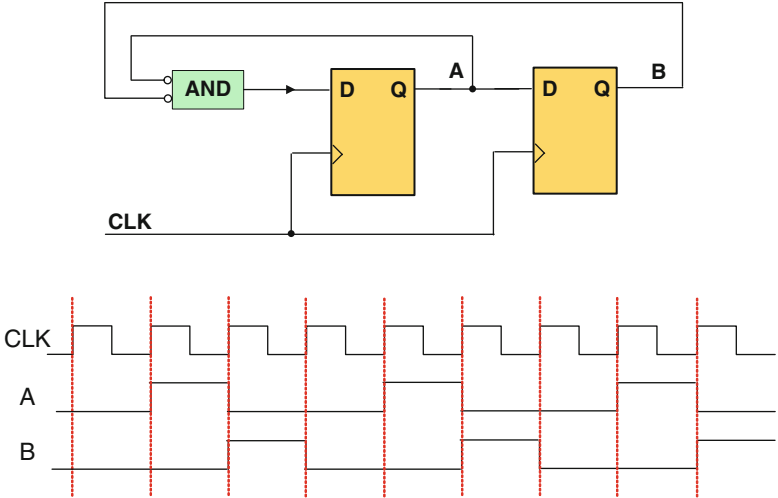


Fig. 4.7 Divide by 1.5 (duty cycle non 50%)

4.5.1 LUT Implementation for Divide by 1.5

Divide by 1.5 is generated by generating a Divide by 3 circuit as shown in Fig. 4.7 where the two flip flops shown form a Divide by 3 circuit (Non 50% duty Cycle) [96].

Reference

1. Arora M (2002) Clock dividers made easy, ST Microelectronics, SNUG Boston

Chapter 5

Low Power Design

5.1 Introduction

In the good old days of IC design, before power became a significant design constraint, most chips were designed without concern for power consumption. This is not true anymore as requirements for lower power consumption continue to increase significantly as components become battery-powered, smaller and require more functionality.

Although low power designs techniques have been employed for years in battery operated applications such as pacemakers and digital watches, several technology trends are driving these techniques into broad use.

The energy consumed is dissipated in the form of heat. Heat management is now also a major concern for device manufacturers. Reliability is a function of heat and it has been said that every 10°C rise in temperature corresponds to an estimated doubling of failure rates. Maintaining low operating temperatures means using heat sinks and/or fans to remove unwanted heat – adding to the overall weight and cost. If power reduction techniques can be incorporated at the SoC level these overheads can be reduced, or possibly even eliminated. The net is a smaller, less expensive and more reliable end-product.

Recently there has been more focus than ever to meet energy efficiency goals. This Chapter describes various design methodologies and techniques to reduce dynamic and static power consumption.

5.2 Sources of Power Consumption

The three main sources of power consumption are inrush, static and dynamic.

Inrush current refers to the maximum, instantaneous input current drawn by an electrical device when first turned on. Inrush current is also known as start-up current for an application.

Inrush current is device-specific. For example, an electric motor when switched on draws huge startup current several times their normal full-load current for the first few cycles.

SRAM-based FPGAs also features a high inrush current because on power-up these devices are not configured and need to actively download data from external memory chips to configure their programmable resources, such as routing connections and lookup tables. Conversely, anti-fuse-based FPGAs do not have a high inrush current since they do not require power-on configuration.

Standby current refers to the current drawn by an application when main supply is removed or system is in standby mode. Much like inrush power, standby power depends heavily on the electrical characteristics of a component. Standby power is also known as Static power. Note that Static power includes leakage currents in the transistors of the circuit.

Dynamic power or switching power is the power dissipated as a result of logic transitions when gate outputs toggle.

Dynamic Power (P_{dynamic}) is defined by the following equation:

$$P_{\text{dynamic}} = S \times C_L \times V_{\text{dd}}^2 \times f_{\text{clk}}$$

Where

C_L = Gate parasitic capacitance

S = Average number of transitions across the entire circuit per clock cycle

f_{clk} = Clock frequency

V_{dd} = Supply voltage

When the output changes from Logic 0 to Logic 1 this capacitance must be charged, and discharged during Logic 1 to Logic 0 transitions. Note that the switching power is also a function of the clock frequency f_{clk} and supply voltage V_{dd} .

Thus, the total power dissipation of an ASIC is defined as

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{static}}$$

These equations, when computed over a number of clock cycles, produce time-averaged power.

Dynamic power tends to dominate large IC designs. For a typical application dynamic power may constitute as high as 80% of the total power.

5.3 Power Reduction at Different Levels of Design Abstraction

Power reduction has to be addressed at all the design levels i.e. at the system level, the architectural level, the logic level and at the physical level. However, higher the level of abstraction is, greater the potential for power savings.

Figure 5.1 shows a variety of design techniques, at several different levels of design abstraction for minimizing power. Though power can be saved at all levels

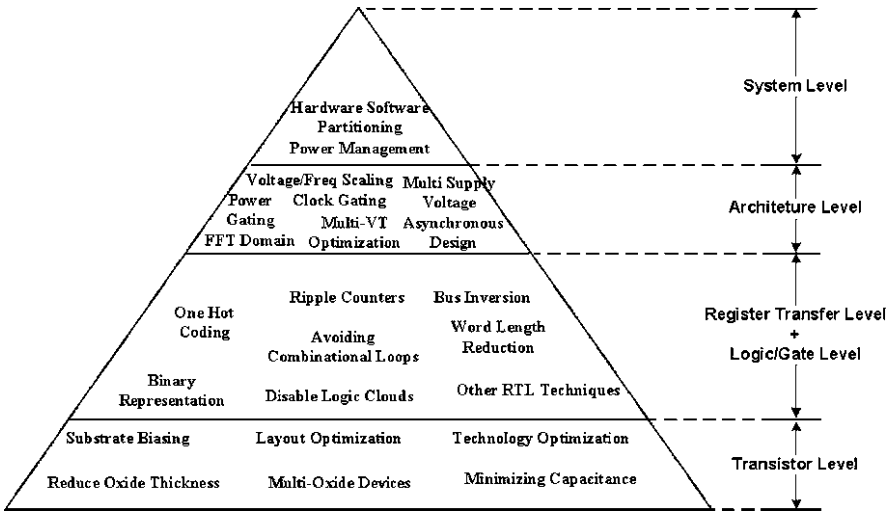


Fig. 5.1 Levels of design abstraction for power saving

Table 5.1 Power reduction opportunities at different levels of abstraction	
Level of abstraction	Power reduction opportunities
System level	10–100X
Architecture level	10–90%
Register transfer level	15–50%
Logic/gate level	15–20%
Transistor level	2–10%

of design abstraction but it is best to address this at higher level of abstractions i.e. Systems and Architectural level to get the maximum saving.

To minimize power consumption to the greatest extent, decisions made at each level of design abstraction must account for power.

Referring back to the equations for total power, it is evident that power can be reduced by lowering the voltage, capacitance, signal frequencies, or cell energies. Specific techniques will address one or more of these factors at a particular level.

Lot of system level decisions strongly depends on application for i.e. one may choose cache based memory or a centralized memory. At architecture level one has to decide for instance to implement a parallel or pipeline structure, a parallel version, being for instance, far more power effective than a multiplexed one. At the logic and layout level, the choice of a mapping method to provide a netlist and the choice of a low power library are crucial. At the physical level, layout optimization and technology have to be chosen.

Table 5.1 shows power reduction opportunities at various level of design abstraction.

Next few sections provide various power reduction techniques at different level of abstraction all the way from System to transistor level.

5.4 System Level Power Reduction

Before starting a design, it is mandatory to think about the system and goal for performance and power consumption.

5.4.1 *System on Chip (SoC) Approach*

For a high end chip in nanometer technology, I/O's can consume as much as 50% of the total power due to the higher voltage level used (Typically 3.3 V) for I/Os than for the Chip core logic. If overall system is built from multiple individual chips, significant power will be wasted in interconnecting these chips. Modern design practices now focus on the system-on-chip (SoC) methodology as a means of saving power, area and ultimately cost.

5.4.2 *Hardware/Software Partitioning*

Since embedded processors are quite common in any large scale digital systems, some of the functionality can be implemented in hardware while the rest is in software.

Communications algorithms tend to be highly recursive in nature, meaning that small blocks of code account for a significant amount of processing resources. In fact up to 90% of execution time can be spent in just 10% of the code. If these resource intensive blocks can be identified and implemented in hardware, significant amounts of power can be saved. These recursive blocks may even be a fraction of a percent of the system, but can save significant power in the system.

The author of [10] presents the HDTV Chromakey algorithm with 22,000 lines of code. Only 15 lines, the critical loops, are implemented in hardware, which results in an energy saving of 77%.

The conventional techniques for co-design is usually done by partitioning the system into hardware and software components at an early stage of the design and then iteratively refining it until a good solution is found. This method is expensive and time consuming.

A typical design process would involve the following [11]:

- Specifications
- Partitioning

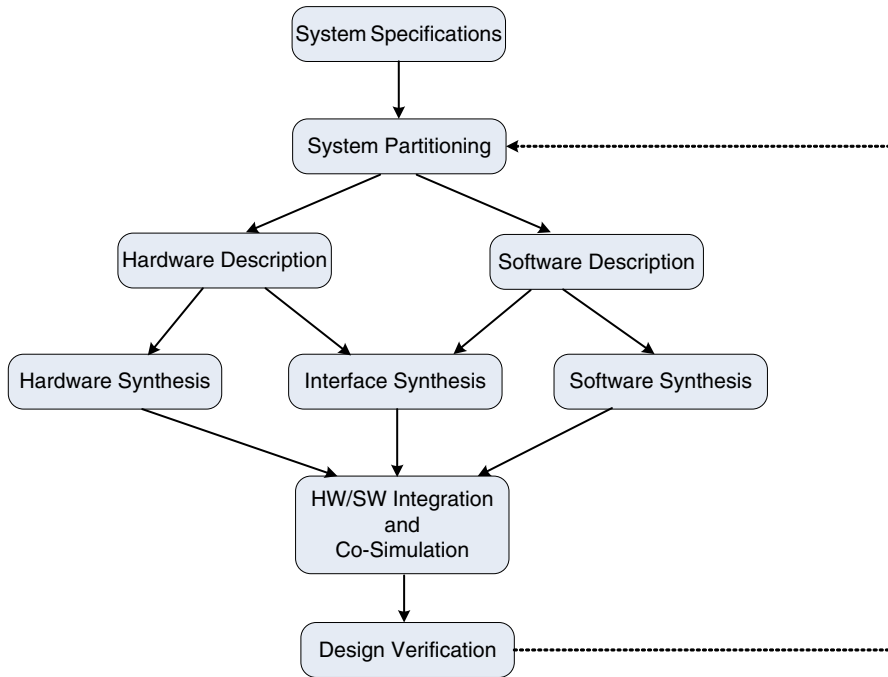


Fig. 5.2 Conventional approach to hardware/software co-design

- Synthesis
- Integration
- Co-Simulation
- Verification

Figure 5.2 shows Conventional Approach to Hardware Software Co-Design [11].

The system design process starts with a specification of the system. The system designer then uses the specification and his experience to make educated guesses on the performance of the system. Based on these guesses, he/she decides which part of the system will be implemented in hardware (as an ASIC) and which part in software.

This also involves writing the behavioral description for the different parts of the system. The hardware part, for example, might be described using VHDL or Verilog and the software model using the C language. In addition, the interfacing logic, including any handshaking or bus logic, has to be decided.

Different tools can be used to extract physical model from behavioral model for example using Hardware synthesis tool to extract physical mode from hardware descriptions written in Verilog or VHDL. Similarly cross compilers compile programs written in a high-level language into the native instruction set of an embedded processor.

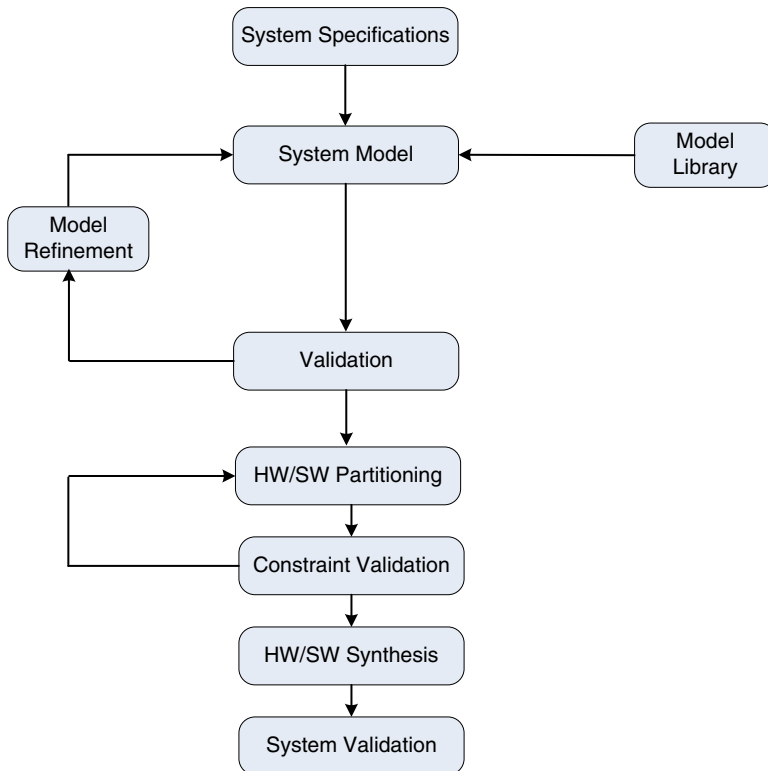


Fig. 5.3 Model based approach to HW/SW co-design

Next Step involves Co-simulation using any commercial available co-simulation platforms that can simulate hardware and software synthesized models in an integrated environment. The results of co-simulation are verified against functional requirements and design constraints from the specification. If the system does not meet the requirements, the entire process, starting with system partitioning, is repeated.

A more efficient approach than conventional approach for HW/SW partitioning is to have a Model based approach as shown in Fig. 5.3.

Here the idea is based on a developing a System Model for system based on given specifications. The model can either be built from scratch or based on existing Model library can be reused. As the library grows over time, design time is greatly reduced.

Modeling can be based on SystemC that is a set of libraries that extend the C++ language to model hardware. A SystemC program can be compiled with a standard C++ compiler and the generated object code can be used to simulate the model.

System C offers lot of flexibility where one can build Model on high or abstract level to cycle accurate to represent the overall system.

Results of the simulation are used to validate the model. The output of the simulation is matched with the expected values. The validation results may also be used to refine the model.

Any further details on System Level Modeling and HW/SW Partitioning is out of scope for this book.

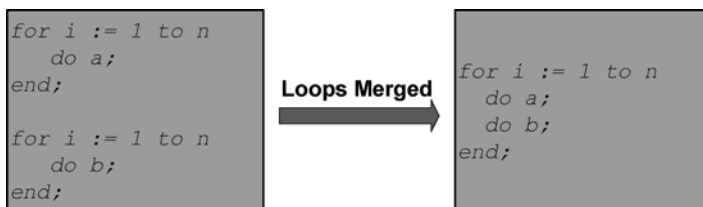
5.4.3 Low Power Software

Hardware designers have always been conscious about power consumption while designing an integrated chip or ASIC. However this does not seem to be true for most of the software engineers. Furthermore, a large part of the power consumption can be saved while modifying the application software thus resulting in 'greener' and more energy-efficient system.

High level languages are convenient tools to achieve results quickly and often have features to do complex things with minimal effort. However some of these constructs are hard to implement and sometimes the runtime environment that implements the high level language does so using polling at a high frequency resulting in high power consumption. So while using high level languages, one should avoid using complex primitives.

For embedded applications, it is quite often the case that an industrial existing C code has to be used to design an application. A "C" code may use several loops. In some applications, the application may be running 90% of the time in loops.

Several techniques can be used to optimize the loops. So if two loops are executed in sequence with the same indices, they can be merged. The number of executed instructions is therefore reduced. For example



With merging the loops as shown above, number of executed instructions is reduced as the loop counter (initialization, increment, and comparison) is removed.

Other consideration may include implementation based on certain hardware architecture or processor instruction and registers. It has to be noticed that a μ P based implementation results in a very high sequencing since the internal registers (ALU, Accumulators) can be utilized to do the operation quickly. For instance, it may take only one step to up-date a hardware counter verses several steps to the same in software as the later involve executing several instructions with many clocks in sequence.

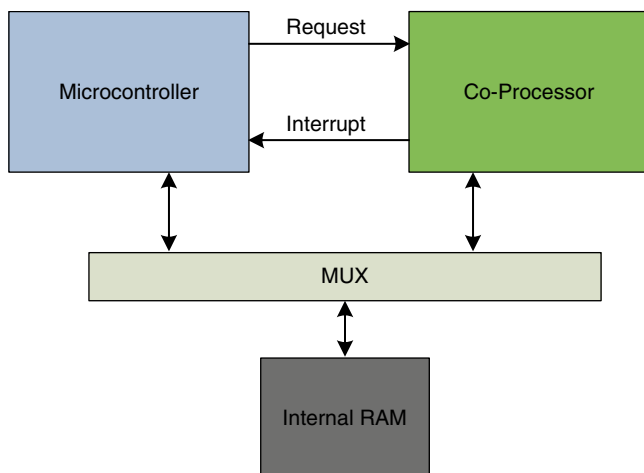


Fig. 5.4 Microcontroller and co-processor

5.4.4 Choice of Processor

Choice of processor can have significant impact on overall power consumption. The first point is to adapt the data width of the processor to the required data. It results in a quite increased sequencing to manage for instance 16-bit data on an 8-bit microcontroller. For a 16-bit multiply, 30 instructions are required (add-shift algorithm) on a 16-bit processor, while 127 instructions are required on an 8-bit machine (double precision). A better architecture is to have a Multiply Accumulate Unit (MAC) or 16×16 bit parallel-parallel multiplier with only one instruction to execute a multiplication.

Incorporating a dedicated DSP processor to do data processing may not make lot of sense if simple MAC satisfies the requirements as this can have significant impact on power budget.

Figure 5.4 shows an interesting architecture to save power [12]. For any applications, there is some control that is performed by a microcontroller while data processing tasks can be carried over by a co-processor or a DSP. The best architecture is to design a specific machine (co-processor) to execute this task. So this task is executed by the smallest and the most energy efficient machine. Most of the time, both microcontroller and co-processor are not running in parallel.

5.5 Architecture Level Power Reduction

A system can be realized using a number of different architectures that can have a significant impact on power consumption. This section presents some of the architecture level trade-off and techniques to reduce power consumption.

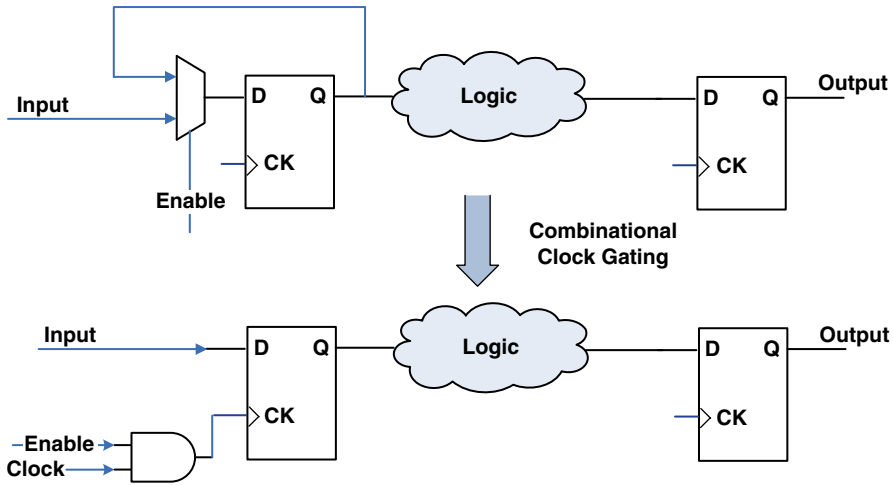


Fig. 5.5 Combinational clock gating

5.5.1 Advanced Clock Gating

In synchronous digital systems, the clock distribution can contribute towards a large percentage of the total circuit switching power. In many situations it is possible to disable significant portions of the circuit by gating the clock when their use is not required.

Combinational Clock Gating has been described previously in Sect. 2.5. Combinational Clock gating is based on the fact that a feedback path exists from output of the flop to input, and thus is also called “Feedback Loop based Clock gating”. Note that in a combination clock gating, there is no change in the functionality of the flop before and after clock gating and thus can be verified by combinational equivalence checkers. Let’s look at some of the advanced clock gating techniques in this section.

Since the combinational clock gating scheme reduces power by disabling the clock on flops when the output is not changing, it can reduce dynamic power consumption by 5–10%. On the contrary, sequential clock gating alters the design structure without affecting design functionality. Sequential clock gating reduces the redundant switching, in the portion of the design connected to the register bank with gated clock.

Figure 5.5 shows combinational clock gating scheme and Fig. 5.6 shows sequential clock gating for the same circuit. Note that with sequential clock gating, subsequent pipeline stages are also gated based on the enable condition.

Figure 5.6 also shows additional logic added for sequential clock gating implementation. Because of the additional logic, this technique is not effective if data is multiple bits wide.

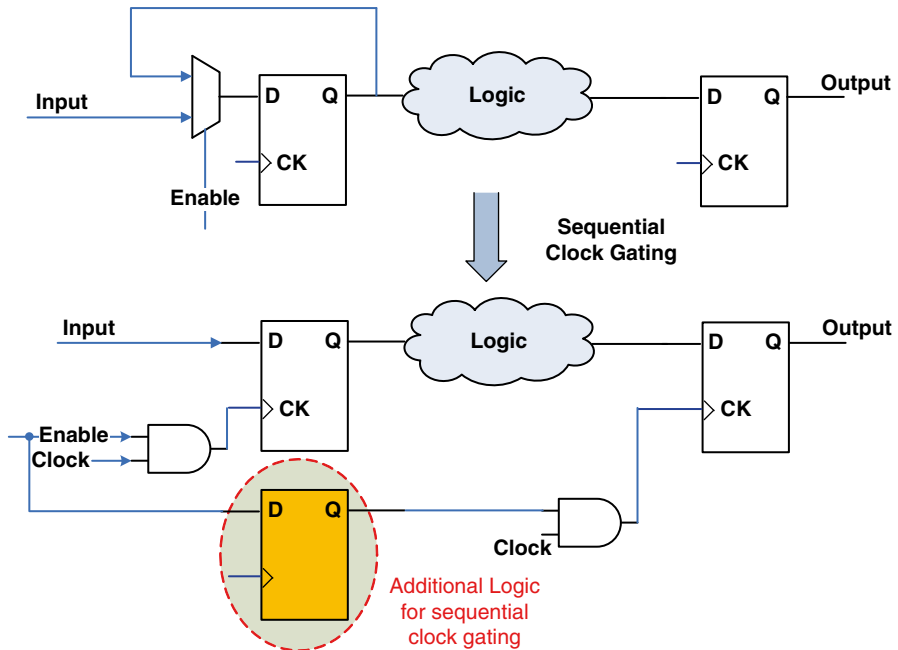


Fig. 5.6 Sequential clock gating

Main challenge with sequential clock gating is to identify “redundant” or “don’t care” states in the pipeline but once done this technique can save significant power, typically reducing switching activity by 15–25% on a given block [14].

As per [14], sequential clock-gating transformation is only effective when more than 16 flops can be gated.

5.5.2 Dynamic Voltage and Frequency Scaling (DVFS)

Dynamic voltage/frequency scaling (DVFS) is a popular method for improving system energy-efficiency. By lowering clock speed and supply voltage during frequency-insensitive application phases, large reduction in power can be achieved with modest performance loss.

As mentioned in Sect. 5.2, power consumption is proportional to the supply voltage with a quadratic relationship between power and supply voltage. Hence reducing the supply voltage will result in a quarter of the power consumption. Unfortunately reducing the supply voltage has a detrimental effect on performance meaning there is a trade-off to be made.

For handheld devices like mobile phones, PDAs there is an inherent conflict in the design goals as they should be designed to maximize battery life, but as intelligent devices, they need powerful processors, which consume more energy than

those in simpler devices, thus reducing battery life. In spite of continuous advances in semiconductor and battery technologies that allow microprocessors to provide much greater computation per unit of energy and longer total battery life, the fundamental tradeoff between performance and battery life remains critically important.

For a typical application high performance is needed only for a small fraction of the time, while for the rest of the time, a low-performance, low-power processor would suffice. This can be achieved by simply lowering the operating frequency of the processor when the full speed is not needed.

Vast majority of microprocessors today designed with CMOS process has a voltage-dependent maximum operating frequency, so when used at a reduced frequency, the processor can operate at a lower supply voltage. Thus, DVFS can potentially provide a very large net energy savings through frequency and voltage scaling.

In these real-time embedded systems, one cannot directly apply most DVFS algorithms known to date, since changing the operating frequency of the processor will affect the execution time of the tasks and may violate some of the timeliness guarantees.

There are several algorithms that incorporate DVFS into the OS scheduler and task management services of a real-time embedded system, providing the energy savings of DVFS while preserving deadline guarantees.

In Particular, Dynamic Voltage Scaling (DVS) relies on special hardware, in particular, a programmable DC-DC switching voltage regulator, a programmable clock generator, and a high-performance processor with wide operating ranges, to provide this best-of-both-worlds capability. In order to meet peak computational loads, the processor is operated at its normal voltage and frequency. When the load is lower, the operating frequency is reduced to meet the computational requirements.

Technology scaling now allows designers to integrate increasing numbers of cores onto a single die. Microprocessor designs are moving away from full-chip voltage/frequency control towards finer-grained methods which allow increased energy-efficiency. For example, AMD's quad-core Opteron allows independent frequency control of all four cores, the shared L3 cache and on-chip northbridge, the DDR interface, and four HyperTransport links [15].

In Summary, by dynamically scaling both voltage and frequency of the processor/system based on computation load, DVS can provide the performance to meet peak computational demands, while on average, providing the reduced power consumption (including energy per unit computation) benefits typically available on low-performance processors.

5.5.3 Cache Based Architecture

For the majority of DSP Applications, Fast Fourier Transforms (FFT) algorithms requires frequent access to data stored in System Memory that is not very power efficient. An enhancement to the FFT architecture has been the inclusion of a small amount of cache memory between the processor and System Memory or RAM.

The original idea of this scheme was to enhance performance by pre-fetching relevant data samples from the main memory just before it is required. Using small localized caches the computation energy costs can be dramatically reduced leading to a very power efficient implementation of the FFT.

5.5.4 *Log FFT Architecture*

For applications that require large number of complex calculations, it can be beneficial to use the logarithmic number system (LNS) as opposed to linear. LNS implements multiplications and divisions using additions and subtractions along with reduction of average bit activity, making it more energy efficient than linear number systems. Hence implementation of an FFT based on LNS has the potential to save a significant amount of power. The downside however is that adder and subtracter widths must be increased – leading to an exponentially larger look-up-table size.

5.5.5 *Asynchronous (Clockless) Design*

For a synchronous design based on clocked architecture, clock distribution consumes majority of the power consumption. Conventional design methodologies have resulted in massive clock tree structures, which substantially increase the average power consumed by the SoC. Also clock distribution requires significant designer overhead. Probably the most significant problem is clock skew, which is the difference in arrival time of the clock signal to different parts of a circuit. When a circuit is large and slow, the clock skew is insignificant. However as circuits shrink and their speeds grow, this difference becomes very significant and extra design time and often extra circuitry needs to be used to solve the problem. It is becoming difficult to distribute clock as network spreads over die and may have irregular layout. This additional overhead results in considerable power consumption.

With all of the problems caused by the clock, it is very tempting to simply remove it from the system. This is the fundamental idea behind asynchronous design. However, it is not as simple as just removing the clock, since the operation of the circuit must still be controlled somehow. Asynchronous circuits essentially govern themselves, and are therefore called self-timed circuits.

Figure 5.8 shows asynchronous system in which two blocks talk to each other with a handshake interface.

Removal of clock results in increased power efficiency. The clock dissipates a lot of power, especially in larger and faster designs, so removing it can yield a substantial improvement in power efficiency. Apart from this, Asynchronous Circuits consume zero dynamic power as inactive components consume negligible power.

Asynchronous Circuits are based on Handshake Interface that relies on delay in-sensitive encoding, most popular being the Dual Rail Encoding.

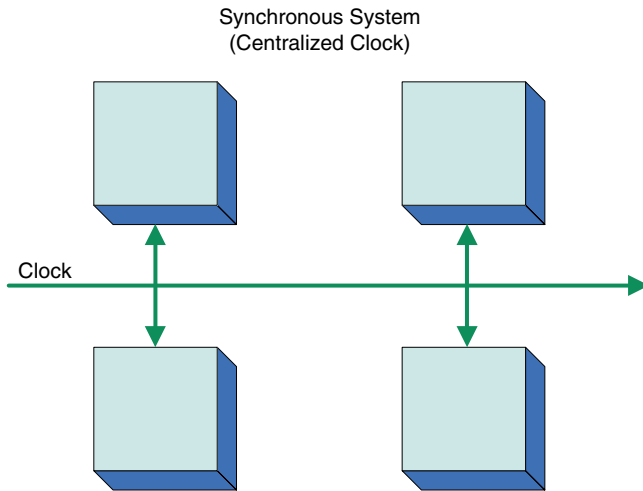


Fig. 5.7 Synchronous system with centralized clock

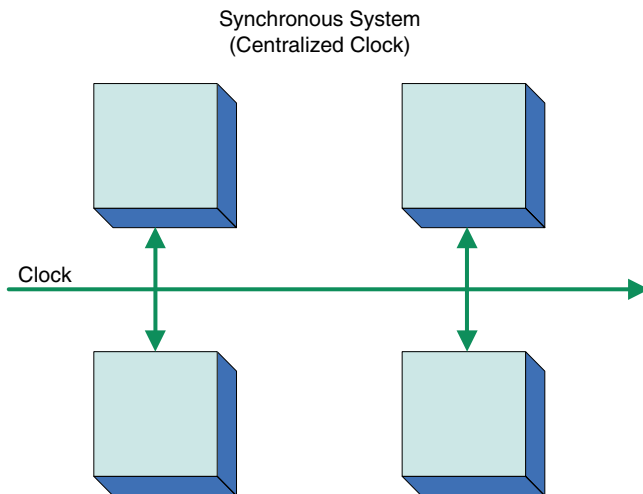


Fig. 5.8 Asynchronous system based on handshake interface (no global clock)

Dual rail uses two wires per bit of data, thus dual rail. (Single Rail uses just one wire to carry one bit of data).

With Dual Rail, one wire is used for signaling Logic 1 and other to indicate Logic 0. Two parties can talk to each other reliably regardless of delays in the wires connecting the two. The protocol is *Delay insensitive*. Encoding is as follows:

“LL” = “spacer”, “LH” = “0”, “HL” = “1” where L = Logic “0” and H = Logic “1”

Spacer means no data. 0 or 1 means valid data. “11” means INVALID

Note that here n -bit data communication requires $2n$ wires. Each bit is Self-Timed.

The described encoding in this section is four phase Dual Rail Encoding. Two phase dual rail also uses two-wires per bit but information is encoded as events (0 to 1, 1 to 0). New codeword is received when one wires makes a transition. There is no empty Value (No “00”). A Valid message acknowledge followed by another valid message.

Other delay-insensitive codes exist (e.g. m -of- n) and event-based signaling, choice being based on pin and power efficiency.

Per the *International Technology Roadmap for Semiconductors* (ITRS, ex. SIA), 1999 edition:

With clock speed possibly exceeding 5 GHz, and across-chip communication taking upwards of 5 to 20 clock cycles, an approach is needed to building a hierarchy of clock speeds with locally synchronous and globally asynchronous interconnects. Tools to handle asynchronous, multi-cycle interconnect as well as locally synchronous, high performance near neighbor communication is needed.

Further details on Asynchronous Circuits are out of scope for this book.

5.5.6 Power Gating

Similar to voltage gating, power gating involves temporarily shutting down blocks in a design when the blocks are not in use.

For the devices designed at 90 nm and below geometries, transistor leakage is increasing exponentially creating design challenges to meet power targets. Reducing this excessive leakage is important so as to have a good battery life specifically for hand-held devices that operate on battery. Power gating is one of the most effective techniques to address this complex challenge by shutting down the logic modules when they are not required in operation.

Power gating, or power switch-off technique, usually refers to placing switches on-chip to selectively turning off current supply based on the application requirement. Designers are employing two types of power gating – fine-grain power gating and coarse-grain power gating.

5.5.6.1 Fine-Grain Power Gating

In fine-grained power gating, a switch transistor is placed between ground and each gate. This approach allows shutting off the connection to ground whenever a series of functions is not in use. This can be done with every cell in the library.

The primary burden of adding switching transistors is left with the library IP provider or standard cell designer [16]. Usually these cell designs conform to the normal standard cell rules and can easily be handled by EDA tools for implementation.

The power gate size must be selected to handle the amount of switching current at any given time. The gate must be bigger such that there is no measurable voltage

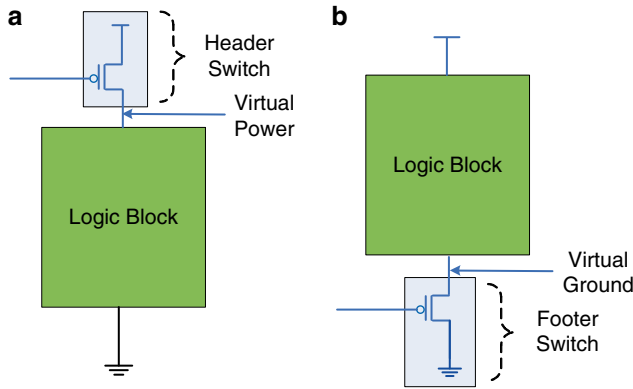


Fig. 5.9 Header and footer gate for power gating

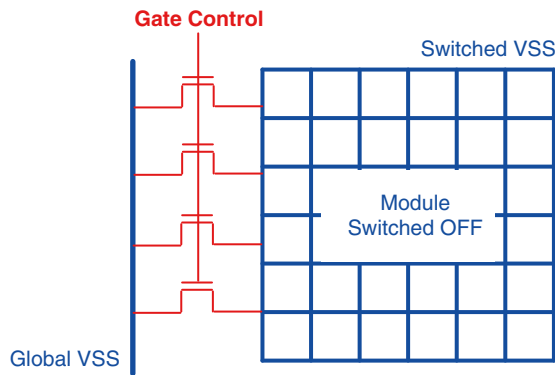


Fig. 5.10 Coarse grain power switching

(IR) drop due to the gate. One can also choose between header (P-MOS) and footer (N-MOS) gate as shown in Fig. 5.9. Usually footer gates tend to be smaller in area for the same switching current. Dynamic power analysis tools can accurately measure the switching current and also predict the size for the power gate.

Fine-grain power gating is an elegant methodology resulting in up to 10X leakage reduction [16]. This type of power reduction makes it an appealing technique if the power reduction requirement is not satisfied by multiple V_t optimization alone.

5.5.6.2 Coarse Grain Power Gating

In coarse-grain power gating, the power-gating transistor is a part of the power distribution network rather than the standard cell. Coarse grain create a power-switch network essentially, a group of switch transistors that in parallel turn entire blocks on and off as shown in Fig. 5.10.

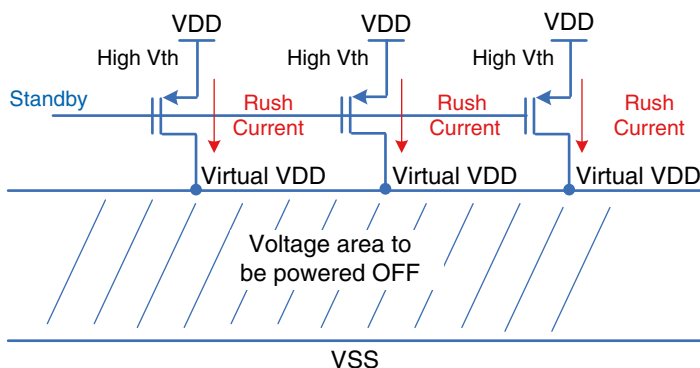


Fig. 5.11 Rush current in coarse grain power switching

Unlike fine-grain power-gating, coarse-grain approach does not rely wholly on the quality of library, but mostly the capability of how EDA tools handle.

The operation of coarse-grain power-gating is same as that of fine-grain power-gating. However, the implementation and analysis behind this mechanism is quite complex. The size and numbers of standby transistors to be placed inside the power off voltage area are parameters to control the driving capability to the region. This will lead to IR (voltage) drop variation and performance regression. When all header switches are turned back on simultaneously, there is an instantaneous charging/discharging current and short-circuit current flowing from VDD to VSS. The behavior is shown in Fig. 5.11.

The aggregate current is known as power-up rush current. This current is critical to avoid device malfunction or potential chip failure. This is also the major concern in coarse-grain power-gating implementation.

Note that implementing any kind of power gating would require changes in the RTL so as to design a power controller that would control the blocks that need to be shut down and the voltage that is being fed to the blocks in concern. A multi-million gate ASIC would easily have more than 20 or even more power domains. That number would be too hard to control with either a true fine-grained or a true coarse-grained technique so practically ASIC may use a combination of both the techniques.

In order to minimize leakage, the power-gating transistors are built using high V_t cells. Coarse-grain power gating offers further flexibility by optimizing the power gating cells where there is low switching activity. These cells may be replaced with filler cells that ensure power distribution integrity.

Coarse-grain power gating technique can deliver the aggressive power reduction. At the time of the publication of this book, only advanced EDA tools can support this methodology. The trade-offs in gate size selection, placement, routing, simultaneous switching analysis and gate control signal slew rate involves complex optimization process. The tool needs be multiple-power-domain-aware so it can perform this optimization without compromising the quality of silicon.

Power Gating would also need Isolation cells to be inserted for the logic or signals going from “off” domain to the “on” domain to preserve design integrity and avoid power losses. When power for a block is shutdown, with its outputs going to a block that is still powered up, power-down nodes get floating, and they can float to the threshold voltage and create unwanted currents. Isolation cells on the inputs in the “on” domain clamp the outputs from power down domain to a one or a zero. Usually a simple OR or AND logic can function as an output isolation device. Isolation cells can either be inserted in the RTL or can be done by the EDA tools by specifying the parameters and blocks to be isolated. The tools are smart enough to take those commands and insert them at the appropriate levels. Some get inserted during synthesis; others get inserted during place and route.

5.5.7 Multi-threshold Voltage

Multiple-cell libraries help to deal with both leakage and dynamic power. A multi-cell library typically comprises at least two sets of identical cells that have different threshold voltages. The cells with higher threshold voltage are slower but have less leakage; conversely, the cells with lower threshold voltage are faster but have higher leakage.

High-threshold-voltage cell typically has 50% less leakage than a low-threshold-voltage cell with no bad side effects, such as area gain.

Flip side of this technique is that Multi Vt cells increase fabrication complexity. It also lengthens the design time. Improper optimization of the design may utilize more Low Vt cells and hence could end up with increased power.

The tradeoffs between the different Vt cells to achieve optimal performance are especially beneficial during synthesis technology gate mapping and placement optimization. The logic synthesis, or gate mapping phase of the optimization process is implemented by synthesis tool, and placement optimization is handled by physical implementation tool. Designers may use different strategy depending on the design goal.

If the ultimate design goal is to meet performance, one may use a low-threshold-voltage library for a first pass through synthesis to get maximum performance and meet timing goals. Next the paths in the design that don’t require the highest performance or low voltage cells can be determined and replaced with high-voltage cells to reduce overall power and leakage of the design. This approach represents the most common use of the multi-threshold-design technique because most applications have timing as a first requirement. Also low-threshold-voltage libraries run faster through synthesis, and synthesis tools ultimately produce smaller design areas from these libraries. Synthesis tools tend to run longer and produce larger design areas when running heavy doses of high-threshold-voltage cells.

However if power is the main goal for an application and area increases are less of an issue, it would be appropriate to first run synthesis with high-threshold-voltage cells, find the critical path, and then swap out the high-voltage cells with low-voltage cells until performance goal is met.

5.5.8 *Multi-supply Voltage*

As the equation on Dynamic Power consumption in Sect. 5.2 indicates, power is proportional to the square of the supply voltage. In multi-supply voltage (MSV) design, design can be partitioned into separate “voltage islands” or “voltage domains”, where each domain operates at a different supply voltage depending on its timing requirements.

One way to partition is to keep timing-critical blocks in one domain operating at the standard supply voltage of say 1.0 V in a 90 nm process. Blocks with less critical timing paths can be aggregated into a second domain, with the voltage scaled down to say 0.8 V – a 36% reduction in dynamic power for that portion of the design.

In past, this approach has introduced additional complexity during physical design. Designers would typically need to manually insert special translation cells, called voltage level shifters, to convert signals between different voltage domains. For a multi-million gate ASIC, this could be several thousand signals crossing from one voltage domain to other making this error prone and risky. Today there are several tools from EDA companies that can insert the level shifters automatically making this approach more practical.

5.5.9 *Gate Memory Power*

In a Typical SoC, SRAM may consume one third of the total power with remaining power being consumed by the clock tree and random logic. Memory Architecture is thus the key for a good power management strategy.

In its simplest form, this approach involves shutting down segments of a memory array when they are not in use. Choice of having one big single array of Memory or having multiple small memories has a good power tradeoff.

SoC may choose to split a memory, if SoC only needs a small portion of memory that has to be on all the time, together with a bigger memory that you can turn on and off depending on the mode of operation. The small memory is in a power “on” state while the big memory may choose to be powered when doing computational intensive tasks.

Also note that if a larger memory is split into multiple small memories, total number of read cycles will be same but energy consumed per read cycle would be much lower.

Another technique in this category is body-biasing memories. In this method designers reverse-bias a memory when it is not in use, which essentially raises the threshold voltage and in turn slows leakage.

Another method gaining popularity is to use multimode power for memories. In this technique, designers employ memory with several power modes. Many designs employ dual-function memories so that, when the CPU accesses a memory to read or write data to run a main application, the memory receives full access to power in order to perform the operation. However, when the memory is not required to read or write designers can program the memory to power down to a level at which the memory gets only enough power to retain its memory content.

Yet another method of the package level is to use stacked memory where the memory is stacked on top of die. Stacking memory significantly lowers interconnect capacitance, and can cut memory power consumption by as much as 30%. At minimum, Performance-critical things that need a lot of memory bandwidth and activity like graphics, multimedia, and modems can be placed in the stacked memory, while the operating system and other applications can be in the external memory.

5.6 Register Transfer Level (RTL) Power Reduction

At least 80% of the power in a large ASIC is committed by the time RTL is finished. Backend flow is not a magic solution to all power problems. It is a systematic method that detects opportunities either directly from RTL or from the mapping results to save power. It cannot fix a broken design – it cannot close your critical path if architecture is wrong. Backend flow cannot fix micro-architecture. Micro-architecture as well as RTL coding style has significant effect on dynamic and static power dissipation.

Thus, effective methodologies require that any power related issues be addressed before synthesis during RTL itself.

5.6.1 State Machine Encoding and Decomposition

Among the various state machine encoding styles, grey encoding seems one of the best encoding style suited for low power designs.

Figure 5.12 compares binary encoded state machine versus grey encoding. Notice that for a binary encoding, there may be more than one flop that can toggle at a time during a state transition for example from State D (“011”) to State E (“100”)

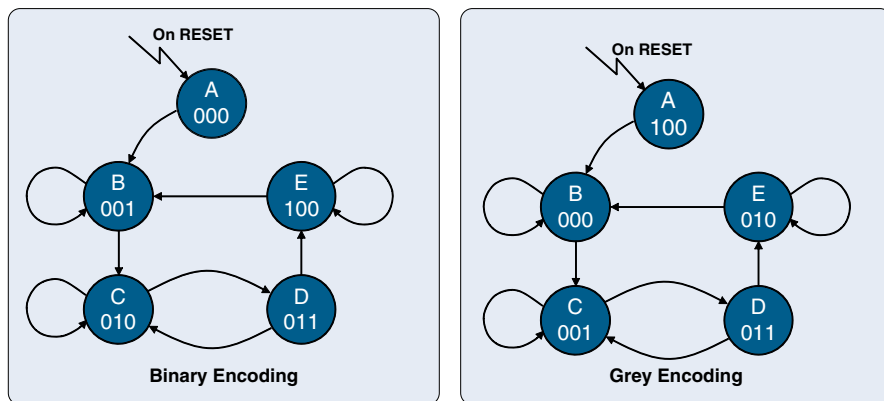


Fig. 5.12 Binary versus grey encoding for low power design



Fig. 5.13 Two's compliment and signed magnitude representation for a binary number

consuming more power than a corresponding implementation in Grey where only one flop toggles during a state transition.

Also note that State machine encoding in Grey eliminates any glitches/hazards on combinational equations that depend on the state.

If for some reason, user chooses a rather different encoding style, there are still opportunities to further reduce the power consumption during state transitions by assigning most frequent transitions to have minimum toggling of the flops. For example, for a 16-state machine, there are 256 possible transitions, typically only a few are actually allowed, and usually some of the allowed transactions occur more often during the circuit operation. So, if there are 30% of the transactions in the state machine from state "0101" to state "1010", it will cause all four state registers to toggle, and also presumably causes many transitions in the combinational logic. If the encoding of "1010" is changed to "0100", then only one state bit toggles in the transition. This reduces the register power for the module by 10%, and can reduce the combinational logic power even more.

Another idea is decomposition of finite state machines for low power that has been proposed in [27]. The basic idea is to decompose the State Transition Graph (STG) of a finite state machine (FSM) into two STGs that jointly produce the equivalent input-output behavior as the original machine. Power is saved because, except for transitions between the two sub-FSMs, only one of the sub-FSMs needs to be clocked.

The technique follows a standard decomposition structure. The states are partitioned by searching for a small subset of states with high probability of transitions among these states and a low probability of transitions to and from other states. This subset of states will then constitute a small sub-FSM that is active most of the time. When the small sub-FSM is active, the other larger sub-FSM can be disabled.

Consequently, power is saved because most of the time only the smaller, more power efficient, sub-FSM is clocked.

5.6.2 Binary Number Representation

For most of the applications, binary number representation in 2's complement is usually preferred over signed magnitude, with the former being more commonly used. However, for some very specific applications signed digit shows advantages in switching.

Figure 5.13 shows a 2's compliment and Signed Magnitude for a number "0" and "1".

Block A :

```

always @(posedge clock or negedge reset_b)
  if (!reset_b)
    test_ff <= 32'b0;
  else
    test_ff <= test_nxt;

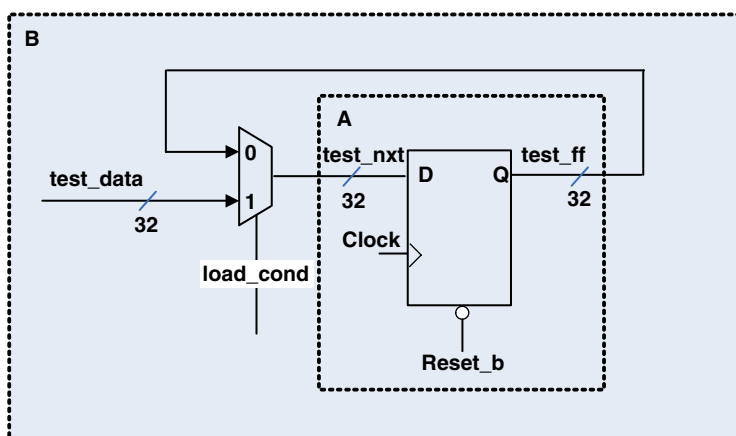
```

Block B :

```

assign test_nxt = load_cond ? test_data : test_ff;

```

Fig. 5.14 RTL code for test logic**Fig. 5.15** Logic implementation for test logic (bad example)

For an application that uses some sort of integrator that does nothing more than summing up values each clock cycle, a 2's complement representation going from “0” to “-1” will result in switching of the entire bit range (thus higher switching power) as compared to signed digit where only two bits will switch when going from “0” to “-1”.

5.6.3 Basic Gated Clock

Clock gating has been discussed in Sect. 2.5 but let's take it again here with respect to RTL coding to infer clock gating.

Let's consider a 32 bit Register “*test_ff*” that loads a 32 bit input data “*test_data*” when a load condition “*load_cond*” goes true else the register retain its old value. Figure 5.14 shows the RTL code for this logic and Fig. 5.15 shows the corresponding implementation.

Block B :

```
always @(posedge clock or negedge reset_b)
  if (!reset_b)
    test_ff <= 32'b0;
  else if (load_cond)
    test_ff <= test_data;
```

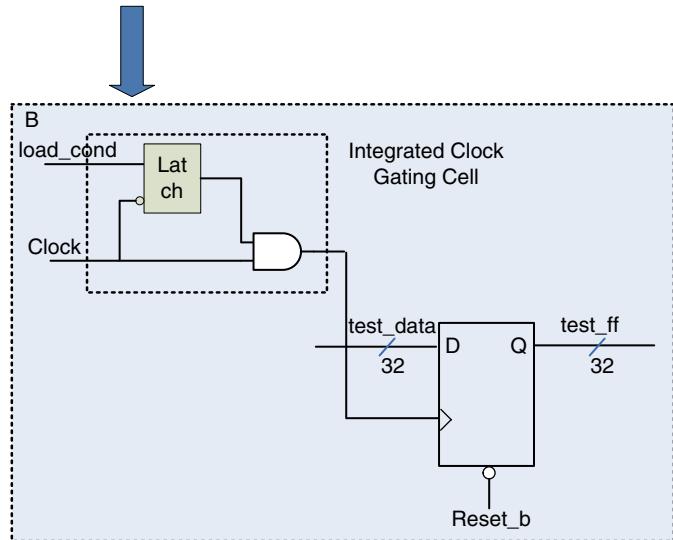


Fig. 5.16 Logic implementation for test logic (good example)

Note that based on RTL, there is no clock gating inferred on the “clock”. Some backend tools may be able to gate over hierarchy or after pre-flattening, but one should not rely on that.

Figure 5.16 shows a good example of same logic coded in a different style so as to automatically infer gating cell on the clock.

With the way RTL is coded, HDL compiler can now see the full picture in one module and detect that “load_cond” as a shared enable for 32 register bits. Also the backend environment setup will add a gated clock (integrated library cell) replacing 32 MUX gates. The integrated clock cell usually has bypass for scan mode (not shown in the Figure).

Some conditions unseen by synthesis can be used for explicit clock gating to dynamically stop the clock to a full function.

Similar to Clock Gating, techniques like Signal gating (described in Sect. 2.5.3) and data path re-ordering (described in Sect. 2.5.4) that should be considered while writing RTL so as to achieve further power reduction.

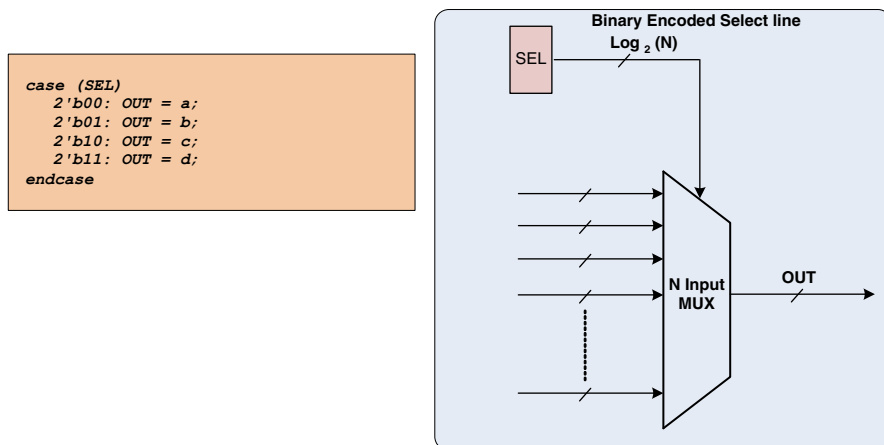


Fig. 5.17 Binary encoding of MUX select line

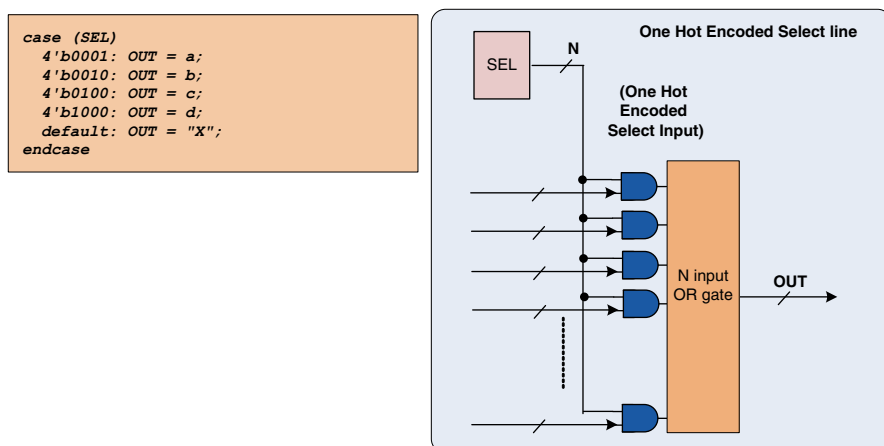


Fig. 5.18 One hot encoding of MUX select line

5.6.4 One Hot Encoded Multiplexer

There are many ways to infer multiplexers in RTL. “Case” statements, “if” statements and state machines are all common sources. Most common way to represent a multiplexer (MUX) is using binary encoding as shown in Fig. 5.17.

Note that if each input to the MUX is a multi-bit bus, it can have significant toggling and thus power consumption.

Instead of binary encoding if the “case” condition for the MUX is coded in “one hot” encoding style as shown in Fig. 5.18, it would have less stabilization effects, faster outputs and the changes of non-selected bus are masked early thus keeping the implementation low power.

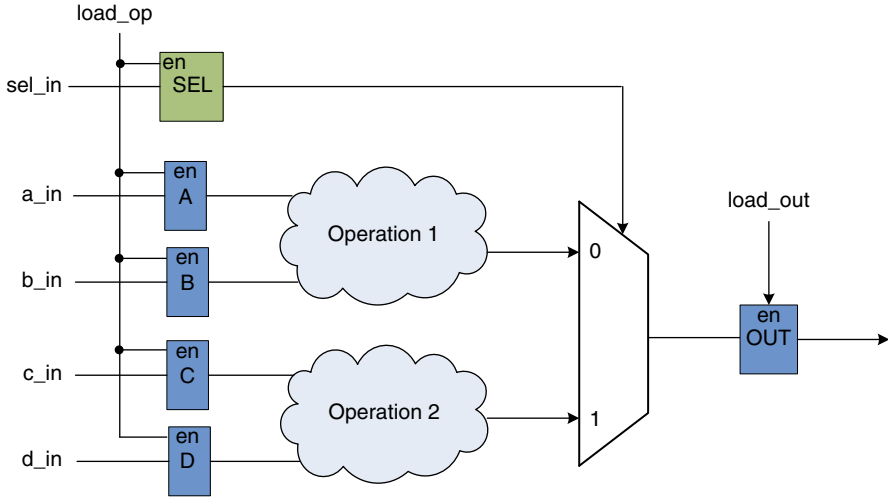


Fig. 5.19 Redundant transitions that consume power

Majority of digital logic for a design can consist of multiplexers and thus avoiding or masking false transitions can significantly drop power consumption.

5.6.5 Removing Redundant Transactions

One may often notice that data on the bus may keep changing from one value to another since there may be no default state. All these redundant transactions get functionally lost and may burn significant power so it is recommended to avoid data toggle where the data is not actually sampled so as to reduce power consumption.

Figure 5.19 shows a redundant transition example where all the operands (“a_in”, “b_in”, “c_in” and “d_in”) are loaded and consume power but all outputs are not used.

Note that “load_op” should not be asserted if it is not going to be followed by “load_out” assertion to save power.

Figure 5.20 shows modified logic to suppress the redundant transitions. “A” and “B” are loaded only when “SEL” is “0” while “C” and “D” are loaded only when “SEL” is “1”.

Figure 5.21 shows another example where same “input_data” goes to all the destination data buses where “data_sel” indicates valid data. Here only one destination samples data, but bus toggles on all four branches, burning un-necessary power.

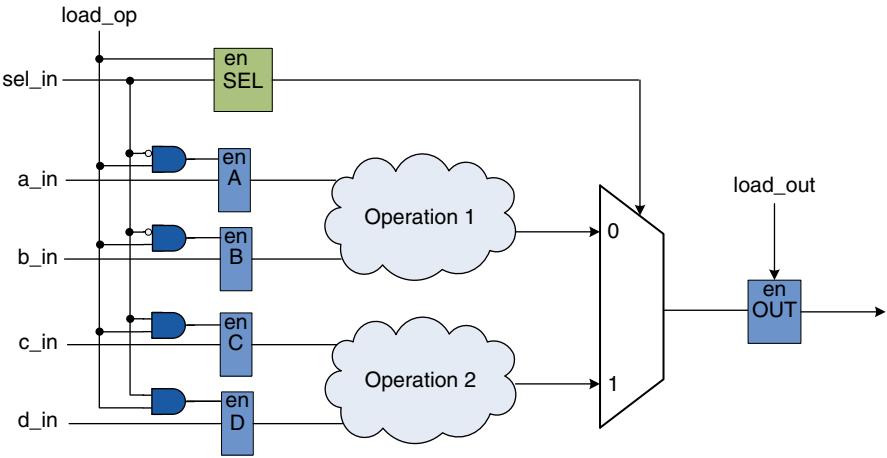


Fig. 5.20 Redundant transitions suppressed to save power

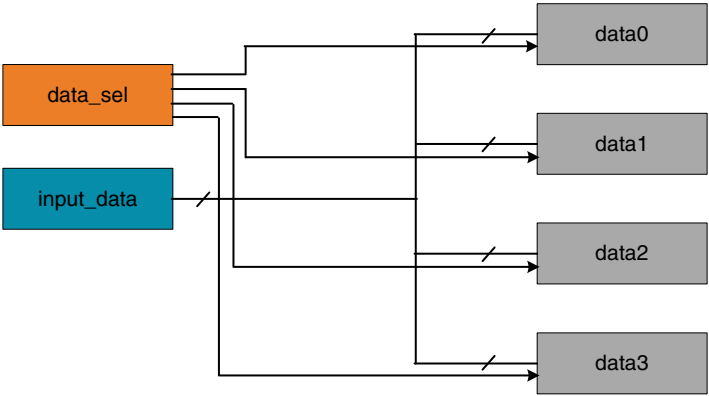


Fig. 5.21 Redundant transitions during point to multi-point bus

Figure 5.22 shows the same logic modified so as to suppress the redundant transitions. There is some power spend by the additional gates in negating a de-selected bus but only destination that samples the data eventually toggles thus saving power.

5.6.6 Resource Sharing

For the design that involves lot of mathematics, care must be taken to avoid any duplication of arithmetic operations where the same operand is used in multiple places. Figure 5.23 shows an example where no resource is shared.

Note that duplicate logic would increase additional area and consume more power.

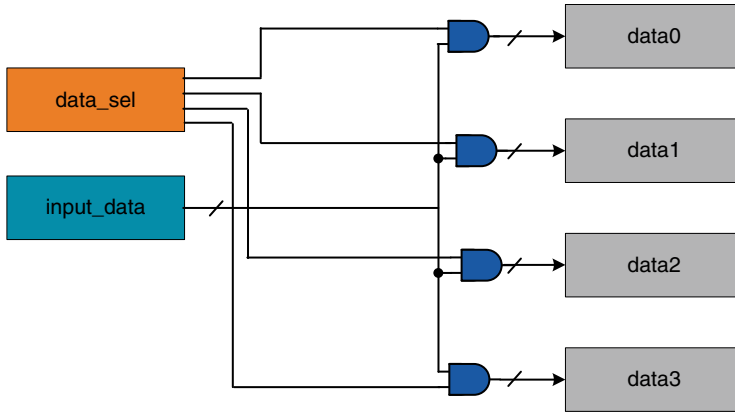


Fig. 5.22 Suppress redundant transactions in point to multi-point bus

```

always@(*)
case (SEL)
  3'b000: OUT = 1'b0;
  3'b001: OUT = 1'b1;
  3'b010: OUT = (value1 == value2);
  3'b011: OUT = (value1 != value2);
  3'b100: OUT = (value1 >= value2);
  3'b101: OUT = (value1 <= value2);
  3'b110: OUT = (value1 < value2);
  3'b111: OUT = (value1 > value2);
endcase

```

Fig. 5.23 No resource sharing in the logic

```

assign cmp_equal = (value1 == value2);
assign cmp_greater = (value1 > value2);

always@(*)
case (SEL)
  3'b000: OUT = 1'b0;
  3'b001: OUT = 1'b1;
  3'b010: OUT = cmp_equal; // ==
  3'b011: OUT = !cmp_equal; // !=
  3'b100: OUT = (cmp_equal || cmp_greater); // >=
  3'b101: OUT = !cmp_greater; // <=
  3'b110: OUT = !cmp_equal && !cmp_greater; // <
  3'b111: OUT = cmp_greater; // >
endcase

```

Fig. 5.24 Logic with resource sharing

Figure 5.24 shows the same logic modified so as to cover all the case conditions with just one comparator (“==”) and one arithmetic comparator (“>”) with each other pair of condition that is complementary.

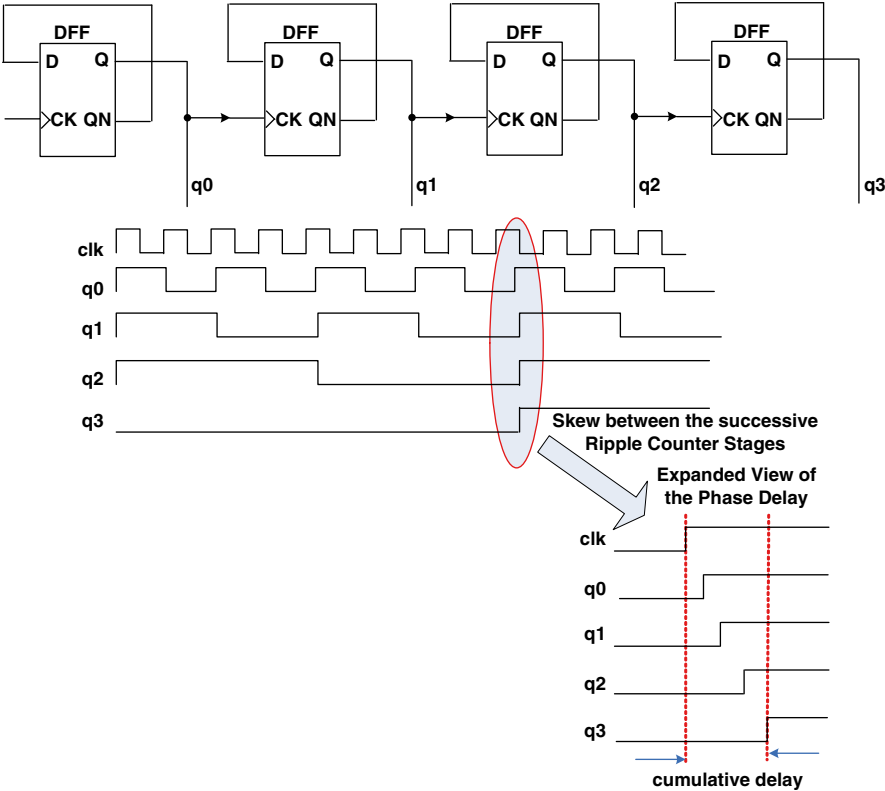


Fig. 5.25 Phase delay in a four stage ripple counter

5.6.7 Using Ripple Counters for Low Power

Ripple counters have been briefly discussed in Sect. 2.4.3 with their usage discouraged and limited but they can be very handy when it comes to low power designs. This section would discuss the challenges with ripple counter along with possible work-arounds to make their use more practical for low power designs.

Let’s consider the figure described in Sect. 2.2.1 again but with a four bit equivalent counter along with detailed skew information as shown in Fig. 5.25.

Each stage divides the frequency by two. It is called a ripple counter because the clock ‘ripples’ through the system, from flip-flop to flip-flop. The clock gets delayed by the propagation delay in each flip-flop, so the flip-flops for more significant bits change later than those for less significant bits.

Note that the counter contains incorrect values (due to glitches) while the clock is rippling. This effect is biggest when the most significant bit (MSB) changes.